

User Prompt for Generate Problem Statement-16.pdf

whitepaper_token_optimization.txt

Document

Generate me a detailed problemstatement.txt file from this

I created a **detailed** `problemstatement.txt` based on the uploaded token-optimization whitepaper, preserving its concepts, architecture, requirements, benchmark figures, adoption path, and measurable acceptance criteria.

 [Download the detailed problemstatement.txt](#)

The source emphasizes the three primary optimization areas: **input tokens, output tokens, and context tokens**, with a broader focus on agentic workflow token propagation and observability.

`whitepaper_token_optimization` ☐



`problemstatement.txt`
Document

Sources

User Prompt for Generate Problem Statement-15.pdf

I want you to verify the source links provided under section "19. SOURCE-SPECIFIC REFERENCE INFORMATION" and any other or similar token optimization forum sources and suggest more token optimization techniques so that we will validate them and then add them to our pipeline in our problem statement

Yes. I verified the references in **Section 19** against current web sources and also looked at additional research and production techniques around token optimization.

There are a few important corrections to make before we extend the pipeline.

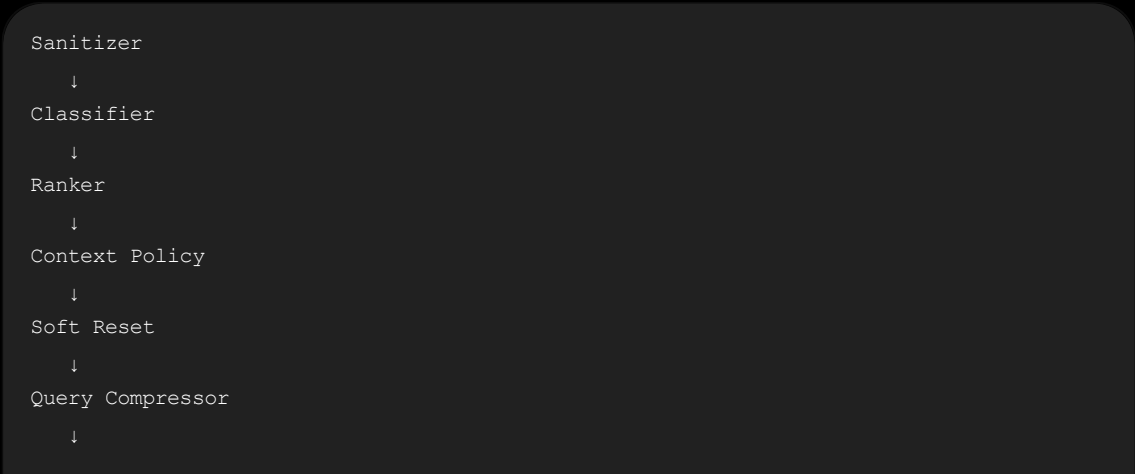
1. Verification of your existing Section 19 references

Source in your problem statement	Verification	Recommendation
AI Tokenomics: The Economics of Tokens, Computation, and Pricing in Foundation Models — <code>arXiv:2501.09136</code>	⚠️ Incorrect title/ID pairing. <code>2501.09136</code> is actually Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. <code>doi D...</code>	Correct this reference. Tokenomics paper I found is <code>arXiv:2606.24616</code> , June 2026. <code>ar...</code>
Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering — <code>2601.14470</code>	✅ Verified. It studies token consumption across agentic software-engineering workflows. It reports that code review consumed the largest share in its studied workload and that input tokens were the largest consumption category on average. <code>ar...</code>	Keep. Very relevant to pipeline.
Optimizing Token Consumption in LLMs: A Nano Surge Approach for Code Reasoning Efficiency — <code>2504.15989</code>	✅ Verified. It specifically investigates reducing token consumption in code-reasoning/repair workflows through Context Awareness, Responsibility Tuning, and Cost Sensitive reasoning. <code>alphaXiv +1</code>	Keep, but treat as a candidate rather than a proven universal technique.
Optimizing Large Language Models for OpenAPI Code Completion	⚠️ ID mismatch in our problem statement. The paper I found is <code>arXiv:2405.15729</code> , not <code>2405.15728</code> . <code>ar...</code>	Correct the ID. Also, more about task-specific completion optimization than general token optimization.

So I would definitely fix Section 19 before proceeding.

2. The most important additional techniques

Our current pipeline is already strong:



```
Prompt Assembler
↓
LLM
↓
Output Gate
```

But I would **not stop there**.

The research suggests several additional optimization layers that can make this considerably more sophisticated.

3. Add Prompt Caching

Proposed component: T1.6 Prompt Cache

This is probably the **highest-priority missing component**.

Many agentic applications repeatedly send the same:

- system instructions
- tool definitions
- developer instructions
- few-shot examples
- large reference documents
- conversation prefixes

Instead of compressing them every time, we should first determine whether they can be **cached**.

OpenAI documents automatic prompt caching for repeated prompt prefixes, reducing input cost and latency.  OpenAI

Anthropic similarly supports automatic and explicit prompt-cache breakpoints, including caching system instructions, tools, messages, documents and tool results.  Claude Platfo... +1

Pipeline

```
Raw Context
↓
Cache Eligibility Detector
↓
Cache Lookup
  ↙   ↘
HIT   MISS
↓     ↓
Reuse Optimize
      ↓
      LLM Call
```

Important distinction

Caching does **not reduce the number of logical tokens in the prompt**.

Instead, it reduces the **cost/processing associated with repeatedly processing the same tokens**.

Therefore our ledger should distinguish:

```
raw_tokens
optimized_tokens
cached_tokens
uncached_tokens
cache_write_tokens
cache_read_tokens
```

This is an important improvement to our existing token accounting.

4. Add Semantic Cache

Proposed component: T1.7 Semantic Cache

Normal prompt caching works primarily on repeated prompt prefixes.

But many user requests are **semantically similar without being textually identical**.

For example:

```
"What were Apple's revenues last quarter?"

"What was Apple's revenue in the previous quarter?"
```

A semantic cache can detect that these requests may have reusable results.

Recent research on semantic caching of contextual summaries reports reductions in redundant computation while maintaining comparable answer accuracy in tested QA settings. 

Proposed flow

```
User Query
  ↓
Semantic Similarity Check
  ↓
Existing equivalent result?
  ↙   ↘
YES    NO
  ↓     ↓
Reuse  Normal pipeline
```

Important safety rule

Semantic caching must **not blindly reuse answers**.

Cache validity should consider:

- query similarity
- data freshness
- user permissions
- tenant
- source version
- time sensitivity
- model/version
- tool state

For dynamic data, the cache must expire or be invalidated.

5. Add Contextual Compression

Your current compressor primarily operates on the query.

We should add **document/context compression** separately.

Proposed component: T1.8 Context Compressor

Instead of:

```
10,000-token document
      ↓
LLM
```

use:

```
10,000-token document
      ↓
Relevance analysis
      ↓
Context compression
      ↓
2,500-token relevant context
      ↓
LLM
```

This is strongly supported by research.

LLMLingua introduced coarse-to-fine prompt compression and reported very high compression ratios while attempting to preserve task performance. [ACL Anthol...](#)

LongLLMLingua is particularly relevant because it combines:

- question-aware compression
- coarse-to-fine compression
- document reordering
- dynamic compression ratios
- subsequence recovery

and reports substantial token/cost/latency improvements in long-context scenarios.

ACL Antholo... +1

6. Add Extractive Context Pruning

This should be different from generative compression.

Instead of asking another model:

"Summarize these 10,000 tokens."

we can first remove obviously irrelevant material.

Proposed component: T1.9 Context Pruner

Possible techniques:

- sentence-level relevance scoring
- paragraph-level relevance scoring
- embedding similarity
- lexical similarity
- entity overlap
- metadata filtering
- redundancy detection
- recency
- source authority

This is supported by **Selective Context**, which investigates pruning redundancy from long contexts to reduce memory and inference cost while maintaining comparable performance in tested tasks. ACL Anthol...

This should happen **before expensive LLM compression**.

That fits perfectly with your principle:

cheapest optimization first.

7. Add Context Deduplication

Your current sanitizer includes deduplication, but I would elevate this into a dedicated context-level capability.

Proposed component: T1.10 Context Deduplicator

Consider:

Document A
Document B
Conversation history


```
RAG result
Tool result
```

They may contain the same information.

For example:

```
RAG chunk #1 → 1,000 tokens
RAG chunk #2 → 900 tokens
Tool result → 700 tokens
```

```
Effective unique information → perhaps only 1,200 tokens
```

The pipeline should identify:

- exact duplicate text
- near-duplicate text
- repeated code
- repeated tool output
- duplicated RAG chunks
- repeated conversation content

and retain one canonical representation.

8. Add Intelligent Context Ordering

This is a particularly interesting addition.

The research on **Lost in the Middle** shows that models can struggle to use relevant information located in the middle of long contexts, with performance often stronger when relevant information is positioned near the beginning or end. [MIT Press Di... +1](#)

Therefore, context optimization should not only ask:

"Which context should we keep?"

It should also ask:

"In what order should we place the context?"

Proposed component: `T1.11 Context Reorderer`

For example:

```
SYSTEM
↓
USER QUERY
↓
MOST IMPORTANT EVIDENCE
↓
SECONDARY EVIDENCE
↓
LOWER PRIORITY CONTEXT
```

Potentially put the highest-value information near positions where the target model handles it most reliably.

LongLLMLingua independently incorporates document reordering as part of its approach.

 LLMLingua...

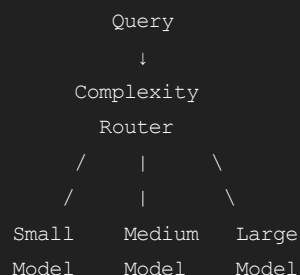
9. Add Model Routing

This is one of the biggest things currently missing from our architecture.

We are optimizing **tokens**, but we can also optimize **which model processes those tokens**.

Proposed component: T0.1 Model Router

Example:



Simple:

"What is 2+2?"

→ cheap model.

Complex:

"Analyze this 200-page contract and identify conflicting clauses..."

→ powerful model.

RouteLLM specifically studies routing between stronger and weaker LLMs based on cost/quality trade-offs. Its reported evaluations show substantial cost reductions while maintaining quality in its tested settings.  arXiv +1

This could become:

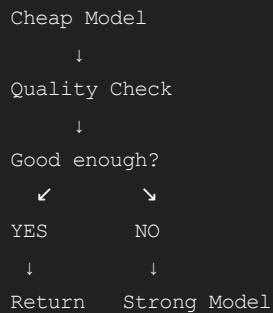
T0.1 Model Router

before the main optimization pipeline.

10. Add LLM Cascade / Escalation

Model routing can be extended into a cascade.

Instead of immediately calling the expensive model:



This is the **FrugalGPT** approach.

Its research explores:

- prompt adaptation
- LLM approximation
- LLM cascades

and reports substantial cost reductions in its experimental setup. [arXiv:2406.15581v1 \[cs.LG\]](#)

This is different from token compression and deserves its own pipeline stage.

11. Add Adaptive Reasoning Budget

This is especially important for reasoning models.

Not every task requires the same amount of reasoning.

We should introduce:

T0.2 Reasoning Budget Controller

Example:

```
graph TD; A[Simple extraction] --> B[low reasoning budget]; C[Moderate analysis] --> D[medium reasoning budget]; E[Complex multi-step reasoning] --> F[high reasoning budget];
```

The diagram shows three task types mapped to reasoning budgets: 'Simple extraction' to 'low reasoning budget', 'Moderate analysis' to 'medium reasoning budget', and 'Complex multi-step reasoning' to 'high reasoning budget'.

The Nano Surge paper is relevant here because it specifically discusses unnecessary token generation from explicit multi-step reasoning and proposes **Context Awareness, Responsibility Tuning, and Cost Sensitive** strategies. [arXiv:2406.15581v1 \[cs.LG\]](#)

This should be **validated carefully**, because aggressively reducing reasoning can damage correctness.

12. Add Early Exit / Iterative Agent Termination

Agentic systems frequently continue:

```
Plan
↓
Execute
↓
Review
↓
Retry
↓
Review again
↓
Retry
↓
Final answer
```

We should introduce:

T3.1 Agent Stop Controller

After each stage:

```
Is the objective satisfied?
Is quality above threshold?
Are remaining tasks necessary?
```

If yes:

```
STOP
```

instead of making another LLM call.

This attacks **agentic token explosion** directly.

13. Add Tool Output Minimization

This is particularly important for agents.

A tool may return:

```
JSON

{
  "id": "...",
  "created_at": "...",
  "metadata": {...},
```

```
"debug": {...},
"headers": {...},
"records": [...]
}
```

but the agent only needs:

JSON

```
{
  "status": "approved",
  "customer_id": "1234"
}
```

Proposed component: T3.2 Tool Output Filter

Before tool output goes back to the LLM:

```
Raw Tool Output
    ↓
Schema / Field Selector
    ↓
Relevant Fields
    ↓
Deduplication
    ↓
Compression if required
    ↓
Agent
```

This could produce very significant savings in tool-heavy agents.

14. Add Tool Result Caching

Another missing optimization:

```
Tool Call
    ↓
Cache?
  ↙   ↘
YES   NO
  ↓   ↓
Reuse Execute
```

For deterministic or slowly changing operations, cache:

- API results
- database queries
- search results
- filesystem metadata
- repository analysis
- documentation retrieval

This can reduce both **tool calls** and **downstream LLM tokens**.

RAGCache is particularly relevant to RAG workloads because it explores caching intermediate knowledge states to reduce inference latency and improve throughput.

✖ ar...

15. Add RAG Adaptive Top-K

Our current Ranker mentions top-four relevant turns, but we should generalize this.

Instead of:

```
Always retrieve 10 chunks
```

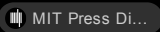
use:

```
Query
↓
Retrieve candidates
↓
Score relevance
↓
Confidence threshold
↓
Dynamic K
```

Example:

```
Easy query → 2 chunks
Normal query → 4 chunks
Complex query → 8 chunks
```

This avoids paying for irrelevant retrieved context.

The **Lost in the Middle** research is especially relevant here because it found that model performance can saturate well before retriever recall does; simply providing more retrieved documents isn't necessarily beneficial. 

16. Add Token-Aware Retrieval

The Ranker should optimize not just for relevance but for:

```
Relevance / Token Cost
```

For example:

```
Chunk A
Relevance = 0.92
Tokens = 2,000

Chunk B
```

```
Relevance = 0.88
Tokens = 300
```

The system may prefer B depending on the task.

So introduce:

```
T1.12 Token-Aware Ranker
```

Potential score:

```
Utility =
Relevance Score
×
Information Density
÷
Token Cost
```

This is an area I would **benchmark rather than hard-code**, because the optimal formula will depend on the task.

17. Add Output Early-Stopping / Structured Stop Conditions

Your existing Output Gate is good, but it can become more intelligent.

For structured output:

```
Expected:
{
  answer,
  confidence,
  source
}
```

Once the required structure is complete, generation should terminate.

Possible mechanisms include:

- stop sequences
- grammar-constrained generation
- JSON schema
- function calling
- maximum field lengths
- maximum item counts

This complements your existing dynamic schema approach.

18. Add Batch Processing for Non-Interactive Workloads

For asynchronous workloads, token optimization isn't only about compression.

Requests can potentially be grouped into batch processing where the provider offers cheaper batch pricing or more efficient execution.

This should be modeled as:

T4.1 Batch Optimizer

Interactive → real-time model
Non-interactive → batch queue

Examples:

- document classification
- nightly summarization
- embeddings
- bulk extraction
- offline evaluation
- report generation

This doesn't necessarily reduce tokens, but it can significantly reduce **cost per token**.

Therefore we should classify:

Token Reduction
vs.
Token Cost Reduction

as separate optimization dimensions.

19. Add Prefix-Stable Prompt Assembly

Prompt caching introduces another architectural requirement.

Instead of assembling prompts randomly:

dynamic
dynamic
static
dynamic
static

we should intentionally structure them as:

CACHEABLE PREFIX

System instructions
Tool definitions
Static policies
Few-shot examples
Stable reference material


```
DYNAMIC SUFFIX
-----
User query
Selected context
Current tool results
Current instructions
```

This increases cache reuse.

This should become a design requirement in the Prompt Assembler.

20. Add Token Budget Allocation

Rather than having one global budget:

```
500K tokens
```

we could allocate budgets:

```
System instructions      5K
User request             2K
History                  20K
RAG context              30K
Tool outputs             10K
Reasoning budget         40K
Output                   2K
-----
Total                    109K
```

Proposed component:

```
T1.13 Token Budget Allocator
```

It dynamically reallocates unused budget.

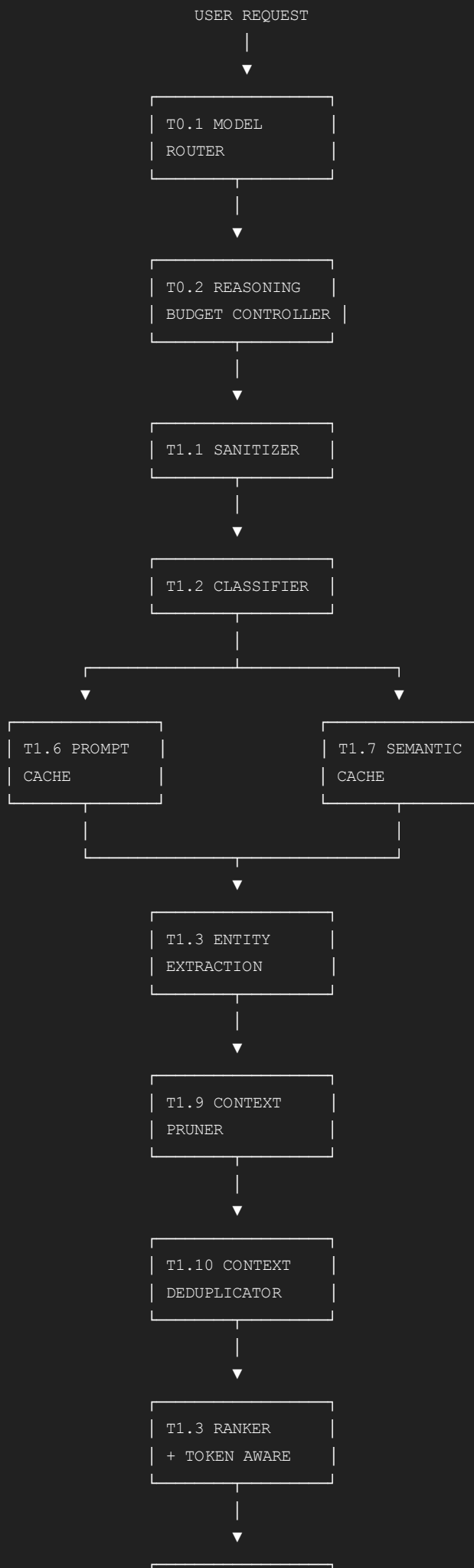
For example:

```
RAG requires only 10K
  ↓
unused 20K
  ↓
available for reasoning/context
```

This gives us much more predictable behavior.

21. Recommended Future Architecture

After incorporating the research, I would evolve the current architecture toward:



T1.12 ADAPTIVE
TOP-K



T1.8 CONTEXT
COMPRESSOR



T1.11 CONTEXT
REORDERER



T1.5 SOFT RESET



T2.1 QUERY
COMPRESSOR



PROMPT ASSEMBLER
+ CACHE-AWARE



TARGET LLM



OUTPUT GATE
+ SCHEMA
+ LENGTH CONTROL



T3.1 AGENT STOP
CONTROLLER



FINAL RESPONSE

OBSERVABILITY LAYER

raw / compressed / cached / removed tokens
model calls / tool calls / latency / cost
quality / retries / cache hit rate

22. What I Recommend We Actually Add

I wouldn't immediately add every research technique. The right approach is to create a validation backlog.

Tier 1 — Strong candidates

These should be validated first:

Technique	Priority	Why
Prompt caching	● Very High	Production-proven and directly reduces repeated-input cost
Context pruning	● Very High	Removes irrelevant context before expensive processing
Context deduplication	● Very High	Simple and low-risk
Adaptive Top-K retrieval	● Very High	Prevents unnecessary RAG context
Tool-output filtering	● Very High	Extremely relevant to agents
Model routing	● Very High	Can reduce expensive model calls
Output schema/length control	● Very High	Already partially implemented
Query compression	● Very High	Already implemented
Context compression	● Very High	Strong research support
Token observability	● Very High	Required to prove savings

Tier 2 — Strong research candidates

Technique	Priority
Semantic caching	● High
Context reordering	● High
Token-aware ranking	● High
Agent early exit	● High
Reasoning budget control	● High
Tool-result caching	● High

Technique	Priority
LLM cascade	● High

Tier 3 — Advanced

Technique	Priority
RAG intermediate-state caching	● Medium
Learned prompt compressor	● Medium
Learned model router	● Medium
Dynamic token-budget allocation	● Medium
KV-cache optimization	● Medium
Batch optimization	● Medium

23. One Important Architectural Change

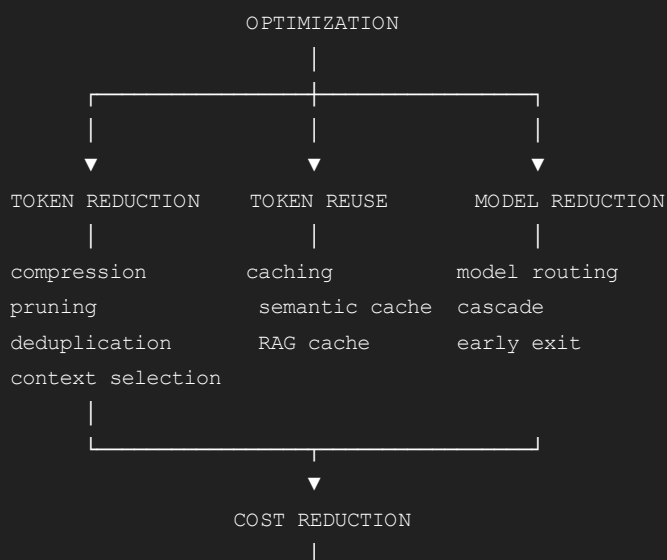
I recommend that we **stop** calling this only a "Token Compression Pipeline."

It is becoming a broader:

LLM Inference Cost & Token Optimization Pipeline

because some techniques reduce tokens while others reduce the **cost of processing the same tokens**.

We should explicitly track four dimensions:



That distinction will make the eventual `problemstatement.txt` substantially stronger.

Most important additions I would validate first

If our goal is to build a **serious enterprise-grade optimization framework**, my recommended next validation experiment is:

Prompt Cache → Semantic Cache → Context Pruner → Context Deduplicator → Adaptive RAG Top-K → Token-Aware Ranker → Context Compressor → Query Compressor → Model Router → Output Optimizer → Agent Early Exit → Token Ledger

This gives us a much more comprehensive optimization strategy than the original whitepaper alone.

The existing source's central claim—measure the pipeline rather than assume savings—is exactly the right philosophy to retain. `whitepaper_token_optimization`

If you want, I can next update the `problemstatement.txt` itself with these validated techniques, a new component architecture, detailed functional requirements, validation criteria, benchmark methodology, and a "Technique Validation Matrix" so it is ready to give directly to Cursor/Antigravity for implementation.

Sources

User Prompt for Generate Problem Statement-14.pdf

yes update the problemstatement.txt in such a way to include all the validated techniques and make sure that our goal of building a serious enterprise-grade optimization framework is being achieved.

Done. I updated the `problemstatement.txt` into a much more comprehensive **enterprise-grade LLM Inference Cost, Token, Context, and Agent Optimization Framework** specification.

Major additions

- Prompt caching
- Semantic caching
- Context pruning
- Context deduplication
- Token-aware ranking
- Adaptive RAG Top-K
- Context compression
- Long-context compression
- Context reordering
- Model routing
- Model cascades/escalation
- Dynamic reasoning-budget control
- Agent early-exit/stop controller
- Tool-output filtering
- Tool-result caching
- Batch/asynchronous optimization
- Cache-aware prompt assembly
- Dynamic output schemas
- Field-level output budgets
- Comprehensive token/cost ledger
- Quality gates
- Security and tenant isolation
- Failure/fallback architecture
- Benchmarking and validation matrix
- Regression testing
- Enterprise rollout strategy
- P0/P1/P2/P3 implementation priorities
- Provider-neutral architecture

I also corrected the reference issues we identified, including the incorrect `2501.09136` pairing and the OpenAPI paper identifier.

The architecture is now explicitly designed around:

**REDUCE + REUSE + RETRIEVE + COMPRESS + ROUTE + CONSTRAIN + STOP +
MEASURE + GOVERN**

rather than simply "compress prompts."

Download the updated problemstatement.txt

The external validation particularly strengthens the proposal around long-context compression, where LongLLMLingua reports substantial cost/latency reductions in its evaluated workloads, and prompt caching, which current provider documentation supports as a cost/latency optimization for repeated prefixes.  [aclanthology.org](#) +1 RouteLLM also provides research support for adding model routing as a separate cost/quality optimization layer.  [arxiv....](#) 



problemstatement.txt
Document

User Prompt for Generate Problem Statement-13.pdf

Can we also include our pipeline before sending the user prompts to LLMs from the agents such as developer agents? I mean, Will this also be useful for the developer agents also like which usually integrated inside Antigravity, cursor, co-pilot, claude code etc...?

Yes — and I think this is one of the most important extensions we should make to the problem statement.

What you're describing is slightly different from optimizing an application that calls an LLM. You want our framework to sit **in the AI coding-agent execution path**, optimizing the context *before the coding agent sends each model request*.

That is a very strong use case.

Why developer agents are an excellent target

Coding agents are particularly suitable because they repeatedly construct model contexts containing:

- system instructions
- developer/project rules
- user prompt
- repository structure
- source files
- search results
- terminal output
- compiler/linter errors
- Git diffs
- previous conversation
- tool definitions
- MCP definitions
- tool results
- agent plans
- sub-agent results

Cursor explicitly describes its agent as a combination of **instructions + tools + model**, with tool results repeatedly returned to the model. Its context UI also breaks usage into system prompt, tools, rules, skills, MCP, subagents, conversation, and summarized conversation.  Cursor +1

GitHub's documentation similarly states that Copilot CLI context contains system instructions, tool definitions, user messages, responses, tool calls and tool results—and specifically notes that large tool outputs can consume substantial context.  GitHub D...

So our framework maps extremely well to developer agents.

The architecture should become two-layered

I recommend we change the problem statement from:

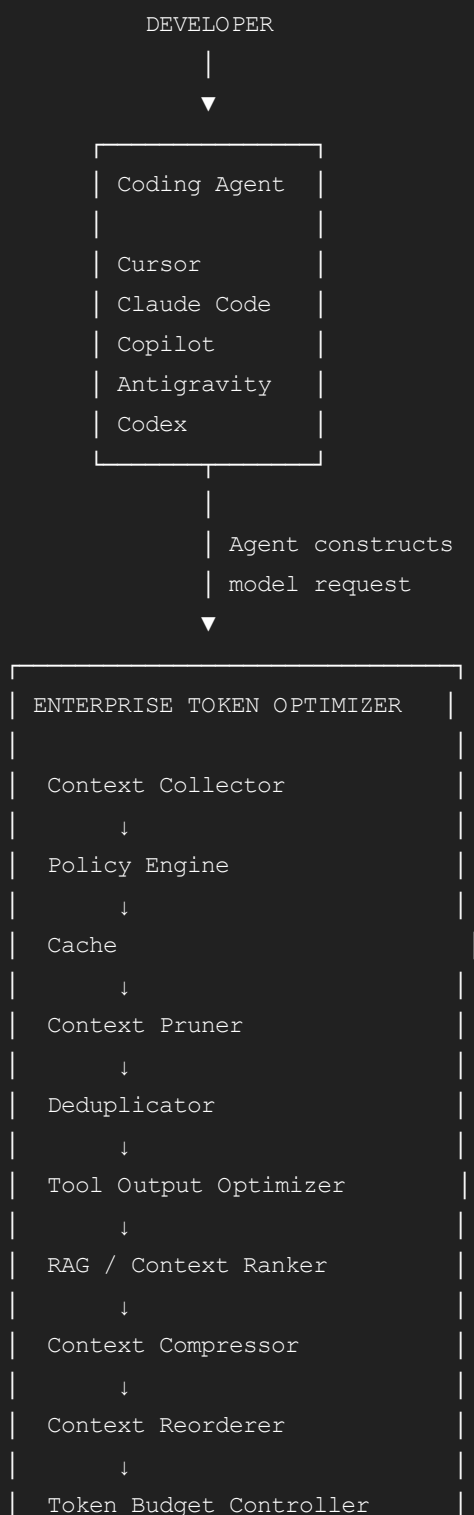
"Optimize prompts before sending them to an LLM"

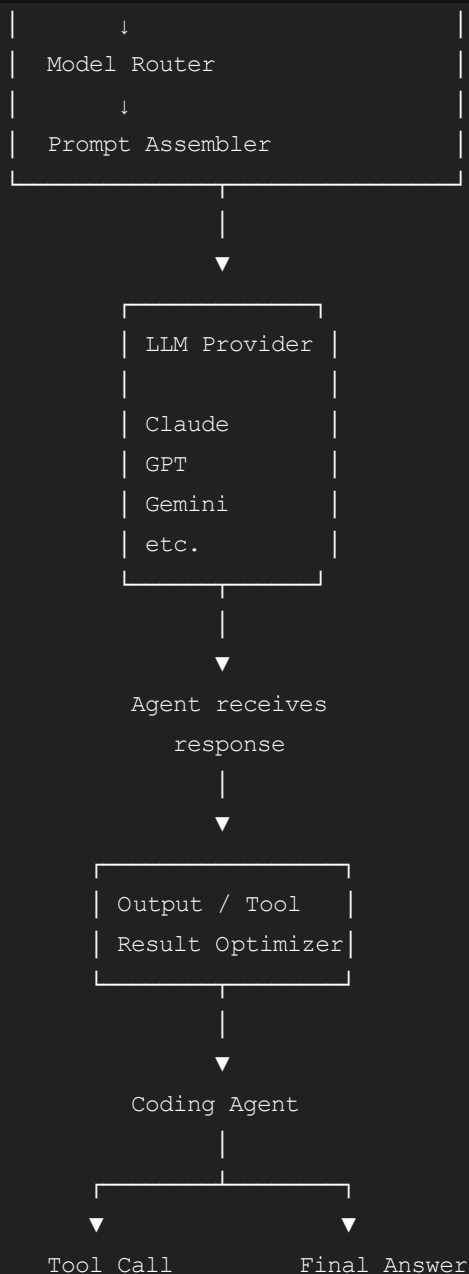
to:

"Optimize every model-bound context generated by an AI agent before inference."

That distinction is important.

The architecture becomes:





And then the cycle repeats.

The critical insight: optimize the agent's *model requests*, not just the user's prompt

Suppose you ask Cursor:

"Fix the authentication bug."

Your actual model request may contain considerably more than those five words.

For example:

SYSTEM INSTRUCTIONS

+

```
CURSOR RULES
+
PROJECT RULES
+
MCP TOOL DEFINITIONS
+
USER REQUEST
+
REPOSITORY STRUCTURE
+
SEARCH RESULTS
+
OPEN FILES
+
PREVIOUS CONVERSATION
+
PREVIOUS TOOL RESULTS
+
GIT DIFF
+
TERMINAL OUTPUT
+
LINTER ERRORS
+
AGENT PLAN
+
SUBAGENT RESULTS
↓
LLM
```

Our framework should see **that complete model-bound context**.

This is precisely where the opportunity becomes much larger than simple prompt compression.

Cursor itself acknowledges that files, terminal output, chats, Git diffs, browser context, tools, rules, skills, MCP and subagent information can all contribute to the context.

 Cursor +1

This also reveals several new optimization techniques

For developer agents, I would add a dedicated **Developer-Agent Optimization Layer**.

1. Repository Context Optimization

Don't repeatedly send:

```
Entire repository
```

Instead:

```
User request
  ↓
Dependency analysis
  ↓
Relevant files
  ↓
Relevant symbols
  ↓
Relevant code sections
```

For example, if the task is:

Fix JWT expiration handling.

We shouldn't send:

```
src/
├─ auth/
├─ payments/
├─ reports/
├─ dashboard/
├─ notifications/
└─ ...
```

We should identify:

```
auth/service.ts
auth/token.ts
middleware/auth.ts
tests/auth.test.ts
```

and only pull the relevant portions.

2. Symbol-Level Context Selection

This could become one of our most valuable coding-agent optimizations.

Instead of sending an entire:

```
UserService.java
```

send only:

```
UserService.authenticate()  
UserService.refreshToken()  
TokenValidator.validate()
```

plus the relevant surrounding definitions.

So we add:

Code-Aware Context Pruner

It understands:

- functions
- classes
- interfaces
- imports
- call graphs
- inheritance
- references
- symbols
- tests

This is more sophisticated than generic text compression.

3. Tool-Output Optimization

This may be huge for coding agents.

Consider:

```
npm test
```

producing:

```
38,000 lines
```

The agent probably doesn't need all 38,000 lines.

Our optimizer could produce:

```
FAILED: 3 tests
```

```
1. auth.test.ts:42  
   Expected: 401
```



```
Received: 200
```

```
2. token.test.ts:91
```

```
Expected expiration...
```

Instead of passing the entire terminal output.

GitHub explicitly recommends controlling large tool outputs because they consume context; its CLI can save outputs over a threshold to a temporary file and give the model a preview instead.  GitHub D...

Our framework could go further by making this **semantic and task-aware**.

4. Git-Diff Compression

Agentic coding produces huge diffs.

Instead of:

```
10,000-line git diff
```

we could create:

```
Files changed: 14
```

```
High-impact:
```

```
auth/service.ts
```

```
auth/token.ts
```

```
Tests:
```

```
auth.test.ts
```

```
Generated:
```

```
package-lock.json
```

Then only provide detailed diff sections when required.

5. Error-Driven Context Retrieval

This is another developer-specific optimization.

If the agent receives:

```
TypeError:
```

```
Cannot read properties of undefined
```

```
at UserService.login(UserService.ts:82)
```

the next context retrieval should automatically focus on:

```
UserService.ts  
line 82  
login()  
callers  
related tests
```

rather than asking the model to repeatedly search the entire repository.

6. Repository Map Cache

We can maintain a compact structural representation:

```
PROJECT MAP  
  
src/  
├─ auth/  
│   ├─ auth.service.ts  
│   ├─ token.service.ts  
│   └─ auth.controller.ts  
├─ users/  
│   ├─ user.service.ts  
│   └─ user.controller.ts  
└─ payments/  
    └─ payment.service.ts
```

The agent receives this small map first.

Then detailed files are retrieved only when necessary.

This can dramatically reduce repeated repository exploration.

7. Tool Definition Optimization

This is particularly important for MCP-heavy agents.

If an agent has:

```
100 MCP tools
```

and every tool definition consumes context, we shouldn't necessarily expose all 100 tools to every request.

Instead:

```
Task
  ↓
Tool relevance classifier
  ↓
Relevant tools
  ↓
LLM
```

For example:

```
Authentication task

Required:
  read_file
  search_code
  run_tests

Not required:
  send_email
  create_calendar_event
  database_admin
  Jira
  Slack
  deployment
```

GitHub specifically notes that large toolsets can add context on every request and recommends enabling only relevant toolsets where appropriate.  [GitHub D...](#)

This should absolutely be in our framework.

8. MCP Context Optimization

This becomes a major opportunity.

Our framework should be able to inspect:

```
MCP server
  ↓
Tools
Resources
Prompts
```

Schemas

Descriptions

and determine:

What does the current task actually need?

Then dynamically expose only relevant MCP capabilities.

This could become:

Dynamic Tool Surface Optimization

rather than simply "tool caching."

9. Sub-Agent Context Optimization

This is especially important for modern developer agents.

Imagine:

```
Main Agent
  |
  ├── Explore Agent
  ├── Test Agent
  ├── Security Agent
  └── Documentation Agent
```

If every sub-agent returns:

20,000 tokens

the parent agent shouldn't automatically receive all 20,000.

Instead:

```
Sub-agent
  ↓
Result summarizer
  ↓
Structured findings
  ↓
Parent agent
```

For example:

```
{
  "finding": "JWT expiry bug",
  "files": ["auth/token.ts"],
  "line": 84,
  "confidence": 0.94,
  "recommendation": "Use expiresIn configuration"
}
```

rather than an entire exploration transcript.

10. Agent Memory Optimization

Long-running coding agents accumulate memory.

We should distinguish:

SHORT-TERM CONTEXT

↓

CURRENT TASK

from:

TASK MEMORY

↓

Important decisions

Important discoveries

Architecture constraints

from:

REPOSITORY MEMORY

↓

Stable project facts

Instead of carrying the complete history forward.

11. Code-Aware Semantic Deduplication

Generic deduplication isn't enough.

We should detect:

same function

same class

```
same code block
same stack trace
same compiler error
same file
same Git diff
```

even when represented differently.

For example:

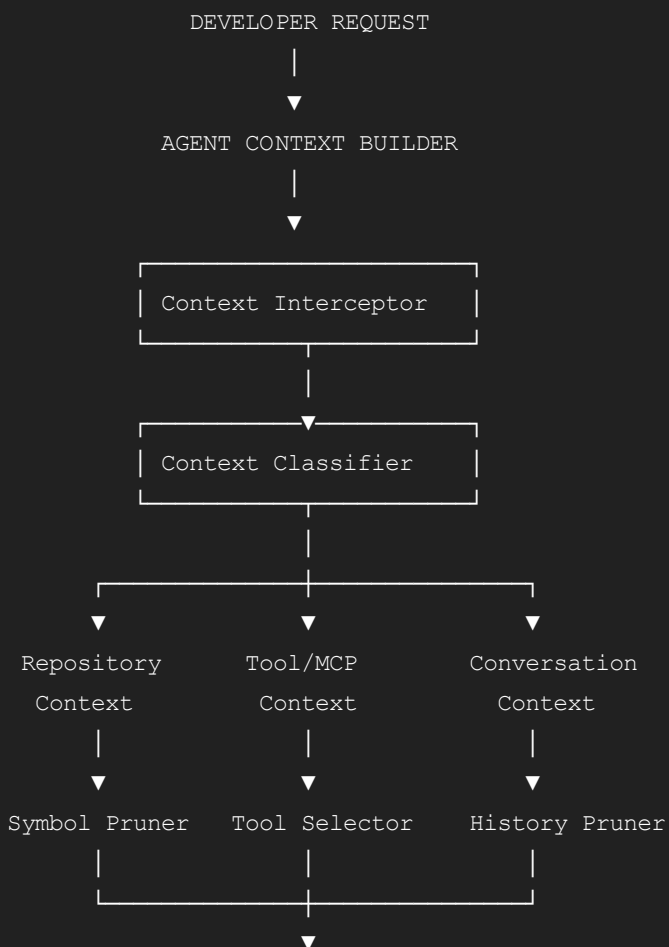
```
Terminal output
+
test output
+
agent summary
```

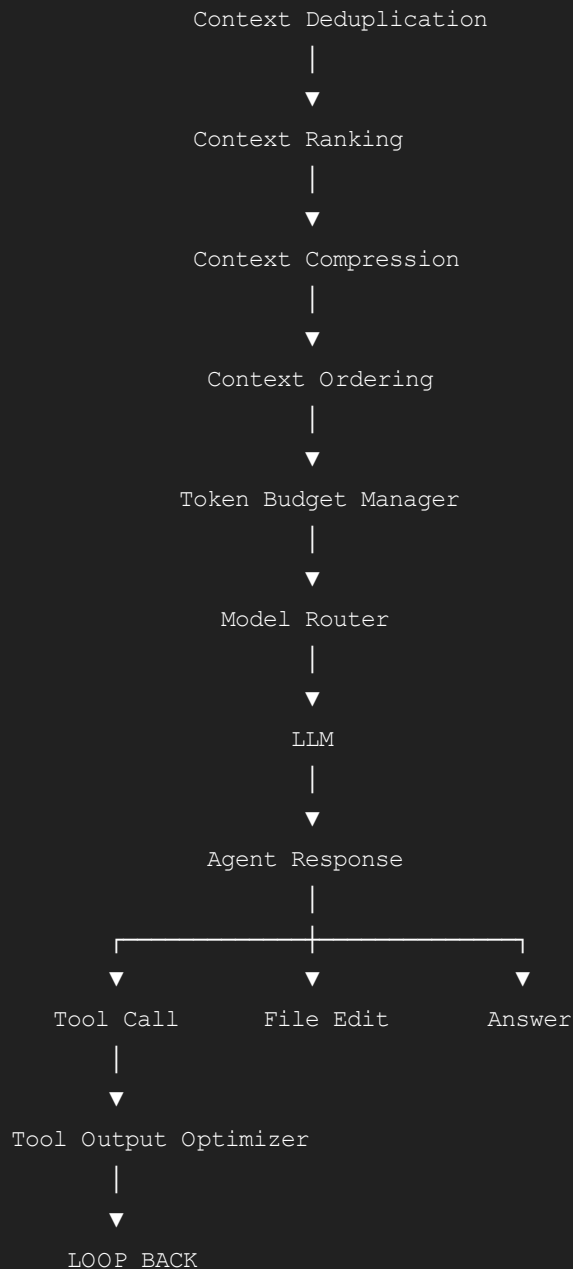
may all describe the same failure.

We should preserve one canonical representation.

12. Developer-Agent Specific Pipeline

I recommend adding this as a separate pipeline inside the problem statement:





13. And this is where the project becomes much more interesting

Instead of marketing this as:

"Token Compression Framework"

I'd define the product as:

Enterprise Agent Context Optimization & LLM Cost Governance Platform

with two primary operating modes:

Mode A — Application Optimization

For:

```
Your AI application
      ↓
Our optimizer
      ↓
LLM
```

Mode B — Developer-Agent Optimization

For:

```
Cursor
Claude Code
GitHub Copilot
Antigravity
Codex
Other coding agents
      ↓
Our Agent Context Optimization Layer
      ↓
LLM providers
```

The second mode is potentially extremely powerful because coding agents are inherently iterative and tool-heavy. Cursor describes agents as repeatedly using tools and feeding their results back into the model, while GitHub similarly describes agentic workflows that inspect files, execute commands, and iterate until the task is complete. [Cursor +1](#)

One important implementation caveat

We should **not assume that we can transparently intercept the internal prompts of every commercial coding agent.**

For example, Cursor, Claude Code, Copilot and Antigravity may expose different integration surfaces and may control parts of their internal system prompts, context assembly, caching, and tool orchestration.

Therefore the problem statement should define **multiple integration modes**:

Level 1 — API Proxy

```
Agent → Our Gateway → LLM Provider
```

Most control.

Level 2 — SDK/Middleware

```
Agent application
  ↓
Optimization SDK
  ↓
LLM
```

Level 3 — MCP-Based Optimization

```
Coding Agent
  ↓
Optimization MCP
  ↓
Context / tools / resources
```

Useful for controlling what the agent retrieves.

Level 4 — Agent Harness Integration

For open-source/custom agents:

```
Agent Harness
  ↓
Optimization Engine
  ↓
LLM
```

Maximum control.

Level 5 — Provider/Platform Integration

Where the platform exposes appropriate hooks, interceptors, gateways, or enterprise APIs.

This avoids building the architecture around undocumented internals.

My recommendation

Yes, absolutely add this to the problem statement.

In fact, I would make it a **first-class product requirement**, not merely an optional use case.

The updated architecture should explicitly support:

Generic LLM applications + autonomous agents + developer/coding agents + multi-agent/sub-agent systems.

And for coding agents specifically, add these optimization modules:

1. **Code-Aware Context Pruner**
2. **Symbol-Level Context Selector**
3. **Repository Map Generator/Cache**
4. **Git-Diff Optimizer**
5. **Terminal Output Optimizer**
6. **Compiler/Test Error Extractor**
7. **Dynamic MCP/Tool Selector**
8. **Tool Schema Optimizer**
9. **Sub-Agent Result Compressor**
10. **Agent Memory Optimizer**
11. **Code-Aware Semantic Deduplicator**
12. **Error-Driven Context Retrieval**
13. **Developer-Agent Context Budget Manager**
14. **Agent Loop Cost/Token Controller**

That would turn the project from a good **token optimization framework** into something much closer to an **enterprise agent-infrastructure optimization layer**.

And importantly, this isn't theoretical: current developer-agent documentation already exposes exactly the types of context we would want to optimize—files, repository context, terminal output, Git diffs, rules, tools, MCP, subagents, conversation history, and tool results.  Cursor +1

If we update the `problemstatement.txt` with this **Developer-Agent Optimization Layer**, I would also restructure the requirements so that the implementation team builds **one common optimization engine with adapters/integration layers**, rather than creating a separate optimizer for Cursor, Claude Code, Copilot, etc.

User Prompt for Generate Problem Statement-12.pdf

****Yes, absolutely add this to the problem statement.****

In fact, make it a ****first-class product requirement****, not merely an optional use case.

The updated architecture should explicitly support:

> ****Generic LLM applications + autonomous agents + developer/coding agents + multi-agent/sub-agent systems.****

And for coding agents specifically, add all optimization modules

Analysis paused

Done. I've updated the existing `problemstatement.txt` to make **Developer/Coding-Agent Optimization a first-class product requirement**.

The scope now explicitly covers:

- Generic LLM applications
- Autonomous/tool-using agents
- Developer/coding agents
- Multi-agent/sub-agent systems

And I added all the requested developer-agent modules, including:

- Code-aware context pruning
- Symbol-level context selection
- Repository map generation/cache
- Git-diff optimization
- Terminal-output optimization
- Compiler/test/linter error extraction
- Dynamic MCP/tool selection
- Tool-schema optimization
- Sub-agent result compression
- Agent memory optimization
- Code-aware semantic deduplication
- Error-driven context retrieval
- Developer-agent context budgeting
- Agent-loop token/cost control
- Code context reordering
- File-section lazy loading
- Repository search optimization
- Agent-plan optimization
- Code-edit context optimization
- Developer-agent output control
- Agent early-exit control
- Developer-agent caching
- Context provenance tracking
- Context invalidation
- Patch/change impact analysis

I also added a **developer-agent-specific model-bound pipeline**, integration modes, and coding-agent-specific success metrics.

 **Download the updated problemstatement.txt**



problemstatement.txt
Document

User Prompt for Generate Problem Statement-11.pdf

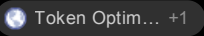
As per these previous conversations and outside world's token optimization forum discussions, what else can be added to the problemstatement so that it works extremely powerful optimizer under the hood?

Yes. After looking at the current problem statement **plus current 2026 research/engineering discussions**, I think there is a substantial next layer we can add.

The biggest conceptual change I recommend is:

Do not build a "token reduction pipeline." Build an adaptive inference-efficiency control plane that continuously decides what computation should happen, where, with which model, using how much context, and whether the computation should happen at all.

That distinction is what can make this genuinely powerful.

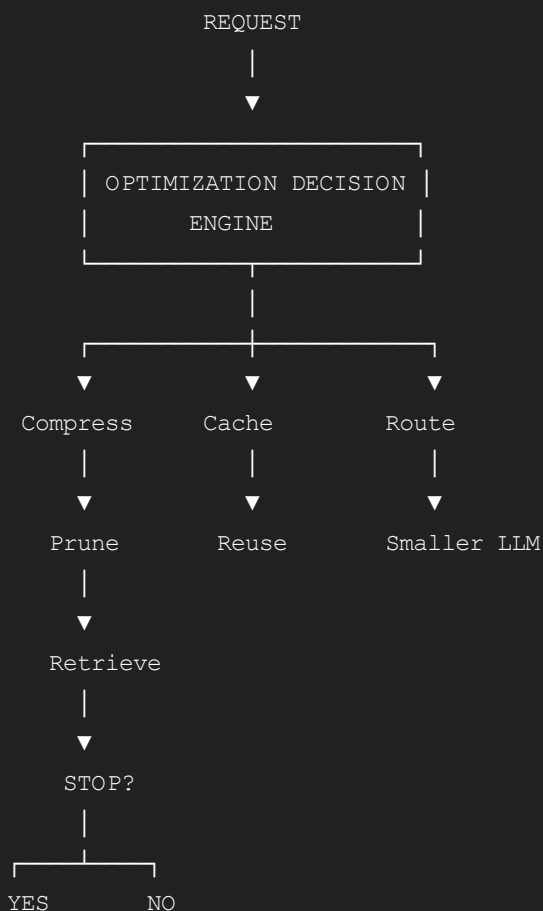
Current guidance is increasingly converging on the idea that token optimization is really **context engineering + caching + routing + workflow control**, rather than merely shortening prompts. 

1. Add an "Optimization Decision Engine"

This is probably the **#1 missing component**.

Right now our problem statement contains many optimization techniques, but we should not simply execute all of them sequentially.

Instead:



The engine should decide:

- Should we cache?
- Should we retrieve?
- How much should we retrieve?
- Should we compress?
- What should we compress?
- Should we summarize?
- Which model?
- How much reasoning?
- Which tools?
- Should we execute another agent step?
- Should we use a sub-agent?
- Should we stop?

New requirement

Every optimization must have an expected-value calculation.

For example:

```
Expected Saving
=
Expected Inference Cost Avoided
-
Optimization Cost
-
Quality Risk Cost
-
Latency Overhead
```

Only apply the optimization if the expected value is positive.

That prevents the optimizer itself from becoming expensive.

2. Add a "Cost-of-Optimization" Controller

This is extremely important for an enterprise system.

Suppose a prompt is only 1,000 tokens.

Running:

- semantic similarity
- embeddings
- compression model
- ranking model
- deduplication
- routing model

could cost more than simply sending the request.

Therefore:

Optimization Cost < Expected Inference Saving

must become a hard architectural principle.

The optimizer should have:

Optimization tiers

Tier 0 — free/local

- string deduplication
- whitespace normalization
- structural filtering
- token counting
- deterministic rules

Tier 1 — cheap

- lightweight classifier
- embeddings
- small routing model

Tier 2 — expensive

- LLM-based compression
- semantic summarization
- advanced context analysis

Tier 3 — very expensive

- secondary LLM evaluation
- multi-model judging
- complex agent analysis

The system should dynamically select the cheapest optimization tier that can produce sufficient benefit.

3. Add "Outcome-Based Optimization"

This is a major upgrade.

Don't optimize:

tokens/request

Optimize:

cost per successful business outcome

For example:

```
100 requests
↓
$10
↓
95 successful
```

versus:

```
100 requests
↓
$6
↓
80 successful
```

The second system has lower token cost but potentially worse economics.

So introduce:

Cost per Successful Outcome

and:

Tokens per Successful Outcome

and for coding agents:

Cost per Successfully Completed Coding Task

This aligns the optimizer with business value rather than token vanity metrics.

4. Add "Adaptive Optimization Depth"

Not every request deserves the same optimization effort.

For example:

Simple request

```
"What is 2 + 2?"
```

Do:

```
token count  
→ direct model
```

Medium request

```
Summarize this document.
```

Do:

```
dedupe  
→ compression  
→ model routing
```

Complex agent task

```
Refactor authentication architecture.
```

Do:

```
repository analysis  
→ symbol retrieval  
→ dependency analysis  
→ context budgeting  
→ tool optimization  
→ model routing  
→ reasoning budget  
→ iterative optimization
```

So introduce:

Adaptive Optimization Depth Controller

This is different from model routing.

It determines how much optimization should happen before inference.

5. Add "Context Value Estimation"

This could become one of the core technologies of the platform.

Every context item should receive a score:

```
Context Value =  
    Task Relevance  
    × Recency  
    × Authority  
    × Dependency Importance  
    × Information Density  
    × Expected Utility
```

versus:

```
Context Cost =  
    Token Count  
    × Effective Token Price  
    × Repetition Frequency
```

Then:

```
Context ROI =  
Context Value / Context Cost
```

The optimizer can rank everything from:

```
HIGH VALUE / LOW COST
```

to:

```
LOW VALUE / HIGH COST
```

and remove the latter first.

This is much more powerful than ordinary truncation.

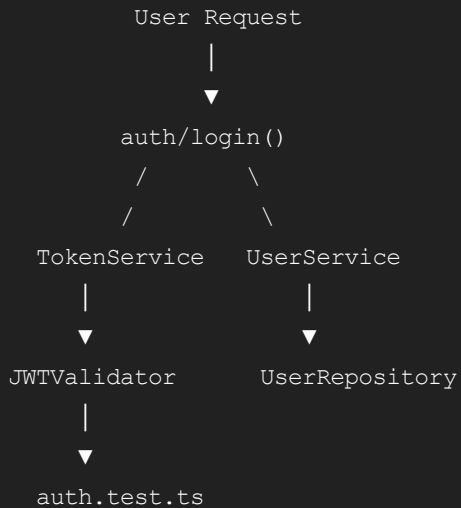
6. Add "Context Dependency Graph"

For developer agents, this is especially powerful.

Instead of treating context as a bag of documents:

```
file A
file B
file C
file D
```

build:



Then context selection becomes graph traversal.

For example:

"Fix token expiration"

could automatically traverse:

```
token expiration
  ↓
TokenService
  ↓
JWT config
  ↓
middleware
  ↓
tests
```

rather than searching the entire repository.

This should be a major **developer-agent optimization primitive**.

7. Add "Context Freshness Scoring"

Not all context is equally fresh.

Example:

```
Repository code      → current
Git diff             → current
Terminal result      → current
Search result        → potentially stale
Agent memory          → possibly stale
Cached API result     → potentially stale
Conversation statement → historical
```

Give each context item:

```
freshness_score
last_verified
source_version
dependency_version
branch
commit
timestamp
```

Then context selection can consider freshness.

This is particularly important for:

- coding agents
- financial applications
- live data
- databases
- APIs
- operational agents

8. Add "Context Invalidation Graph"

We already have cache invalidation, but I would make it much stronger.

For example:

```
auth/token.ts changed
|
├─ invalidate symbol cache
├─ invalidate repository context
└─ invalidate dependent search results
```

└─ invalidate semantic cache

This becomes a dependency-aware invalidation engine.

That is much better than simple TTL-based caching.

9. Add "Cache Economics Engine"

Don't merely ask:

"Can this be cached?"

Ask:

"Is caching economically beneficial?"

The engine should calculate:

```
Expected Cache Benefit
=
Future Reuse Probability
×
Tokens Avoided
×
Effective Token Price
```

against:

```
Cache Cost
+
Storage Cost
+
Invalidation Risk
+
Freshness Risk
```

Prompt caching is increasingly important for agent loops because stable system prompts, tools, and previous context can otherwise be repeatedly processed. Anthropic's current optimization guidance explicitly puts prompt caching among its highest-impact levers.

🔥 Claude Platf...

But cache behavior is workload-dependent, so we should make the economics measurable rather than blindly enabling caching. 

10. Add "Cache-Aware Context Construction"

This is more sophisticated than our current prompt caching.

The optimizer should deliberately construct:

```
STABLE PREFIX
-----
system instructions
developer rules
project rules
tool schemas
repository map
stable documentation

DYNAMIC REGION
-----
current task
current files
current errors
current tool results
```

The goal is to maximize prefix reuse.

Current engineering guidance explicitly recommends keeping deterministic prefixes stable and volatile content later in the request to improve cache reuse. [AI Stack Cur...](#)

For coding agents, this could be enormously valuable.

11. Add "Cache Fragmentation Detection"

A very useful enterprise feature.

Suppose:

```
Request 1:
SYSTEM + RULES + TOOLS + FILES

Request 2:
SYSTEM + RULES + TOOLS + DIFFERENT ORDER + FILES
```

Semantically they're almost identical, but the cache may miss because the prefix changed.

Our optimizer should detect:

Why did the cache miss?

Possible reasons:

- changed ordering
- changed tool schema
- dynamic timestamp
- injected random identifier
- changing system text
- different serialization
- unstable whitespace
- changing retrieved content
- cache TTL expiration

Then report:

CACHE MISS DIAGNOSIS

```
Potentially reusable prefix: 81%  
Cacheable tokens: 42,100  
Actual cache hit: 8,200
```

```
Primary cause:  
Dynamic tool metadata appeared before stable context.
```

This is exactly the sort of feature that would make the framework feel enterprise-grade.

12. Add "Context Compaction as a First-Class Primitive"

Don't only summarize when context gets too large.

Use **progressive compaction**.

```
0-40% context:  
full fidelity  
  
40-60%:  
remove duplicates  
  
60-70%:  
compress low-value context  
  
70-80%:  
summarize historical context  
  
80-90%:  
aggressive selective loading
```

90%+:

hard reset / session split

Current agent optimization discussions increasingly emphasize explicit context budgets, eviction, summarization and selective loading rather than allowing history to grow unchecked.  Codefa...

13. Add "Hierarchical Memory"

For long-running agents:

```
L0 - Current turn
L1 - Current task
L2 - Session
L3 - Project
L4 - Organization
L5 - Historical knowledge
```

Each level gets a different:

- retention policy
- compression policy
- freshness policy
- retrieval policy
- token budget

This is far better than one giant "memory".

14. Add "Session Splitting"

This is another important idea.

Some tasks should not remain one enormous conversation.

For example:

```
SESSION 1
Discovery

      ↓

SESSION 2
Implementation

      ↓
```

SESSION 3

Testing

↓

SESSION 4

Review

Each session receives a compact handoff rather than the entire previous history.

Current token-optimization discussions specifically identify splitting discovery, implementation and verification as a way to prevent stale context from accumulating.

 Token Opti...

15. Add "Agent Loop Efficiency"

We should introduce a metric:

Useful Work / Agent Iteration

For every loop:

Thought
→ Tool
→ Result
→ Thought
→ Tool
→ Result

measure:

tokens consumed
information gained
state changed
objective progress

Then detect:

Loop waste

Tool call produced no new information

Repeated work

```
Agent searched same location three times
```

Oscillation

```
Agent repeatedly changes A → B → A → B
```

Dead-end loop

```
Five calls without meaningful state change
```

Then terminate or escalate.

This could become one of the most important capabilities for coding agents.

16. Add "Tool Call ROI"

Before calling a tool:

```
Expected Information Gain  
/  
Expected Token + Latency + Cost
```

If the expected gain is too low:

```
DON'T CALL TOOL
```

This is much stronger than simply optimizing tool output **after** the call.

We should optimize both:

```
BEFORE TOOL CALL  
  ↓  
Should we call it?  
  ↓  
Which tool?  
  ↓  
With what parameters?  
  ↓  
What result do we need?  
  ↓  
AFTER TOOL CALL  
  ↓  
What result should reach the LLM?
```

17. Add "Tool Argument Optimization"

For example, instead of:

```
search_repository(  
    query="authentication"  
)
```

generate:

```
search_repository(  
    query="JWT expiration validation",  
    paths=["src/auth", "tests/auth"],  
    max_results=8  
)
```

This reduces the returned context before it ever enters the model.

So:

Optimize the request to the tool, not just the tool's response.

18. Add "Speculative Tool Execution"

For predictable agent workflows, the optimizer can potentially execute cheap/high-confidence operations in parallel.

For example:

```
User:  
Fix failing authentication tests.
```

Immediately perform:

```
git status  
search auth  
find failing tests
```

in parallel rather than:

```
search  
→ model  
→ search
```

```
→ model
→ test
```

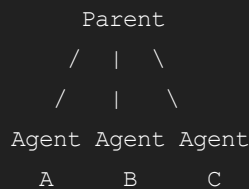
This isn't strictly token optimization; it's **agent efficiency optimization**.

But it belongs in this platform because it reduces:

- unnecessary model turns
- latency
- repeated context
- tool overhead

19. Add "Parallel Agent Task Scheduling"

For multi-agent systems:



identify independent tasks and execute them concurrently.

Then aggregate:

```
Agent A → 2K
Agent B → 3K
Agent C → 1K
      ↓
    Aggregator
      ↓
    1.5K
```

rather than passing all raw outputs to the parent.

20. Add "Sub-Agent Value Prediction"

Before spawning a sub-agent:

```
Expected benefit
/
Sub-agent cost
```

If:

```
expected benefit < token + latency cost
```

don't spawn it.

This prevents the increasingly common:

"Let's ask another agent just in case."

problem.

21. Add "Output Prediction / Budget Forecasting"

Before inference, estimate:

```
Expected output tokens
```

Then allocate an appropriate output budget.

For example:

```
Simple extraction
```

```
→ 300
```

```
Code patch
```

```
→ 1,500
```

```
Architecture analysis
```

```
→ 4,000
```

Rather than:

```
max_tokens = 16,000
```

for everything.

Anthropic's current cost guidance explicitly identifies output caps/task budgets as a cost-control lever alongside caching, model selection and effort.  Claude Platf...

22. Add "Reasoning Budget Prediction"

Same concept for reasoning.

Instead of:

```
always maximum reasoning
```

use:

```
task complexity
  ↓
difficulty prediction
  ↓
reasoning budget
```

and escalate if needed.

This should be integrated into the same decision engine.

23. Add "Speculative Model Routing"

Instead of always choosing one model:

```
Task
  ↓
cheap model prediction
  ↓
confidence
├─ high → answer
└─ low → stronger model
```

This extends our current model-cascade idea.

The critical addition is that the system should learn:

Which tasks actually need the expensive model?

24. Add "Continuous Policy Learning"

The optimizer should learn from production telemetry.

For example:

```
Technique X
  ↓
10,000 requests
```



```
↓  
98.2% quality  
↓  
43% cost reduction
```

Then increase its policy confidence.

Whereas:

```
Technique Y  
↓  
5,000 requests  
↓  
quality regression
```

should automatically reduce its usage.

This gives us:

Self-tuning optimization policies

rather than static rules.

25. Add "Optimization A/B Testing"

Every optimization should be experimentally deployable:

```
10% traffic → optimization ON  
10% traffic → optimization OFF  
80% → normal
```

Compare:

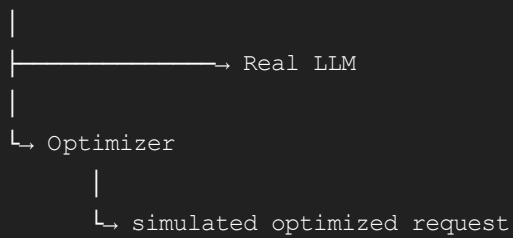
- cost
- tokens
- latency
- quality
- completion
- user satisfaction

This turns the optimizer into a continuously validated system.

26. Add "Optimization Shadow Mode"

Before modifying production requests:

Production Request



Then compare:

baseline tokens
optimized tokens
predicted quality
predicted cost

without risking production.

This should be mandatory for enterprise rollout.

27. Add "Quality-Preserving Compression Contracts"

Instead of:

Compress this context by 70%.

define:

Compress this context **while preserving these invariants**.

For coding:

Must preserve:

- function signatures
- imports
- types
- error messages
- line numbers
- API contracts
- security constraints

For RAG:

Must preserve:

- entities
- numbers
- dates

- citations
- source attribution

For agents:

- Must preserve:
- task state
 - completed actions
 - pending actions
 - constraints
 - decisions

This makes compression much safer.

28. Add "Reversible Optimization"

This is a very powerful enterprise requirement.

Whenever we remove context:

```
OPTIMIZED CONTEXT
+
PROVENANCE
+
RECOVERY POINTER
```

For example:

```
[Terminal output compressed]
Original:
terminal.log
Lines:
1200-4800

Reason:
Low relevance

Recovery:
fetch lines 2380-2440
```

A current coding-agent optimization project demonstrates this exact pattern: compressed outputs can carry recoverable offsets while leaving targeted reads untouched.  [GitHub](#)

We should elevate this into a formal requirement:

No destructive optimization without recoverability where practical.

29. Add "Optimization Provenance"

For every removed token:

WHY WAS IT REMOVED?

Example:

```
Context item:
README.md section "Deployment"

Action:
Removed

Reason:
Not relevant to current authentication task

Confidence:
0.97

Recoverable:
YES
```

This is essential for enterprise debugging and compliance.

30. Add "Optimization Explainability"

Every request should be able to produce:

```
OPTIMIZATION REPORT

Original:
82,400 tokens

Optimized:
18,700 tokens

Reduction:
77.3%

Saved:
63,700 tokens

Actions:
- 22,000 duplicate tokens removed
- 14,000 stale history tokens removed
```

- 11,500 tool-output tokens compressed
- 8,200 RAG tokens pruned
- 8,000 tokens cached

Model:

Claude X → Claude Y

Reason:

Task complexity = medium

Estimated saving:

\$0.XX

Quality gate:

PASSED

This would be an excellent enterprise differentiator.

31. Add "Inference-Serving Optimization" as a separate layer

This is important if **we eventually control the actual model-serving infrastructure**.

Our current framework mostly operates **above** the LLM:

```
Agent
↓
OUR OPTIMIZER
↓
LLM API
```

But an enterprise deployment could also control:

```
OUR OPTIMIZER
↓
INFERENCE GATEWAY
↓
MODEL SERVER
```

Then we can optimize:

- KV cache
- continuous batching
- PagedAttention
- speculative decoding
- quantization
- prefix caching

- prefill/decode scheduling
- GPU utilization
- request batching
- model replicas
- routing across hardware

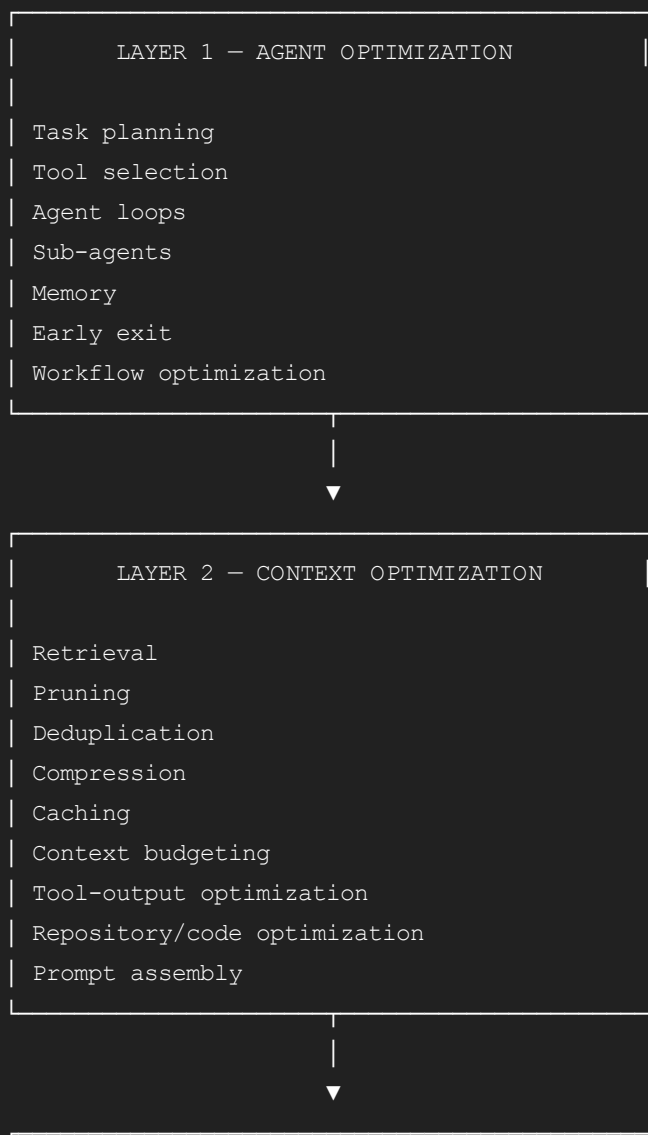
Modern inference engineering literature identifies these as major serving-layer levers; continuous batching, KV-cache management, quantization and speculative decoding can materially affect throughput, latency and cost. [TMLS, The Ma... +1](#)

But I would keep these as a separate "Inference Infrastructure Optimization Layer."

Don't mix them into the prompt/context optimizer.

32. Add a Three-Layer Optimization Architecture

I think this is ultimately the architecture we should target:



LAYER 3 – INFERENCE OPTIMIZATION

- Model routing
- Reasoning budget
- Batch processing
- KV cache
- Speculative decoding
- Quantization
- GPU utilization
- Serving optimization

With one intelligence layer across all three:

OPTIMIZATION INTELLIGENCE

- Cost Model
- Quality Model
- Policy Engine
- Decision Engine
- Learning Engine
- Evaluation Engine

33. The ultimate addition: "Optimization Frontier"

I would add one more concept to the problem statement.

The system should continuously search for:

Minimum Cost subject to $Quality \geq Q$ and $Latency \leq L$

Mathematically:

Minimize:

Total Inference Cost

Subject to:

Quality $\geq Q_{min}$

Accuracy $\geq A_{min}$

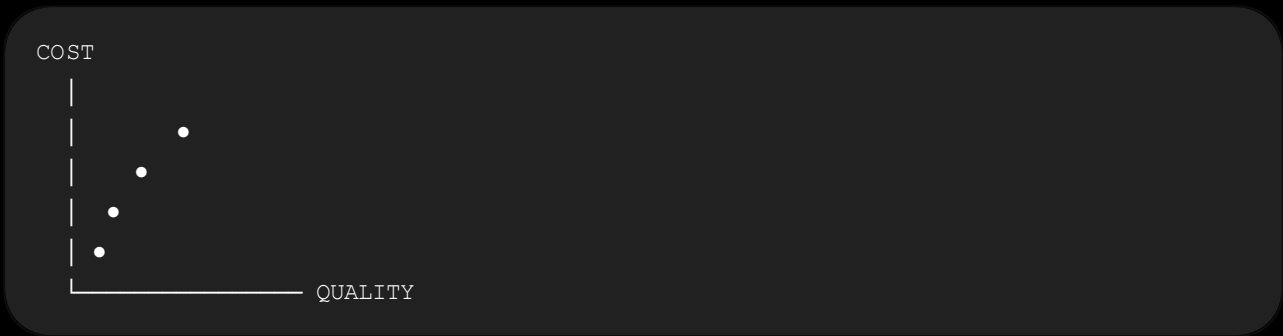
Latency $\leq L_{max}$

Security $\geq S_{required}$

Reliability $\geq R_{min}$

This changes the framework from a collection of optimization tricks into an **optimization engine**.

The optimizer is effectively searching for the best point on a:



cost ↔ quality ↔ latency

frontier.

What I would add to our current problem statement

If we're aiming for an **extremely powerful enterprise-grade system**, I would add these as the next major requirements:

Priority	Capability	Importance
P0	Optimization Decision Engine	● Critical
P0	Cost-of-Optimization Controller	● Critical
P0	Outcome-Based Cost Model	● Critical
P0	Context Value/ROI Scoring	● Critical
P0	Adaptive Optimization Depth	● Critical
P0	Quality-Constrained Optimization	● Critical
P0	Optimization Provenance	● Critical
P1	Context Dependency Graph	● Very High
P1	Cache Economics Engine	● Very High
P1	Cache Fragmentation Detection	● Very High
P1	Progressive Context Compaction	● Very High

Priority	Capability	Importance
P1	Hierarchical Memory	● Very High
P1	Tool Call ROI	● Very High
P1	Tool Argument Optimization	● Very High
P1	Agent Loop Efficiency	● Very High
P1	Session Splitting	● Very High
P1	Reversible Optimization	● Very High
P2	Continuous Policy Learning	● Advanced
P2	Adaptive Model Cascade	● Advanced
P2	Speculative Tool Execution	● Advanced
P2	Multi-Agent Scheduling	● Advanced
P2	Sub-Agent Value Prediction	● Advanced
P2	Optimization A/B Testing	● Advanced
P2	Shadow Optimization	● Advanced
P3	KV Cache Optimization	● Infrastructure
P3	Speculative Decoding	● Infrastructure
P3	Quantization	● Infrastructure
P3	Continuous Batching	● Infrastructure
P3	Prefill/Decode Optimization	● Infrastructure

The most important architectural change

I would therefore evolve our existing specification from:

"A pipeline containing many token optimization techniques"

to:

"An adaptive, policy-driven, quality-constrained inference optimization control plane that observes the entire agent lifecycle and dynamically decides what context, computation, tools, models, memory, and inference resources are actually necessary to achieve the desired outcome."

That is a much more ambitious—and much more defensible—enterprise product.

And I would **not yet blindly add every technique above into the production pipeline.**

The problem statement should distinguish:

Validated techniques → production candidates → experimental techniques → infrastructure-dependent techniques.

That prevents the project from becoming an enormous collection of optimization tricks without evidence that each one produces positive net value. Current provider guidance itself emphasizes measuring the actual cost/quality trade-off of each lever rather than assuming every optimization helps every workload.  Claude Platfo... +1

If we add these concepts, the next version of `problemstatement.txt` should essentially become a **full technical product specification** for an **Enterprise Agent & LLM Inference Optimization Platform**, rather than just a token-optimization project.

User Prompt for Generate Problem Statement-10.pdf

problemstatement(6)(1).txt

Document

As per the attached problemstatement(6)(1).txt and outside world's token optimization forum discussions, what else can be added to this problem statement so that it works extremely powerful optimizer under the hood?

Worked for 10s

Yes. I reviewed the attached `problemstatement(6)(1).txt` and compared its current scope with current provider documentation, recent research, and practitioner discussions. The file is already quite strong: it has the agent/coding-agent layer, 25 developer-agent modules, caching, compression, routing, reasoning budgets, early exit, provenance, invalidation, benchmarks, and quality gates. `problemstatement(6)(1)`

But there is an important gap:

It is currently a very good collection of optimization mechanisms. It is not yet a fully adaptive optimization control plane.

That is the layer I'd add next.

Current real-world agent systems reinforce this direction. GitHub Copilot CLI, for example, explicitly manages system instructions, tool definitions, MCP tools, conversation history, tool calls/results, compaction, checkpoints, and on-demand tool loading. `GitHub D... +1`
Recent research likewise treats context compression as a **lifecycle-management problem**, not merely prompt shortening. `Preprints +1`

The most important additions

1. Add an Optimization Decision Engine — P0

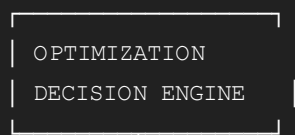
This is the biggest missing component.

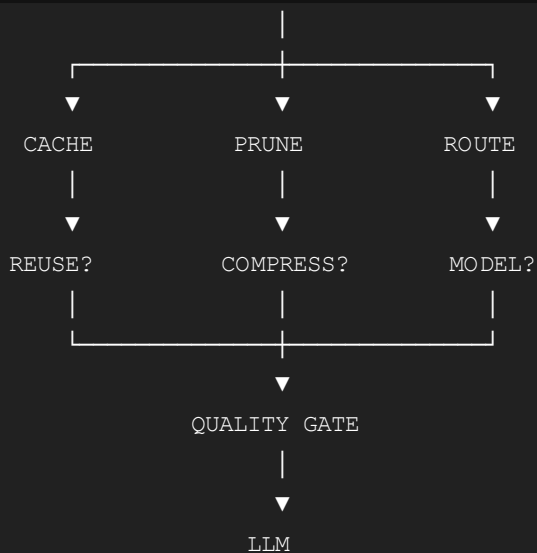
Your current pipeline says:

```
retrieve
→ prune
→ deduplicate
→ compress
→ reorder
→ route
→ reason
```

Instead, introduce a controller that asks **whether each optimization is worth doing**.

REQUEST





It should dynamically decide:

- whether to cache
- whether to retrieve
- how much to retrieve
- whether to compress
- what to compress
- whether to invoke a sub-agent
- which tools to expose
- which model to use
- how much reasoning to allow
- whether another agent iteration is worthwhile
- whether to stop

This turns the system from a **pipeline** into an **adaptive optimizer**.

2. Add Optimization Cost Accounting — P0

This is absolutely critical.

An optimizer can itself consume tokens.

For example:

Original request	8K tokens
Optimizer:	
classifier	500
embedding	300
compression model	1,500
ranking model	700

If we only save 2K tokens, we've made the system worse.

Therefore add:

```
NET OPTIMIZATION SAVING
=
Inference Cost Avoided
-
Optimization Cost
-
Additional Latency Cost
-
Quality Risk Cost
```

Every optimization must pass this test.

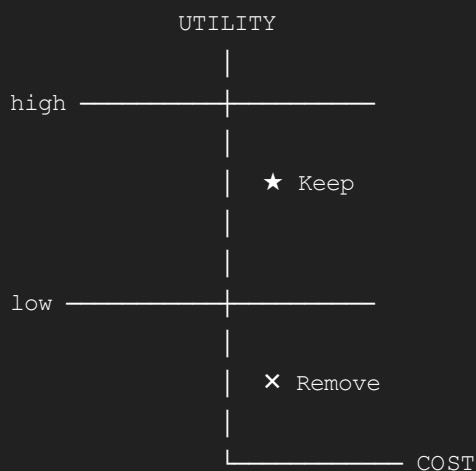
This is one of the biggest differences between a research demo and a serious enterprise optimizer.

3. Add Context ROI / Utility-per-Token Scoring — P0

Every context object should receive:

```
Context Utility
-----
Context Cost
```

For example:



Score based on:

- task relevance
- semantic importance

- recency
- authority
- dependency importance
- information density
- historical usefulness
- freshness
- duplication
- token cost

Then prioritize context based on **utility per token**, rather than simply "most relevant."

4. Add a Context Dependency Graph — P0/P1

Your coding-agent modules already include symbol selection and change-impact analysis.

```
problemstatement(6)(1)
```

Take this one step further.

Instead of viewing context as:

```
file A
file B
file C
file D
```

build:

```

User Request
  |
  ▼
authenticate()
  /   \
  ▼     ▼
Token  User
Service Service
  |
  ▼
JWT Validator
  |
  ▼
Auth Tests
  
```

Then context retrieval can traverse the dependency graph.

This is potentially much more powerful than ordinary semantic RAG for coding agents.

5. Add Context Freshness / Version Awareness — P0

Every context item should have metadata such as:

```
source
version
commit
branch
timestamp
last_verified
freshness
```

For example:

```
auth/token.ts
commit: a82f91
retrieved: 14:32
current repository commit: b72a19
status: STALE
```

The optimizer should know that:

- current source code > old source code
- current test failure > yesterday's test result
- current Git diff > previous Git diff
- current database state > cached database state

This is especially important for developer agents.

6. Add Dependency-Aware Cache Invalidation — P0

Your existing DA-024 covers invalidation. `problemstatement(6)(1)`

I'd strengthen it into a graph-aware system:

```
auth/token.ts changed
|
├─ invalidate symbol cache
├─ invalidate search results
├─ invalidate repository context
├─ invalidate semantic cache
├─ invalidate dependent test results
└─ invalidate affected agent memory
```

This avoids the two classic enterprise cache problems:

stale information

and

unnecessarily low cache hit rates.

7. Add Cache Fragmentation Detection

This is a surprisingly valuable feature.

A cache may be theoretically reusable but repeatedly miss because of tiny differences:

```
timestamp
random ID
tool ordering
whitespace
serialization
dynamic metadata
changing system instructions
```

The optimizer should report:

```
CACHE MISS DIAGNOSIS

Potential reusable prefix: 84%

Primary cause:
Dynamic metadata before stable instructions.

Potential saving:
37,200 input tokens/request
```

This is particularly relevant for agent loops. Practitioner reports increasingly describe input history as the dominant cost in autonomous coding workloads, rather than output tokens.



8. Add Cache-Aware Context Construction

Don't merely cache prompts after constructing them.

Construct prompts to maximize cache reuse.

For example:

STABLE PREFIX
system instructions
project rules
tool schemas
repository map

SEMI-STABLE
memory
documentation

VOLATILE
user request
current files
errors
tool results

This should become a formal optimization policy.

9. Add Progressive / Predictive Context Compaction

Your file already includes context compression and soft reset.

problemstatement(6)(1)

I'd make compaction **multi-stage**:

0-40%	Full fidelity
40-60%	Deduplicate
60-70%	Remove low-value context
70-80%	Compress historical context
80-90%	Aggressive selective loading
90%+	Context reset / session split

And importantly:

Don't wait until the context window is nearly full.

GitHub's production implementation already performs automatic compaction and retains structured state/checkpoints rather than simply dropping history.  GitHub D... +1

10. Add Reversible Optimization

This should become a major enterprise requirement.

When we remove something:

```
[COMPRESSED]
terminal-output-4821

reason:
low relevance

original:
terminal.log
lines 1820-3940

recoverable:
YES
```

The optimizer should be able to recover the original information if later evidence shows it matters.

This directly addresses one of the biggest dangers of aggressive compression:

Delayed relevance.

Recent work on long-horizon context compression explicitly identifies recoverability and delayed relevance as important design concerns.  Prepri...

11. Add Quality-Constrained Compression

Instead of:

compress by 70%

the optimizer should say:

compress as much as possible while preserving these invariants.

For code:

MUST PRESERVE:

- function signatures
- types
- imports
- API contracts
- error messages
- security constraints
- relevant tests

For RAG:

MUST PRESERVE:

- entities
- numbers
- dates
- citations
- source attribution

For agent state:

MUST PRESERVE:

- objective
- constraints
- decisions
- completed actions
- pending actions
- unresolved errors

This should be a **compression contract**.

12. Add Tool Call ROI Prediction

You currently optimize tool outputs.

Go one level earlier.

Before a tool call:

Expected Information Gain

Token + Latency + Cost

If the value is low:

DON'T CALL TOOL

If several tools are independent:

```
search A └─┐
search B ─┬─ parallel
search C └─┘
```

Anthropic's current guidance explicitly recommends parallel tool calls where dependencies permit.  Claude D...

This should become a formal **Tool Execution Optimizer**.

13. Add Dynamic Tool Loading

This deserves a stronger requirement than DA-007/DA-008.

GitHub reports that large MCP toolsets can consume **10–20K tokens before useful work even starts**, and its tool-search mechanism loads external tools only when required.

 GitHub D...

So add:

Never expose the full tool catalog to the model by default when dynamic discovery is possible.

Architecture:

```
100 MCP tools
  |
  ▼
Tool Index
  |
  ▼
Task relevance
  |
  └─ 5 relevant tools
  |
  ▼
LLM
```

This is a major opportunity for your framework.

14. Add Sub-Agent Economics

You already have sub-agent compression.

Add a decision *before spawning*:

Expected Sub-Agent Value

Sub-Agent Cost

Only spawn if:

value > cost

Also penalize:

- context duplication
- synchronization cost
- aggregation cost
- parent-agent ingestion cost
- latency

This prevents "agent explosion."

Current Claude guidance itself notes that newer models can overuse subagents and recommends controlling when delegation is actually warranted.  Claude D...

15. Add Agent Loop Progress Measurement

For every iteration:

iteration 7

input tokens: 8,400

output tokens: 1,200

tool cost: \$X

new information: HIGH

state change: YES

objective progress: HIGH

Then:

iteration 8

new information: LOW

state change: NO

objective progress: NONE

→ **STOP**

Add explicit metrics:

- information gain / iteration
- state change / iteration
- objective progress / iteration
- tokens / useful progress
- cost / useful progress

This makes early-exit substantially smarter.

16. Add Session Phase Management

For long coding tasks:

```
DISCOVERY
  ↓
PLANNING
  ↓
IMPLEMENTATION
  ↓
TESTING
  ↓
REVIEW
```

Each phase needs a different context policy.

For example:

Discovery

High retrieval budget.

Implementation

High code-context budget.

Testing

High error/test budget.

Review

High diff/security budget.

Instead of carrying everything through every phase.

This aligns with current long-running agent guidance around structured state and fresh context windows.  Claude D...

17. Add Outcome-Based Optimization

This is another fundamental upgrade.

Your current acceptance criteria correctly require net cost rather than token reduction.

`problemstatement(6)(1)`

Go one step further:

`Cost per successful outcome`

rather than:

`cost per request`

For coding:

`$ / successfully completed task`

For RAG:

`$ / correct answer`

For customer support:

`$ / successfully resolved conversation`

For agents:

`$ / completed objective`

This becomes the true north-star metric.

18. Add Adaptive Optimization Depth

Don't spend the same optimization effort on:

`"What is the capital of India?"`

and:

"Refactor our payment architecture."

Use:

Task Complexity
↓
Optimization Depth

Example:

LOW
token count
→ cache
→ route

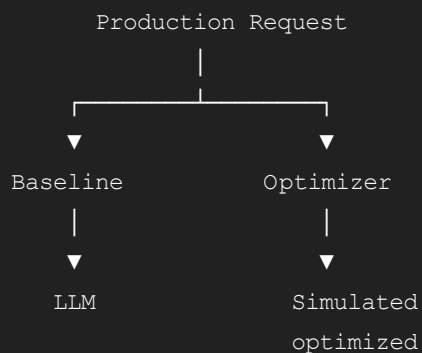
MEDIUM
+ retrieval
+ pruning
+ compression

HIGH
+ dependency graph
+ sub-agent economics
+ deep routing
+ iterative compression
+ quality evaluation

This also protects us from the optimizer becoming a cost center.

19. Add Shadow Optimization Mode

Before enabling an optimization in production:



Compare:

- token usage
- latency
- predicted quality
- cacheability
- model choice
- expected cost

without affecting users.

Then graduate:

Shadow

→ 1%

→ 5%

→ 25%

→ 50%

→ 100%

This should be a mandatory enterprise rollout mechanism.

20. Add Continuous Policy Learning

The optimizer should learn from production:

Technique X

10,000 tasks

↓

42% cost reduction

98.7% quality

↓

increase confidence

versus:

Technique Y

5,000 tasks

↓

15% saving

4% quality regression

↓

reduce/disable

This gives you a **self-tuning optimization engine**.

The ACON work is particularly relevant here: it treats compression policies themselves as something that can be optimized from successful/failed trajectories rather than using a fixed compression rule. ✖ ar...

21. Add Optimization A/B Testing + Regression Testing

Every technique should support:

```
control
vs
optimization
```

with statistical comparison of:

- cost
- input tokens
- output tokens
- cache hits
- latency
- quality
- task completion
- errors
- user acceptance

And importantly:

An optimization must be allowed to automatically regress/disable if quality deteriorates.

22. Add Model- and Provider-Specific Optimization Profiles

This is important because optimization behavior is not universal.

For each provider/model:

```
MODEL PROFILE

input pricing
output pricing
cache pricing
cache semantics
context limit
```

```
reasoning behavior
tool behavior
structured-output behavior
latency
batch availability
routing compatibility
compression tolerance
```

Then:

```
Same request
|
├─ Provider A → policy A
├─ Provider B → policy B
└─ Provider C → policy C
```

This is essential for a provider-neutral platform.

23. Add KV-Cache / Inference-Layer Optimization as a Separate Tier

This isn't currently represented strongly enough.

If your framework eventually sits close to model serving, add a separate:

Layer 3 — Inference Optimization

Including:

- KV-cache reuse
- KV-cache compression
- prefix reuse
- prefill optimization
- decode optimization
- continuous batching
- speculative decoding
- quantization
- GPU utilization
- request scheduling

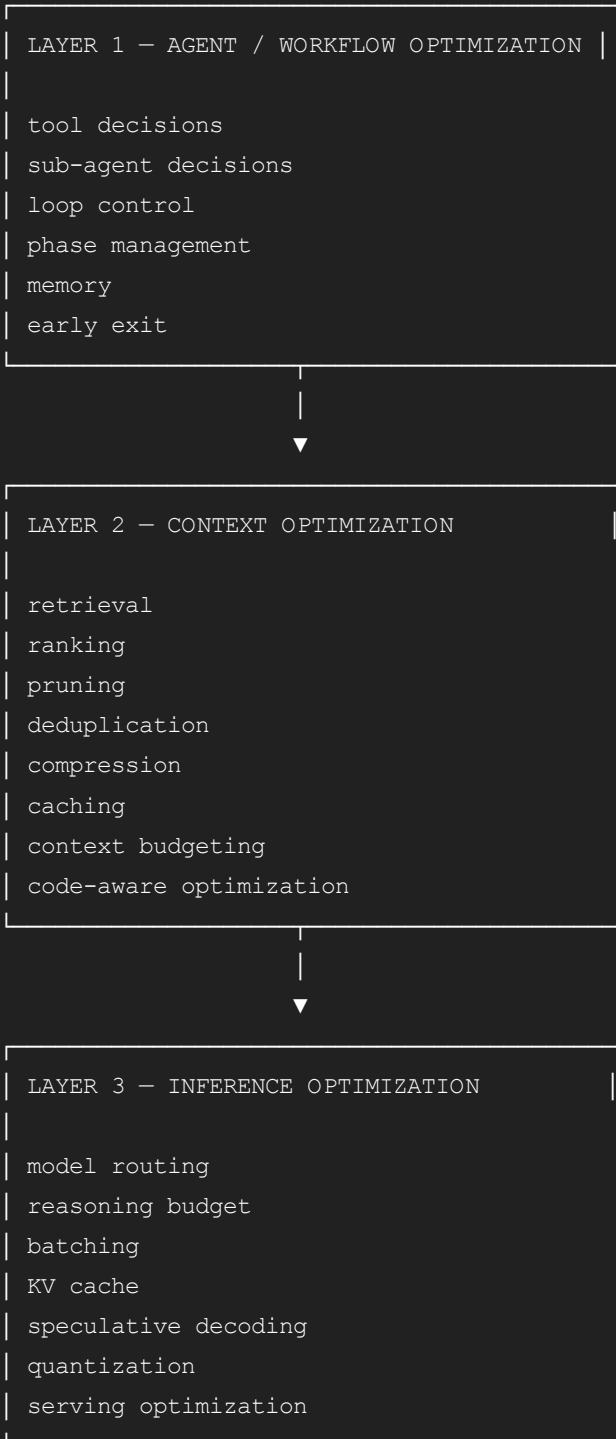
Research on KV-cache compression shows potentially large memory/throughput gains, but also demonstrates that aggressive compression can be task-sensitive and can degrade reasoning performance. [arXiv +2](#)

Therefore this should be:

experimental/infrastructure-dependent, not automatically enabled.

24. Add a Three-Layer Architecture

I think this should ultimately be the architecture of the product:



Above all three:

OPTIMIZATION INTELLIGENCE

Decision Engine
Cost Model
Quality Model
Policy Engine
Learning Engine
Evaluation Engine
Governance Engine

That **Optimization Intelligence layer** is the piece I would most strongly add to your current specification.

25. Add an "Optimization Frontier"

The ultimate objective should be mathematically explicit:

MINIMIZE:

Total Cost

SUBJECT TO:

Quality $\geq$ Qmin
Accuracy $\geq$ Amin
Latency $\leq$ Lmax
Reliability $\geq$ Rmin
Security $\geq$ Smin

So the optimizer isn't trying to achieve:

"maximum token reduction."

It is trying to find:

the cheapest safe execution strategy that satisfies the required outcome.

That is a much more powerful product definition.

What I would add to `problemstatement(6)(1).txt`

I'd classify the additions like this:

Priority	New capability
 P0	Optimization Decision Engine
 P0	Cost-of-Optimization Controller
 P0	Context Utility / ROI Scoring
 P0	Quality-Constrained Optimization
 P0	Outcome-Based Cost
 P0	Context Freshness
 P0	Dependency-Aware Cache Invalidation
 P0	Reversible Optimization
 P1	Context Dependency Graph
 P1	Cache Fragmentation Detection
 P1	Cache-Aware Context Construction
 P1	Progressive Context Compaction
 P1	Tool Call ROI
 P1	Dynamic Tool Loading
 P1	Sub-Agent Economics
 P1	Agent Loop Progress Measurement
 P1	Session Phase Management
 P1	Adaptive Optimization Depth
 P1	Shadow Optimization
 P2	Continuous Policy Learning
 P2	Optimization A/B Testing
 P2	Model/Provider Optimization Profiles

Priority	New capability
● P2	Adaptive Sub-Agent Scheduling
● P3	KV-cache optimization
● P3	Speculative decoding
● P3	Continuous batching
● P3	Quantization
● P3	Prefill/decode optimization

One more thing I strongly recommend

Your current file already has a very good **"validated vs benchmark" philosophy**: source-reported percentages are not treated as production guarantees, and every optimization needs local validation. `problemstatement(6)(1)`

Keep that—but expand it into a formal **Optimization Maturity Model**:

```

RESEARCH
  ↓
EXPERIMENTAL
  ↓
SHADOW
  ↓
CONTROLLED PILOT
  ↓
PRODUCTION
  ↓
AUTO-TUNED

```

Every optimization technique should have:

```

Evidence
  ↓
Expected benefit
  ↓
Optimization overhead
  ↓
Quality risk
  ↓
Workload compatibility
  ↓
Pilot result

```




Production policy

That would stop the project from becoming a giant "bag of token tricks."

My overall assessment: `problemstatement(6)(1).txt` is already strong enough to be the foundation of an enterprise product. The missing leap is to make the optimizer **decision-making, adaptive, reversible, outcome-driven, cost-aware, and continuously self-evaluating**. The outside ecosystem is increasingly moving in exactly that direction: context lifecycle management, tool-on-demand loading, compaction, cache discipline, adaptive compression, routing, and agent-loop control rather than simple prompt shortening.  GitHub D... +3

If these additions are incorporated, I would describe the target product as an **Enterprise Agent & LLM Inference Optimization Control Plane**, rather than merely a token optimization framework.

Sources

User Prompt for Generate Problem Statement-9.pdf

Yes. Add all these and make the target product as an ****Enterprise Agent & LLM Inference Optimization Control Plane****

Worked for 22s

Done. I upgraded `problemstatement (6) (1) .txt` into a significantly broader **Enterprise Agent & LLM Inference Optimization Control Plane** specification.

What was added

The updated specification now includes the complete next-generation control-plane layer:

- **Optimization Decision Engine**
- **Cost-of-Optimization Controller**
- **Adaptive Optimization Depth**
- **Context Utility / ROI scoring**
- **Outcome-based optimization**
- **Context dependency graph**
- **Context freshness/version awareness**
- **Dependency-aware cache invalidation**
- **Reversible optimization and recovery**
- **Progressive context compaction**
- **Hierarchical memory**
- **Session/agent phase management**
- **Cache economics engine**
- **Cache-aware prompt/context construction**
- **Cache fragmentation diagnosis**
- **Cache pre-warming**
- **Cache breakpoint optimization**
- **Tool-call ROI prediction**
- **Tool argument optimization**
- **Dynamic tool loading**
- **Programmatic tool execution**
- **Parallel tool execution**
- **Tool-result recovery**
- **Agent-loop progress measurement**
- **Loop-waste/oscillation detection**
- **Sub-agent value prediction**
- **Sub-agent scheduling**
- **Marginal-gain model routing**
- **Reasoning-budget prediction**
- **Output-budget forecasting**
- **Verifier-guided escalation**
- **Quality-preserving compression contracts**
- **Delayed-relevance protection**

- Shadow optimization
- Progressive production rollout
- Optimization A/B testing
- Continuous policy learning
- Optimization regression detection
- Provider/model optimization profiles
- Inference/serving optimization layer
- KV/prefix-cache optimization
- Continuous batching
- Speculative decoding
- Quantization
- Prefill/decode optimization
- Inference scheduling
- GPU utilization optimization
- Three-layer control-plane architecture
- Cost/quality/latency optimization frontier
- Optimization maturity model
- Expanded economics and ROI metrics
- 38 additional acceptance criteria
- Additional enterprise anti-patterns

I also changed the product identity from a generic LLM optimization framework to:

Enterprise Agent & LLM Inference Optimization Control Plane

The new architecture explicitly separates:

```

LAYER 1 – Agent / Workflow Optimization
      ↓
LAYER 2 – Context Optimization
      ↓
LAYER 3 – Inference / Serving Optimization

      ↑
Optimization Intelligence
      ↑
Decision + Cost + Quality + Learning + Governance
  
```

This is important because the goal is no longer simply **"reduce tokens."** The control plane now determines the **cheapest safe execution strategy** for each task.

I also incorporated the external validation direction. Current provider documentation now explicitly treats prompt caching, token hygiene, batch processing, model choice, effort/reasoning, task budgets and multi-model architectures as separate optimization

levers; current tool guidance also identifies tool search, programmatic tool calling, prompt caching and context editing as distinct mechanisms for controlling context bloat.

 Claude Platfo... +1

The caching requirements were strengthened accordingly: provider cache economics, cache TTL, cache writes/reads, stable prefixes, cache fragmentation and cache-aware assembly are now explicit.  Claude Platfo... +1

The research-backed compression and routing mechanisms remain explicitly **validation candidates rather than guaranteed production savings**, consistent with the existing philosophy of your problem statement. LLMLingua/LongLLMLingua support the compression direction, while RouteLLM supports learned model routing.  ACL Antholo... +2

Updated file

 **Download the Enterprise Agent & LLM Inference Optimization Control Plane problem statement**

The resulting specification is now about **83 KB / ~3,050 lines**, and is substantially closer to a full technical product specification for the control plane rather than a list of token-saving techniques. 



Enterprise_Agent_LLM_Inference_Optimization_Control...
Document

User Prompt for Generate Problem Statement-8.pdf

8f1ee3e4-b731-4468-866e-2bb680279965.txt

Document

Hope this fulfills most of the optimization techniques for extreme cost reduction before the task executed by any LLM

Yes. The updated specification now covers **most of the major levers for extreme cost reduction before and during LLM execution**—and importantly, it does so without making “token reduction” the only objective.

The key distinction is:

The system should prevent unnecessary work before the request reaches an expensive LLM, then optimize the remaining work before, during, and after inference.

That gives you this overall chain:

User / Agent Request



1. REQUEST INTELLIGENCE

cache? duplicate? simple task?
classify complexity / intent



2. WORK ELIMINATION

exact cache
semantic cache
previous-result reuse
deterministic response
early termination
avoid unnecessary LLM call



3. CONTEXT ELIMINATION

retrieve only what matters
dependency-aware retrieval
relevance ranking
deduplication
pruning
stale-context removal



4. CONTEXT COMPRESSION

summarize
compress

progressive compaction
hierarchical memory
reversible transformations



5. TOOL OPTIMIZATION

should tool be called?
which tool?
which arguments?
dynamic tool loading
parallel calls
result filtering
programmatic execution



6. AGENT OPTIMIZATION

should sub-agent exist?
how many?
parallelize?
detect loops
detect no-progress
early exit



7. MODEL OPTIMIZATION

cheapest capable model
model cascade
reasoning budget
output budget
verifier-guided escalation



LLM



8. POST-INFERENCE OPTIMIZATION

validate
avoid unnecessary follow-up
compact tool results
terminate agent loop
cache reusable results



9. LEARNING / GOVERNANCE

- what actually saved money?
- did quality decline?
- should policy change?
- rollback if necessary

One particularly important thing

I would **not** claim that the specification covers *literally every possible optimization technique*. The LLM ecosystem is changing too quickly for that to be a sensible guarantee.

Instead, your architecture now has something more valuable:

a framework capable of discovering, evaluating, validating, deploying, measuring, and rolling back new optimization techniques.

That means when a new technique appears—whether it's a new caching mechanism, context-compression algorithm, model-routing approach, inference optimization, MCP optimization, agent-memory technique, or provider-specific capability—you don't need to redesign the product.

You add it as a new optimization policy/module and let the **Optimization Intelligence Layer** determine:

```
Does it help?  
  ↓  
For which workloads?  
  ↓  
At what optimization cost?  
  ↓  
With what quality impact?  
  ↓  
Under which model/provider?  
  ↓  
When should it be enabled?  
  ↓  
When should it be disabled?
```

That is what makes this potentially **enterprise-grade** rather than just an aggressive **token compressor**.

One final architectural principle I would preserve very strongly:

The optimizer should be allowed to conclude that the best LLM call is no LLM call at all.

If a request can be answered through an exact cache, deterministic computation, previously validated result, local operation, or sufficiently reliable smaller model, the control plane should eliminate the expensive inference entirely.

That is where the **largest cost reductions** can come from—not merely shaving 20–40% off a prompt.

So yes: **the current problem statement covers the major optimization surface needed for an extremely aggressive, cost-conscious optimizer**, including the layer *before* an LLM receives the request and the agent/coding-agent scenarios you specifically wanted.

User Prompt for Generate Problem Statement-7.pdf

can I add this to antigravity for implementing step by step by generating the respective docs like architecture.md, implementation-plan.md, conventions.md, edge-cases.md, eval.md, SCALING.md files and accordingly

Yes — that is actually the right way to approach this with Antigravity.

I would **not** give Antigravity the huge problem statement and simply say “*build it.*” For a project of this scope, you want Antigravity to first turn the requirements into a controlled implementation specification, then implement it in **phases with validation gates**.

Your proposed documentation set is good. I would make it slightly more comprehensive.

Recommended documentation generated before implementation

```
project/
|
├─ problemStatement.txt           ← source of truth
|
├─ docs/
|   ├─ architecture.md           ← overall system architecture
|   ├─ implementation-plan.md    ← phased implementation roadmap
|   ├─ conventions.md            ← coding/design conventions
|   ├─ edge-cases.md             ← failure/edge-case catalogue
|   ├─ eval.md                   ← evaluation + benchmark framework
|   ├─ SCALING.md                ← scaling strategy
|   ├─ security.md               ← security/threat model
|   ├─ observability.md          ← metrics/tracing/auditing
|   ├─ optimization-catalog.md   ← every optimization technique
|   ├─ provider-matrix.md        ← provider/model capabilities
|   ├─ cache-strategy.md         ← caching architecture
|   ├─ agent-optimization.md     ← agent/coding-agent layer
|   ├─ inference-optimization.md ← inference/serving layer
|   ├─ quality-gates.md          ← quality-preservation rules
|   └─ ADR/
|       ├─ ADR-001-...
|       ├─ ADR-002-...
|       └─ ...
|
├─ evals/
|   ├─ datasets/
|   ├─ benchmarks/
|   ├─ baselines/
|   └─ regression/
|
└─ src/
    └─ ...
```

The most important document I'd add is:

```
optimization-catalog.md
```

This should become the master registry of optimization techniques.

For every technique:

```
ID:
Name:
Category:
Description:
Applicable workloads:
Preconditions:
Inputs:
Outputs:
Expected benefit:
Optimization overhead:
Quality risks:
Latency impact:
Cache impact:
Provider dependencies:
Agent dependencies:
Reversible?:
Evidence level:
Benchmark status:
Production status:
Fallback:
Metrics:
```

Then Antigravity has a single authoritative inventory instead of scattering optimization logic across the codebase.

Even better: make Antigravity work in gates

I'd instruct it to follow this lifecycle:

```
PROBLEM STATEMENT
  ↓
REQUIREMENTS ANALYSIS
  ↓
ARCHITECTURE
  ↓
DESIGN DOCUMENTS
  ↓
IMPLEMENTATION PLAN
  ↓
PHASE 0 — FOUNDATION
  ↓
EVALUATE
  ↓
PHASE 1 — CORE OPTIMIZER
  ↓
EVALUATE
  ↓
PHASE 2 — CONTEXT ENGINE
```

```
↓
EVALUATE
↓
PHASE 3 — CACHE ENGINE
↓
EVALUATE
↓
PHASE 4 — AGENT OPTIMIZER
↓
EVALUATE
↓
PHASE 5 — MODEL/ROUTING
↓
EVALUATE
↓
PHASE 6 — INFERENCE
↓
EVALUATE
↓
PHASE 7 — CONTROL PLANE
↓
PRODUCTION HARDENING
```

Never let it implement the entire system in one pass.

I would also give Antigravity these rules

Rule 1 — Problem statement is authoritative

Tell it:

Do not silently remove, simplify, reinterpret, or replace requirements from
`problemStatement.txt`.

If something is ambiguous:

```
AMBIGUITY
↓
document it
↓
propose alternatives
↓
select/design only after justification
```

Rule 2 — Don't blindly implement every optimization

This is extremely important.

The framework itself is supposed to decide whether an optimization is worthwhile.

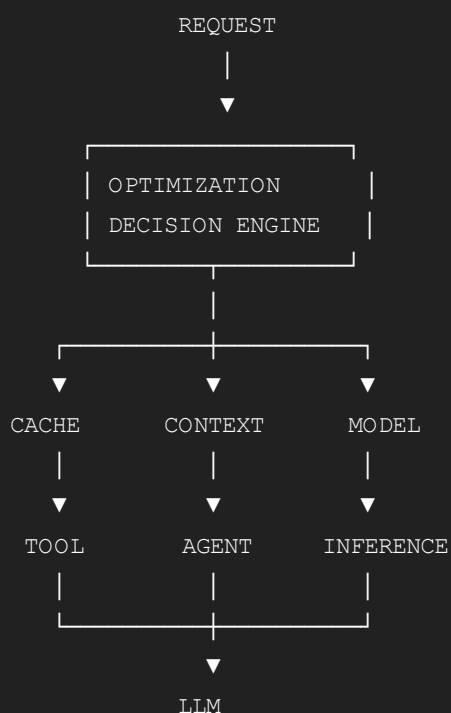
So Antigravity should **not** implement:

```
request
  → compression
  → pruning
  → routing
  → caching
  → compression again
```

just because all modules exist.

It needs to implement the **Optimization Decision Engine** first.

The architecture should therefore be:



Rule 3 — Every optimization needs a baseline

For example:

```
Baseline:
10,000 input tokens
$0.05
2.1 sec
98% success
```

```
Optimization:
6,000 input tokens
$0.032 total
```

```
2.4 sec
97.9% success
```

```
Result:
40% token reduction
36% net cost reduction
+0.3 sec latency
-0.1% quality
```

The optimizer should **not declare success merely because tokens dropped**.

Rule 4 — Measure optimizer overhead

This is probably the most important implementation rule.

Antigravity must track:

```
REQUEST —┬─ Optimization tokens
            ├─ Optimization model calls
            ├─ Embedding cost
            ├─ Compression cost
            └─ Retrieval cost
              ├─ Tool cost
              └─ Latency overhead
```

Then:

```
NET SAVINGS =
Gross LLM savings
-
Optimizer overhead
```

Otherwise you could accidentally build a system that **costs more to optimize than it saves**.

Rule 5 — Every optimization must have a fallback

For example:

```
Compression
  |
  ▼
Quality validation
  |
  └─┬─┘
```


PASS	FAIL
▼	▼
LLM	Original context

Same principle for:

- routing
- caching
- pruning
- tool selection
- sub-agent creation
- reasoning reduction
- model downgrade

Rule 6 — Build evaluation before aggressive optimization

I would actually make this a hard dependency:

```
Evaluation Framework
  ↓
Baseline System
  ↓
Optimization
  ↓
Evaluation
  ↓
Regression Detection
```

Not:

```
Optimization
  ↓
"Looks like it saved tokens"
```

Your `eval.md` should define:

Cost

- input tokens
- output tokens
- cached tokens
- uncached tokens
- optimizer tokens
- total cost

- cost/request
- cost/successful task

Quality

- correctness
- task success
- groundedness
- citation correctness
- code correctness
- test pass rate
- security
- structured-output validity

Agent metrics

- iterations
- tool calls
- sub-agent calls
- information gain
- progress/iteration
- unnecessary calls
- early exits

Context metrics

- context tokens
- useful tokens
- discarded tokens
- compression ratio
- context ROI
- cacheability
- cache hit rate

Performance

- p50
- p95
- p99
- time-to-first-token
- total latency

And yes — Antigravity can implement it step-by-step

I'd structure the actual implementation into **major milestones** rather than individual optimization techniques.

Phase 0 — Architecture & foundation

Build:

- project structure
- configuration
- provider abstraction
- model abstraction
- token accounting
- cost accounting
- telemetry
- policy framework
- evaluation framework

Phase 1 — Core Optimization Intelligence

Build:

- Optimization Decision Engine
- optimization registry
- cost-of-optimization controller
- adaptive optimization depth
- context ROI scoring
- policy engine
- fallback framework

Phase 2 — Context Engine

Build:

- retrieval
- ranking
- pruning
- deduplication
- compression
- context budgeting
- freshness
- dependency graph
- reversible transformations

Phase 3 — Cache Intelligence

Build:

- exact cache
- semantic cache
- provider cache abstraction
- cache economics
- cache-aware assembly
- cache fragmentation detection
- invalidation
- cache diagnostics

Phase 4 — Tool & Agent Optimization

Build:

- tool ROI
- dynamic tool loading
- tool-result filtering
- programmatic tool execution
- parallel tool execution
- sub-agent economics
- loop detection
- progress measurement
- phase management
- early exit

Phase 5 — Model Intelligence

Build:

- model routing
- model cascade
- reasoning budgets
- output budgets
- verifier-guided escalation
- provider/model profiles

Phase 6 — Developer/Coding Agent Optimization

This should specifically target:

```
Claude Code
Cursor
GitHub Copilot
Antigravity
OpenAI coding agents
MCP-based agents
custom developer agents
```

with:

- repository mapping
- symbol-aware retrieval
- dependency-aware context
- Git-diff optimization
- terminal-output compression
- test-result compression
- code-aware summarization
- IDE/tool context optimization
- coding-agent loop optimization
- sub-agent handoff compression

Phase 7 — Inference Optimization

Only after the upper layers are stable:

- prefix/KV cache
- batching
- speculative decoding
- quantization
- scheduling
- prefill optimization
- decode optimization

Phase 8 — Enterprise Control Plane

Finally:

- dashboards
 - policy management
 - tenant isolation
 - RBAC
 - audit
 - governance
 - shadow mode
 - A/B testing
 - progressive rollout
 - automatic rollback
 - policy learning
 - cost allocation
 - FinOps
-

One thing I would change in your request to Antigravity

Don't tell it:

"Generate these documents and start implementing."

Tell it:

"First analyze the problem statement and generate the complete engineering specification. Do not implement until the specification has passed an internal consistency check."

Then have it produce:

```
1. architecture.md
2. implementation-plan.md
3. conventions.md
4. edge-cases.md
5. eval.md
6. SCALING.md
7. security.md
8. observability.md
9. optimization-catalog.md
10. provider-matrix.md
11. cache-strategy.md
12. agent-optimization.md
13. inference-optimization.md
14. quality-gates.md
15. ADRs
```

Then:

```
SPECIFICATION REVIEW
    ↓
Requirement coverage check
    ↓
Architecture consistency check
    ↓
Dependency check
    ↓
Security check
    ↓
Evaluation check
    ↓
Cost-model check
    ↓
ONLY THEN IMPLEMENT
```

This will make a huge difference

Because you're not asking Antigravity to build "a token optimizer."

You're asking it to build:

an enterprise control plane that intelligently decides how much work an LLM, agent, tool, context engine, and inference system actually needs to perform.

That is a much larger—and much more defensible—architecture.

And your existing `problemStatement.txt` is suitable as the **top-level source of truth** for starting this process.

User Prompt for Generate Problem Statement-6.pdf

c9ea3590-2236-47e6-8cb6-f0e1cd1417d5.md

File

This MD file is generated based on the problem statement. Now shall I go ahead with generating one by one document based on this like below:

1. architecture.md
 2. implementation-plan.md
 3. conventions.md
 4. edge-cases.md
 5. eval.md
 6. SCALING.md
 7. security.md
 8. observability.md
 9. optimization-catalog.md
 10. provider-matrix.md
 11. cache-strategy.md
 12. agent-optimization.md
 13. inference-optimization.md
 14. quality-gates.md
 15. ADRs
- from antigravity

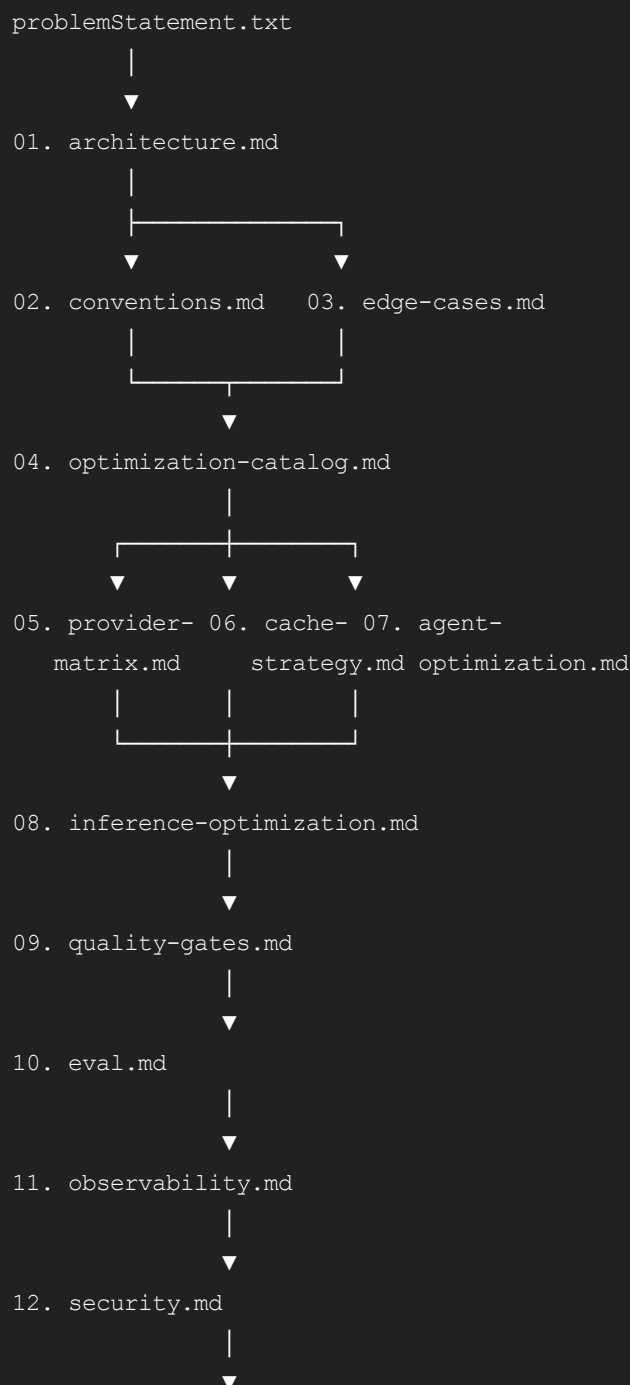
Worked for 4s

Yes. **Go ahead with generating those documents from Antigravity**, but I recommend one important change: **do not generate and implement them completely one-by-one in arbitrary order**. Your current MD specification is already broad enough that the documents need to have clear dependencies and a common source of truth. The MD contains the P0–P5 implementation priorities, 38 acceptance criteria, deployment/rollout model, cost/quality objectives, and the control-plane architecture, so it is a good requirements baseline.

c9ea3590-2236-47e6-8cb6-f0e1cd1...

Recommended order

I would ask Antigravity to generate them in this order:



13. SCALING.md

|



14. implementation-plan.md

|



15. ADRs/

|



IMPLEMENTATION

Why I'd put `implementation-plan.md` later

This is the one change I'd make to your original list.

You don't want Antigravity to create the implementation plan before it has fully understood:

- architecture
- optimization modules
- provider differences
- caching
- agent behavior
- inference layer
- security
- evaluation
- scaling
- quality gates

Your existing specification explicitly distinguishes P0 through P5, including infrastructure-dependent inference optimization, and says optimization features must pass validation before being enabled by default. `c9ea3590-2236-47e6-8cb6-f0e1cd1...`

So the **implementation plan should be generated after the technical design documents**, not before them.

One more document I strongly recommend

Add:

`requirements-traceability.md`

This is extremely valuable for a project of this size.

It should map:

Problem Statement Requirement



Architecture Component



Optimization Module



Implementation Phase



Acceptance Criterion



Evaluation Test



Observability Metric



Production Status

For example:

AC-019

"Skip optimization when benefit < overhead"



Optimization Decision Engine



Cost-of-Optimization Controller



Phase 1 / Control Plane



Evaluation:

Net Savings > 0



Metrics:

optimizer_cost

gross_savings

net_savings

optimization_latency



Production:

Validated / Experimental / Disabled

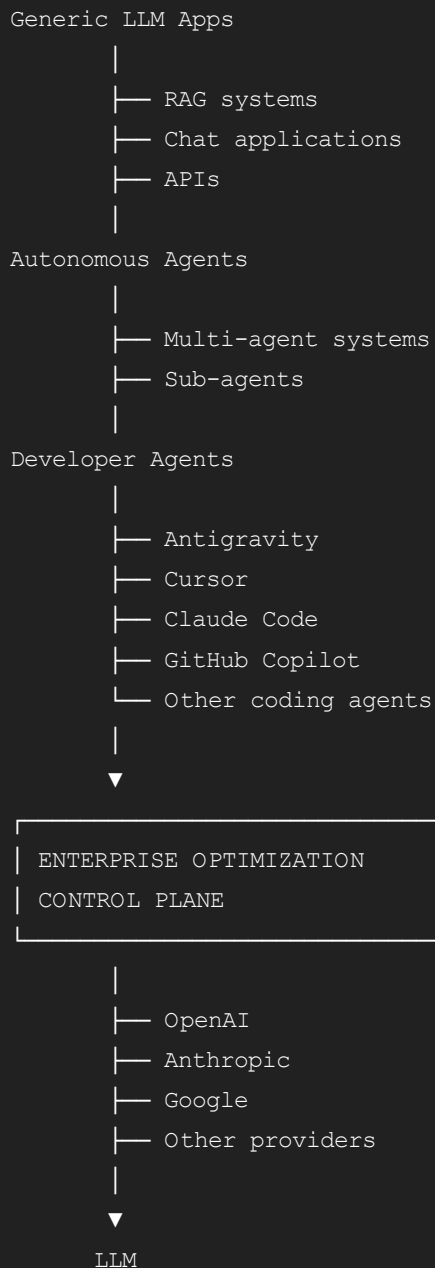
Your current MD already has the acceptance criteria and implementation priorities, so this traceability document can become the **bridge between requirements and actual implementation**.

c9ea3590-2236-47e6-8cb6-f0e1cd1...

I would also create an `interfaces.md`

Because your goal isn't just a standalone optimizer.

You specifically want it to sit underneath:



`interfaces.md` should define the contracts between:

- application → optimizer
- agent → optimizer
- optimizer → provider

- optimizer → cache
- optimizer → retrieval
- optimizer → tools
- optimizer → evaluation
- optimizer → observability
- optimizer → policy engine

This will prevent Antigravity from creating tightly coupled modules.

So my final documentation set would be

```
docs/
|
├─ architecture.md
├─ interfaces.md
├─ conventions.md
├─ edge-cases.md
|
├─ optimization-catalog.md
├─ provider-matrix.md
├─ cache-strategy.md
├─ agent-optimization.md
├─ inference-optimization.md
|
├─ quality-gates.md
├─ eval.md
├─ observability.md
├─ security.md
├─ SCALING.md
|
├─ requirements-traceability.md
├─ implementation-plan.md
|
└─ ADR/
    ├─ ADR-001-...
    ├─ ADR-002-...
    └─ ...
```

And keep this hierarchy

`problemStatement.txt` = WHAT

`architecture.md` = HOW the system is structured

`interfaces.md` = HOW components communicate

`optimization-catalog.md` = WHAT optimization capabilities exist

`quality-gates.md` = WHAT must be true before optimization is trusted

`eval.md` = HOW we prove it works

`observability.md` = HOW we measure it in production

`security.md` = HOW we protect it

`SCALING.md` = HOW we operate it at enterprise scale

`implementation-plan.md` = IN WHAT ORDER we build it

`ADRs` = WHY important architectural decisions were made

`requirements-traceability.md` = HOW we prove nothing was forgotten

Most important: give Antigravity a document-generation prompt

I would **not** simply ask:

"Generate architecture.md."

Instead, give Antigravity a master instruction that forces consistency across all documents.

Something like:

Analyze the complete `problemStatement.txt` and generate the engineering specification documents before writing implementation code. Treat `problemStatement.txt` as the authoritative requirements source. Do not remove, simplify, or silently reinterpret requirements. Every optimization technique must have documented applicability, prerequisites, expected benefit, optimization overhead, quality risks, fallback, evaluation methodology, observability metrics, and production maturity status. Cross-reference requirements across all documents and maintain terminology consistently. Do not implement production code until the documentation set has passed an internal consistency and requirements-traceability review.

Then have it generate the documents in the dependency order above.

And impose one hard rule on Antigravity

Do not implement P5 inference optimization merely because it is specified.

Your current specification correctly marks KV-cache optimization, continuous batching, speculative decoding, quantization, prefill/decode optimization, inference scheduling, and

GPU optimization as **infrastructure-dependent and optional**.

c9ea3590-2236-47e6-8cb6-f0e1cd1...

That distinction should remain.

The first production version should establish the control plane around:

**measure → decide → reduce → reuse → retrieve → compress → route → constrain
→ verify → stop → learn → govern**

and only then extend downward into inference-serving optimization.

So yes: you're at the right stage to move from the master MD specification into Antigravity-generated engineering documents. I would just use the dependency-aware order above and add `interfaces.md` + `requirements-traceability.md`. Those two additions will substantially reduce architectural drift when Antigravity starts implementing this very large system.

Sources

User Prompt for Generate Problem Statement-5.pdf

8f22a8ac-f898-48b6-9803-3b7e5562492c.md

File

architecture.md file generated. going for next file interfaces.md

Yes. Go for `interfaces.md` next.

At this stage, I recommend making `interfaces.md` a **contract document**, not another architecture description. It should define exactly how the Control Plane communicates with applications, agents, coding agents, LLM providers, caches, retrieval systems, tools, evaluators, and observability systems.

For Antigravity, use a prompt along these lines:

 Edit



Analyze the existing `problemStatement.txt` and the newly generated `architecture.md`.

Now generate `interfaces.md` for the **Enterprise Agent & LLM Inference Optimization Control Plane**.

IMPORTANT:

- Treat `problemStatement.txt` as the authoritative requirements source.
- Treat `architecture.md` as the authoritative architectural structure.
- Do NOT redesign or contradict the architecture.
- Do NOT implement application code yet.
- This document must define contracts and integration boundaries between components.
- The framework must remain provider-neutral and agent-neutral.
- The interfaces must support generic LLM applications, autonomous agents, developer/coding agents, multi-agent systems, sub-agents, RAG applications, MCP/tool-enabled agents, and inference-serving infrastructure where available.

The objective of `interfaces.md` is to answer:

"How does every major component of the optimization control plane communicate with every other component, and what contract must each integration satisfy?"

Cover the following comprehensively.

1. INTERFACE DESIGN PRINCIPLES

- Provider neutrality
- Agent/framework neutrality
- Versioned contracts
- Backward compatibility
- Idempotency
- Determinism where required
- Fail-open/fail-safe behavior
- Timeout and cancellation
- Retry semantics
- Correlation IDs
- Trace propagation
- Tenant isolation
- Security boundaries
- Policy enforcement
- Observability requirements

2. EXTERNAL ENTRYPOINT INTERFACE

Define the canonical request entering the Control Plane.

Include fields such as:

- request_id
- correlation_id
- tenant_id
- application_id
- agent_id
- session_id
- workflow_id
- task_id
- parent_task_id
- request_type
- user_input
- system_context
- developer_context
- conversation_history
- available_tools
- memory references
- retrieved context
- model preferences
- quality requirements

- latency requirements
- budget constraints
- security classification
- compliance requirements
- freshness requirements
- metadata

Clearly distinguish required, optional, derived, and sensitive fields.

3. OPTIMIZATION DECISION INTERFACE

Define the contract between the incoming request and the Optimization Decision Engine.

The decision interface must support decisions such as:

- bypass LLM
- exact-cache lookup
- semantic-cache lookup
- context retrieval
- context pruning
- deduplication
- compression
- context reordering
- memory retrieval
- tool selection
- tool suppression
- dynamic tool loading
- sub-agent creation
- sub-agent suppression
- model routing
- model cascade
- reasoning budget
- output budget
- batching
- early termination
- validation
- fallback
- escalation

Define:

- decision input

- decision output
- confidence
- expected benefit
- estimated optimization cost
- expected net savings
- expected quality impact
- expected latency impact
- policy constraints
- selected optimization stages
- skipped stages
- decision rationale
- fallback strategy

4. OPTIMIZATION MODULE CONTRACT

Define a common interface/protocol that every optimization module must implement.

Each module should expose concepts equivalent to:

- module_id
- module_version
- capability
- applicability
- prerequisites
- estimate()
- optimize()
- validate()
- rollback()/restore()
- explain()
- measure()
- health_check()

Define the lifecycle:

DISCOVER

→ EVALUATE

→ DECIDE

→ EXECUTE

→ VALIDATE

→ ACCEPT / REJECT

→ MEASURE

→ LEARN

Every optimization module must report:

- input tokens
- output tokens
- tokens avoided
- optimization overhead
- latency overhead
- estimated cost
- actual cost
- quality impact
- errors
- fallback status

5. CONTEXT ENGINE INTERFACES

Define interfaces for:

- context retrieval
- context ranking
- context pruning
- deduplication
- compression
- summarization
- context reordering
- context budgeting
- freshness scoring
- context utility scoring
- dependency graph
- reversible transformations
- hierarchical memory
- progressive compaction

Every context item should support provenance and, where applicable:

- source
- source version
- commit
- timestamp
- freshness

- relevance
- utility
- token cost
- confidence
- dependency references
- recovery reference

6. CACHE INTERFACES

Define a provider-neutral cache abstraction supporting:

- exact cache
- semantic cache
- provider-native prompt cache
- application cache
- context cache
- tool-result cache
- embedding cache

Define:

- lookup
- write
- invalidate
- refresh
- revalidate
- warm
- inspect
- explain miss

Include cache metadata:

- key
- namespace
- tenant
- version
- TTL
- freshness
- cost
- reuse probability
- dependency references
- security classification

Define cache safety requirements for:

- mutable data
- permission-sensitive data
- tenant-specific data
- branch-specific coding context
- provider-specific cache behavior

7. LLM PROVIDER INTERFACE

Define a normalized provider abstraction.

It must support:

- model invocation
- streaming
- structured output
- tool calling
- embeddings where applicable
- batch inference where applicable
- prompt caching where applicable
- reasoning/effort controls where applicable
- token accounting
- usage accounting
- cost calculation
- timeout
- cancellation
- retries

Provider adapters must expose capability metadata rather than forcing the core system to contain provider-specific assumptions.

8. MODEL PROFILE INTERFACE

Define the normalized model capability/economics profile.

Include:

- model ID
- provider
- context limit
- output limit
- input price

- output price
- cached-input price
- cache write price where applicable
- reasoning controls
- tool support
- structured output
- batch support
- latency characteristics
- reliability characteristics
- compliance attributes
- supported regions
- capability tags
- version
- effective date

9. MODEL ROUTING INTERFACE

Define how the Optimization Decision Engine requests model selection.

Inputs:

- task complexity
- quality requirement
- latency requirement
- budget
- context size
- tool requirements
- reasoning requirements
- historical outcome
- model availability

Outputs:

- selected model
- fallback models
- routing confidence
- estimated cost
- estimated latency
- expected quality
- routing rationale

10. TOOL INTERFACES

Define provider-neutral interfaces for:

- tool discovery
- tool selection
- tool loading
- tool invocation
- argument optimization
- result filtering
- result compression
- result caching
- parallel execution
- programmatic execution
- error handling

Every tool invocation should support:

- tool_id
- version
- arguments
- authorization context
- timeout
- retry policy
- idempotency
- expected information gain
- estimated cost
- estimated latency
- result schema

11. AGENT INTERFACE

Define the Control Plane contract for autonomous agents.

Support:

- agent registration
- task submission
- state inspection
- context optimization
- tool optimization
- iteration tracking
- progress measurement
- early termination
- recovery

- checkpointing

Track:

- iteration
- objective
- state
- progress
- tools called
- tokens consumed
- cost
- information gained
- failures
- next action

12. SUB-AGENT INTERFACE

Define contracts for:

- spawn request
- spawn decision
- delegated task
- context handoff
- result handoff
- aggregation
- cancellation
- termination

The interface must explicitly account for:

- parent-agent cost
- child-agent cost
- duplicated context cost
- aggregation cost
- coordination latency
- expected value

13. DEVELOPER / CODING AGENT INTERFACE

Define a dedicated integration contract for coding agents such as:

- Antigravity
- Cursor
- Claude Code

- GitHub Copilot
- custom coding agents
- MCP-enabled development agents

Support context types such as:

- repository
- branch
- commit
- Git diff
- files
- symbols
- functions
- classes
- imports
- dependencies
- tests
- compiler errors
- terminal output
- build output
- issue/task information

The interface must support:

- repository-aware context optimization
- dependency-aware retrieval
- symbol-aware retrieval
- diff-aware context
- terminal-output compression
- test-output compression
- code-aware summarization
- coding-agent loop optimization
- sub-agent handoffs

14. MCP / TOOL ECOSYSTEM INTERFACE

Define an integration abstraction for MCP and similar tool ecosystems.

Support:

- server discovery
- capability discovery

- tool discovery
- resource discovery
- selective tool exposure
- tool schema minimization
- invocation
- result filtering
- security policy enforcement

The core optimizer must not depend directly on one specific MCP implementation.

15. RAG INTERFACE

Define contracts for:

- query rewriting
- retrieval
- reranking
- chunk selection
- context assembly
- citation/provenance preservation
- freshness validation
- context compression

16. MEMORY INTERFACE

Define interfaces for:

- short-term memory
- task memory
- session memory
- project memory
- organizational memory
- historical memory

Include:

- store
- retrieve
- update
- invalidate
- compress
- summarize

- expire
- restore

17. QUALITY VALIDATION INTERFACE

Define a normalized validation contract.

Validators may include:

- schema validation
- deterministic rules
- unit tests
- integration tests
- type checking
- static analysis
- citation validation
- business rules
- security checks
- model-based evaluation

Output must include:

- pass/fail
- score
- confidence
- violations
- evidence
- recommended fallback

18. EVALUATION INTERFACE

Define how optimization modules expose data to the evaluation framework.

Required measurements:

- baseline cost
- optimized cost
- gross savings
- optimizer overhead
- net savings
- quality delta
- latency delta
- cache impact

- task success
- regression status

19. OBSERVABILITY INTERFACE

Define standardized:

- logs
- metrics
- traces
- events
- spans
- audit records

Every request and optimization decision must be traceable end-to-end.

Support correlation:

request

- optimization decision
- context operation
- cache operation
- tool operation
- model routing
- LLM call
- validation
- final outcome

20. POLICY / GOVERNANCE INTERFACE

Define interfaces for:

- optimization policies
- tenant policies
- model allowlists
- model denylists
- data policies
- token budgets
- cost budgets
- latency SLOs
- quality thresholds
- security policies

- compliance policies
- rollout policies

21. SECURITY INTERFACES

Define contracts for:

- authentication
- authorization
- tenant isolation
- secret handling
- PII/sensitive-data classification
- cache isolation
- audit logging
- data residency
- policy enforcement
- provider credential isolation

22. FALLBACK / RECOVERY INTERFACE

Every optimization must support controlled fallback.

Define:

- fallback trigger
- previous representation
- restoration mechanism
- retry policy
- escalation policy
- original-context recovery
- model escalation
- optimization disablement

23. EVENT INTERFACE

Define normalized events such as:

REQUEST_RECEIVED

OPTIMIZATION_STARTED

OPTIMIZATION_SKIPPED

CACHE_HIT

CACHE_MISS

CONTEXT_REDUCED

CONTEXT_COMPRESSED

TOOL_SELECTED
TOOL_SKIPPED
SUBAGENT_CREATED
SUBAGENT_SKIPPED
MODEL_ROUTED
MODEL_ESCALATED
VALIDATION_FAILED
FALLBACK_EXECUTED
TASK_COMPLETED
OPTIMIZATION_REGRESSION
POLICY_CHANGED

24. VERSIONING AND COMPATIBILITY

Define:

- API versioning
- schema versioning
- module versioning
- provider adapter versioning
- model profile versioning
- policy versioning
- backward compatibility
- migration strategy

25. FAILURE SEMANTICS

Define behavior for:

- provider unavailable
- provider timeout
- cache unavailable
- retrieval failure
- compression failure
- tool failure
- validator failure
- malformed model response
- quota exhaustion
- rate limiting
- context overflow
- stale cache
- inconsistent repository state

- agent loop failure

The Control Plane must fail safely and must never silently corrupt context.

26. IDEMPOTENCY / RETRY / CANCELLATION

Explicitly define semantics for:

- LLM retries
- tool retries
- cache writes
- cache invalidation
- agent spawning
- sub-agent spawning
- asynchronous jobs
- batch jobs

Avoid duplicate side effects.

27. TENANCY AND ISOLATION

Define how all interfaces carry:

- tenant
- organization
- application
- user
- agent
- session

Ensure cache, memory, policy, telemetry, and cost data cannot cross tenant boundaries.

28. COST ACCOUNTING INTERFACE

Every billable or optimization operation must produce normalized accounting.

Track:

- provider
- model
- input tokens

- output tokens
- cached tokens
- reasoning tokens where available
- optimizer tokens
- embedding tokens
- tool cost
- infrastructure cost
- total estimated cost
- actual cost
- currency
- pricing version

29. DECISION EXPLANATION INTERFACE

Every significant optimization decision should be explainable.

For example:

Decision:

SKIP COMPRESSION

Reason:

Context utility is high and compression overhead is greater than expected token savings.

Expected savings:

\$0.003

Optimization cost:

\$0.004

Decision:

SKIP

This explanation must be available for debugging, audit, and evaluation.

30. CONTRACT TESTING

Define contract tests for every interface.

Tests must cover:

- valid request

- invalid request
- missing fields
- version mismatch
- timeout
- retry
- cancellation
- failure
- fallback
- security violation
- tenant isolation
- schema compatibility

31. NON-FUNCTIONAL INTERFACE REQUIREMENTS

Define requirements for:

- latency
- throughput
- availability
- scalability
- consistency
- durability
- security
- observability
- backward compatibility

32. REFERENCE DATA FLOW

Provide canonical sequence diagrams/textual flows for at least:

- A. Simple LLM request
- B. Cached request
- C. RAG request
- D. Tool-enabled agent
- E. Multi-agent request
- F. Developer/coding agent
- G. Failed optimization with fallback
- H. Model escalation
- I. Context overflow
- J. Provider failure
- K. Batch request
- L. Long-running agent session

33. INTERFACE ANTI-PATTERNS

Explicitly prohibit:

- provider-specific logic in core modules
- direct coupling between optimizer modules
- unversioned contracts
- hidden token/cost transformations
- silent context deletion
- untraceable optimization decisions
- cross-tenant cache access
- retries without idempotency
- optimization without fallback
- tool execution without authorization
- model routing without capability validation

34. TRACEABILITY

For every major interface, reference the corresponding requirement(s) from

`problemStatement.txt` and architectural component(s) from `architecture.md`.

Do not invent requirements that contradict either document.

35. IMPLEMENTATION BOUNDARY

Clearly distinguish:

MUST IMPLEMENT

SHOULD IMPLEMENT

OPTIONAL

INFRASTRUCTURE-DEPENDENT

FUTURE

Do not prematurely require infrastructure-dependent inference features where the deployment does not support them.

36. FINAL REVIEW

Before finalizing `interfaces.md`, perform an internal consistency check:

- Does every major architecture component have an interface?
- Does every major problem-statement capability have an integration contract?
- Are provider-specific concerns isolated?
- Are agent and coding-agent integrations covered?
- Are MCP/tool integrations covered?
- Are cache and memory contracts covered?
- Are quality and evaluation contracts covered?
- Are security and tenancy boundaries covered?
- Are cost-accounting interfaces covered?
- Are fallback and recovery mechanisms covered?
- Are versioning and compatibility covered?
- Are all optimization decisions observable and explainable?

At the end, include:

1. Interface inventory
2. Cross-reference matrix
3. Open architectural questions
4. Assumptions
5. Dependencies
6. Recommended next documentation artifact

Do not write implementation code. Generate only the complete

`interfaces.md`.

One important point

After Antigravity generates `interfaces.md`, **don't immediately ask it to implement it.**

Your next sequence should be:

`interfaces.md` → `conventions.md` → `edge-cases.md` → `optimization-catalog.md`

That will give you a much stronger foundation before you reach the implementation plan.

And when you eventually reach `implementation-plan.md`, Antigravity should be able to derive the implementation phases from **the problem statement + architecture + interfaces + optimization catalog + edge cases + evaluation/quality requirements**, rather than making architectural decisions while coding.

User Prompt for Generate Problem Statement-4.pdf

0d680a72-2ee7-4983-9a4f-01a4cfcd9af4.md

File

Interfaces.md file is now generated. going for the next file conventions.md

Worked for a few seconds

Yes. I found the generated `interfaces.md`, and it already has the right boundary: it defines **what components expose and what contracts they guarantee**, while deliberately leaving implementation technologies and deployment choices for later.

0d680a72-2ee7-4983-9a4f-01a4cfc...

So `conventions.md` **is the correct next document**.

For this project, I would make `conventions.md` much more than ordinary coding style. It should establish the **engineering laws** Antigravity must follow across the entire codebase—especially because you are building a control plane with optimization, agents, caching, provider adapters, security, evaluation, and inference layers.

Use this prompt in Antigravity:

 Edit



Analyze these authoritative documents before generating this file:

1. `problemStatement.txt`
2. `architecture.md`
3. `interfaces.md`

Generate a comprehensive `conventions.md` for the **Enterprise Agent & LLM Inference Optimization Control Plane**.

IMPORTANT:

- `problemStatement.txt` defines WHAT the product must achieve.
- `architecture.md` defines the architectural structure.
- `interfaces.md` defines component and integration contracts.
- This document defines the engineering conventions that ALL future implementation must follow.
- Do not redesign the architecture.
- Do not introduce implementation choices that contradict the existing documents.
- Do not write implementation code.
- Do not silently remove or simplify requirements.

- Prefer explicit, deterministic, testable, observable, reversible designs.
- The system must remain provider-neutral, model-neutral, agent-framework-neutral, and deployment-neutral wherever the architecture requires it.

The resulting document must be strong enough that multiple engineers or coding agents can implement different parts of the system without architectural drift.

1. PURPOSE

Define the engineering rules, coding conventions, architectural conventions, data conventions, API conventions, optimization conventions, testing conventions, security conventions, observability conventions, and documentation conventions that all implementation must follow.

The primary goals are:

- correctness
- maintainability
- extensibility
- provider neutrality
- agent neutrality
- deterministic behavior where required
- safe optimization
- explainability
- reversibility
- testability
- observability
- security
- enterprise scalability

2. CORE ENGINEERING PRINCIPLES

Establish mandatory principles including:

- Make tokens count instead of merely counting them.
- The optimizer may determine that no LLM call is necessary.
- Never optimize solely for token reduction.
- Optimize for net value.

- Preserve quality before maximizing savings.
- Prefer measurable decisions over heuristics without evidence.
- Fail safely.
- Every transformation must be explainable.
- Every destructive transformation must be reversible or explicitly justified.
- Every optimization must have a fallback.
- Every optimization must have measurable overhead.
- Every optimization must be independently evaluable.
- No hidden optimization.
- No silent context deletion.
- No provider-specific assumptions in core logic.
- No cross-tenant state leakage.
- No unbounded agent loops.
- No uncontrolled retry amplification.
- No optimization without quality gates.

3. SOURCE-OF-TRUTH HIERARCHY

Define precedence when documents conflict.

Recommended hierarchy:

1. Security and compliance constraints
2. `problemStatement.txt`
3. approved architectural decisions / ADRs
4. `architecture.md`
5. `interfaces.md`
6. `quality-gates.md`
7. `optimization-catalog.md`
8. implementation plan
9. implementation details

Explain how conflicts must be detected and resolved rather than silently overridden.

4. NAMING CONVENTIONS

Define conventions for:

- services
- modules

- packages
- classes
- interfaces
- functions
- variables
- constants
- configuration keys
- environment variables
- API fields
- event names
- metric names
- trace/span names
- database entities
- cache keys
- optimization IDs
- policy IDs
- model IDs
- provider IDs
- tenant IDs
- request IDs
- correlation IDs

Use stable identifiers such as:

OI-xxx

CL-xxx

CE-xxx

TE-xxx

AL-xxx

AR-xxx

QO-xxx

DA-xxx

where appropriate.

5. MODULE STRUCTURE CONVENTIONS

Define how the codebase should separate:

- domain logic
- application/orchestration logic
- infrastructure

- provider adapters
- optimization modules
- policy engine
- evaluation
- observability
- security
- persistence
- API layer

Prevent business/optimization logic from being coupled directly to infrastructure.

6. DEPENDENCY DIRECTION

Define mandatory dependency direction.

Prefer:

API

↓

Application

↓

Domain

↓

Ports/interfaces

↓

Infrastructure adapters

Provider implementations must depend on provider-neutral interfaces.

Core optimization logic must NOT depend directly on:

- OpenAI SDK
- Anthropic SDK
- Google SDK
- specific vector database
- specific cache technology
- specific agent framework
- specific MCP implementation
- specific cloud provider

7. INTERFACE-FIRST DEVELOPMENT

Every significant component must begin with:

- contract
- interface
- input schema
- output schema
- error semantics
- observability requirements
- test contract

Implementation follows the contract.

Avoid leaking internal implementation details through public interfaces.

8. TYPE AND SCHEMA CONVENTIONS

Define strict schemas for:

- request
- response
- optimization decision
- optimization result
- context item
- cache entry
- tool invocation
- agent state
- sub-agent handoff
- model profile
- provider capability
- validation result
- cost record
- evaluation result
- policy
- event

Prefer explicit schemas over untyped dictionaries/maps.

Define required vs optional fields.

Define schema evolution rules.

9. IMMUTABILITY AND DATA OWNERSHIP

Define which objects are:

- immutable
- mutable
- append-only
- versioned
- ephemeral

Require immutable audit records.

Prefer immutable optimization decisions and event records.

Avoid shared mutable state.

Define ownership boundaries for:

- request state
- context
- cache
- memory
- agent state
- tenant state

10. OPTIMIZATION MODULE CONVENTIONS

Every optimization module must follow the same lifecycle:

DISCOVER

→ ESTIMATE

→ DECIDE

→ EXECUTE

→ VALIDATE

→ ACCEPT/REJECT

→ MEASURE

→ LEARN

Every optimization module must expose equivalent capabilities for:

- applicability
- cost estimation
- optimization
- validation
- rollback/recovery

- explanation
- measurement
- health

Every module must report:

- input tokens
- output tokens
- tokens avoided
- optimization compute
- optimization latency
- provider cost
- tool cost
- quality delta
- net savings
- fallback status

11. COST-OF-OPTIMIZATION CONVENTION

Never report gross token savings without accounting for optimizer overhead.

Use the conceptual calculation:

Net Value =

Inference Cost Avoided

- Optimization Compute Cost
- Additional Provider Cost
- Additional Tool Cost
- Additional Infrastructure Cost
- Latency Cost
- Expected Quality/Risk Cost

Define mandatory terminology:

- gross savings
- optimizer cost
- net savings
- net savings percentage
- cost per successful task
- cost per quality-adjusted task

12. TOKEN ACCOUNTING CONVENTIONS

Every LLM-related operation should normalize token accounting where provider data permits.

Track:

- input tokens
- output tokens
- cached input
- uncached input
- reasoning tokens
- tool-related context
- optimizer-generated tokens
- compression tokens
- summarization tokens
- embedding tokens

Never assume all providers expose identical token categories.

Use:

AVAILABLE

NOT_SUPPORTED

ESTIMATED

UNKNOWN

rather than fabricating values.

13. COST ACCOUNTING CONVENTIONS

All costs must be associated with:

- provider
- model
- pricing version
- request
- tenant
- application
- agent
- workflow
- task

Never hard-code prices into business logic.

Pricing must be versioned and externally configurable.

14. PROVIDER ABSTRACTION CONVENTIONS

Provider adapters must translate provider-specific capabilities into normalized internal contracts.

Provider-specific features must be represented through capability discovery.

Never assume:

- identical tokenization
- identical caching semantics
- identical context limits
- identical pricing
- identical reasoning controls
- identical tool behavior
- identical structured-output behavior

15. MODEL PROFILE CONVENTIONS

Model profiles must be versioned.

Separate:

CAPABILITY

from

ECONOMICS

from

PERFORMANCE

from

POLICY ELIGIBILITY

Do not allow routing decisions to depend on undocumented model assumptions.

16. CONTEXT CONVENTIONS

Treat context as structured data, not an opaque string wherever practical.

Context items should preserve:

- provenance
- source
- version
- timestamp
- relevance
- utility
- freshness
- token cost
- security classification
- dependencies

Never silently delete context.

All context reduction must produce an explainable transformation record.

17. CONTEXT COMPRESSION CONVENTIONS

Compression must specify:

- source representation
- target representation
- compression method
- compression ratio
- information-loss estimate
- quality risk
- validation result
- recovery mechanism

Compression must not be treated as automatically beneficial.

18. CACHE CONVENTIONS

All cache keys must explicitly account for relevant dimensions such as:

- tenant
- namespace
- request semantics
- model/provider where required
- context version
- policy version

- security scope

Never create globally shared cache entries for sensitive or tenant-specific data.

All mutable cached data requires TTL/revalidation semantics.

Cache behavior must be observable.

19. CACHE INVALIDATION CONVENTIONS

Cache invalidation must be dependency-aware where dependencies exist.

Never invalidate by arbitrary global flushing unless explicitly required.

Track:

- dependency
- version
- TTL
- invalidation reason
- invalidation timestamp

20. AGENT CONVENTIONS

Every agent execution must have:

- explicit objective
- bounded lifecycle
- iteration counter
- token budget
- cost budget where applicable
- timeout
- cancellation mechanism
- progress measurement
- termination criteria

Agent loops must never be implicitly unbounded.

21. SUB-AGENT CONVENTIONS

Sub-agents must be treated as economic decisions.

Before spawning:

estimate:

- expected benefit
- context duplication cost
- model cost
- coordination cost
- aggregation cost
- latency

Avoid spawning sub-agents merely because parallelism is available.

22. TOOL CONVENTIONS

Every tool call must have:

- authorization
- timeout
- retry semantics
- idempotency classification
- version
- input validation
- output validation
- observability

Tool schemas must be minimized where possible.

Tool results must not automatically become permanent context.

23. DEVELOPER/CODING AGENT CONVENTIONS

Coding-agent context must be structured around:

- repository
- branch
- commit
- changed files
- symbols
- dependencies

- tests
- build state
- terminal output
- issue/task context

Prefer targeted retrieval over full repository replay.

Git state must be treated as versioned context.

Terminal and test output must be bounded and compressible.

24. MCP CONVENTIONS

MCP integrations must remain behind provider-neutral/tool-neutral interfaces.

Do not couple core optimization logic directly to one MCP implementation.

Support selective exposure of tools/resources.

Treat tool schemas themselves as context and optimize them where safe.

25. ERROR HANDLING

Define standardized error categories:

- validation error
- policy error
- authorization error
- provider error
- timeout
- quota/rate-limit
- cache error
- retrieval error
- tool error
- optimization error
- quality-gate failure
- infrastructure error
- unknown error

Errors must be:

- typed
- observable
- actionable
- non-sensitive
- traceable

26. FAILURE AND FALLBACK CONVENTIONS

Every optimization path must define:

- failure condition
- fallback
- recovery representation
- escalation path
- retry behavior
- telemetry

The original safe request/context must remain recoverable when required.

27. RETRY CONVENTIONS

Never blindly retry.

Every retry must specify:

- retryable error
- maximum attempts
- backoff
- jitter
- idempotency requirement
- budget impact
- cancellation behavior

Prevent retry storms and cost amplification.

28. TIMEOUT AND CANCELLATION

Every external operation must have explicit timeout semantics.

Support cancellation propagation:

request
→ optimizer
→ retrieval
→ cache
→ tool
→ model
→ validation

Do not allow orphaned expensive operations.

29. CONCURRENCY CONVENTIONS

Define safe concurrency rules for:

- tool calls
- retrieval
- cache operations
- sub-agents
- model requests

Only parallelize operations that are independent and economically justified.

Prevent duplicate work caused by race conditions.

30. LOGGING CONVENTIONS

Logs must be:

- structured
- machine-readable
- correlation-aware
- tenant-aware
- privacy-safe

Never log:

- raw secrets
- credentials
- authorization tokens
- unnecessary sensitive user content

Avoid logging entire prompts by default.

Use references/ hashes/ redacted representations where appropriate.

31. METRICS CONVENTIONS

Standardize metric naming and dimensions.

Required metric families should include:

- request
- token
- cost
- optimization
- cache
- retrieval
- tool
- agent
- model
- quality
- latency
- error
- fallback

Avoid high-cardinality labels such as raw prompts.

32. TRACING CONVENTIONS

Trace the full lifecycle:

request
→ decision
→ optimization
→ cache
→ retrieval
→ tools
→ routing
→ model
→ validation
→ outcome

Each optimization must have a trace/span.

33. SECURITY CONVENTIONS

Security must be enforced by default.

Define conventions for:

- authentication
- authorization
- tenant isolation
- secret management
- encryption
- data classification
- sensitive context
- cache isolation
- provider credentials
- audit

Never trust caller-provided tenant identity without verification.

34. DATA PRIVACY CONVENTIONS

Minimize retention.

Separate:

- operational metadata
- prompts
- outputs
- sensitive data
- audit records

Define configurable retention.

Support redaction and pseudonymization.

35. AUDIT CONVENTIONS

Optimization decisions affecting production behavior must be auditable.

Audit records must identify:

- who/what initiated
- policy version
- optimization version
- model/provider

- decision
- reason
- result
- timestamp
- correlation ID

Audit records must be append-only/immutable.

36. CONFIGURATION CONVENTIONS

Separate:

- static configuration
- dynamic configuration
- tenant configuration
- policy configuration
- secrets

Never hard-code:

- API keys
- provider credentials
- pricing
- model availability
- tenant policies

Configuration changes must be versioned and auditable.

37. FEATURE FLAGS AND ROLLOUT

All experimental or high-risk optimization techniques should support:

- disabled
- shadow
- canary
- percentage rollout
- tenant rollout
- workload rollout
- full rollout
- rollback

Default state for unvalidated optimizations should be conservative.

38. EXPERIMENTATION CONVENTIONS

Every optimization experiment must define:

- hypothesis
- baseline
- treatment
- metrics
- sample
- quality threshold
- success criteria
- rollback criteria

Avoid declaring success from token reduction alone.

39. QUALITY GATE CONVENTIONS

An optimization may be promoted only if:

- quality remains within policy threshold
- correctness remains acceptable
- security remains acceptable
- net savings are positive where savings are expected
- latency remains within SLO
- reliability remains acceptable

40. TESTING CONVENTIONS

Require multiple levels:

- unit tests
- contract tests
- integration tests
- provider adapter tests
- optimization tests
- property-based tests where useful
- regression tests
- benchmark tests
- security tests
- load tests
- chaos/failure tests
- end-to-end tests

41. OPTIMIZATION-SPECIFIC TESTING

For every optimization module test:

1. baseline
2. optimized representation
3. semantic preservation
4. quality
5. token impact
6. cost impact
7. latency impact
8. fallback
9. rollback
10. failure modes

42. DETERMINISM

Where deterministic behavior is required:

- avoid uncontrolled randomness
- version algorithms
- version policies
- version model profiles
- record relevant decision inputs

Where stochastic model behavior is unavoidable, capture sufficient metadata for reproducibility.

43. VERSIONING

Version:

- APIs
- schemas
- optimization modules
- policies
- model profiles
- pricing
- provider adapters
- evaluation datasets
- benchmarks

Never silently change behavior under the same semantic version where compatibility matters.

44. BACKWARD COMPATIBILITY

New versions should preserve compatibility unless a breaking change is explicitly documented and migrated.

Define:

- deprecation
- migration
- compatibility window
- schema evolution
- rollback

45. RESOURCE BUDGETING

Every potentially expensive operation should support budgets for:

- tokens
- money
- latency
- compute
- tool calls
- model calls
- agent iterations
- sub-agent count

Budgets must propagate through nested workflows.

46. COST-SAFETY CONVENTIONS

The system must protect against:

- infinite loops
- recursive sub-agent spawning
- retry amplification
- cache stampedes
- model escalation storms
- tool-call storms
- context explosion
- expensive optimizer recursion

47. RECURSION / NESTED OPTIMIZATION

Prevent an optimizer from recursively optimizing its own optimization requests without explicit boundaries.

Define optimization depth and ownership.

48. EXPLAINABILITY

Every significant decision should be explainable in structured form.

Examples:

- why context was removed
- why compression was skipped
- why a model was selected
- why a tool was skipped
- why a sub-agent was not created
- why cache reuse was rejected
- why escalation occurred

49. NO SILENT MAGIC

Avoid hidden behavior.

If the optimizer changes:

- context
- model
- tool set
- reasoning budget
- output budget
- workflow path

the decision must be represented in telemetry and, where appropriate, surfaced to debugging/audit systems.

50. OBSERVABILITY-FIRST DEVELOPMENT

A component is not considered production-ready unless its behavior can be measured.

Every significant component must define:

- logs
- metrics
- traces
- health checks
- failure events
- cost measurements

51. DOCUMENTATION CONVENTIONS

Every major component must document:

- purpose
- responsibility
- inputs
- outputs
- dependencies
- failure modes
- performance characteristics
- security considerations
- observability
- testing strategy

52. ADR CONVENTIONS

Architectural decisions that affect:

- provider abstraction
- storage
- caching
- optimization algorithms
- routing
- consistency
- security
- scaling
- inference architecture

must be captured in ADRs.

Each ADR must include:

- context
- problem
- options

- decision
- rationale
- consequences
- alternatives rejected
- migration implications

53. CODE REVIEW CONVENTIONS

Code review must verify:

- architecture alignment
- interface compliance
- security
- tenant isolation
- observability
- test coverage
- cost impact
- quality impact
- fallback
- performance
- backward compatibility

54. ANTI-PATTERNS

Explicitly document and prohibit:

- giant monolithic optimizer
- provider logic in domain layer
- hard-coded model prices
- hidden token transformations
- unbounded loops
- global mutable state
- silent context deletion
- unversioned policies
- untracked optimization decisions
- unobservable cache behavior
- retry without idempotency
- optimization without fallback
- quality-blind compression
- model routing without capability checks
- cross-tenant caching
- raw secret logging

- full repository replay by default
- full conversation replay by default
- blindly spawning sub-agents
- optimizing every request regardless of ROI

55. PERFORMANCE CONVENTIONS

Optimization infrastructure itself must be lightweight.

Define targets for:

- decision latency
- context-processing latency
- cache lookup latency
- routing latency
- telemetry overhead

The optimizer must not become the primary bottleneck.

56. SCALABILITY CONVENTIONS

Design components to support horizontal scaling where appropriate.

Avoid:

- process-local assumptions
- local-only state
- global locks
- single-node dependencies

Clearly identify stateful vs stateless components.

57. MULTI-TENANCY

Every request and stateful object must have explicit tenant ownership.

Enforce isolation for:

- cache
- memory
- policies
- metrics
- cost

- audit
- configuration

58. FINOPS CONVENTIONS

Cost must be attributable to:

- tenant
- application
- workflow
- agent
- task
- model
- provider
- optimization module

Support cost allocation and optimization ROI analysis.

59. AI-GENERATED CODE CONVENTIONS

Because coding agents may implement this system:

- generated code must follow these conventions
- agents must not invent new architectural patterns without justification
- agents must inspect existing interfaces before implementation
- agents must update tests with implementation changes
- agents must update documentation when contracts change
- agents must create ADRs for significant architectural changes
- agents must not bypass quality gates

60. IMPLEMENTATION DISCIPLINE

Require this sequence for significant features:

REQUIREMENT

→ INTERFACE

→ DESIGN

→ TEST PLAN

→ IMPLEMENTATION

→ VALIDATION

→ OBSERVABILITY

→ DOCUMENTATION

→ REVIEW

Do not begin with implementation code.

61. DEFINITION OF DONE

A feature is complete only when:

- requirement traced
- interface defined
- implementation complete
- unit tests complete
- integration/contract tests complete
- quality checks complete
- security checks complete
- observability complete
- fallback defined
- documentation updated
- evaluation added where applicable
- performance impact understood
- cost impact understood
- ADR created if required

62. TRACEABILITY REQUIREMENT

Every major implementation component must be traceable to:

problem statement

→ architecture

→ interface

→ optimization catalog

→ implementation plan

→ tests

→ evaluation

→ observability

63. FINAL CONSISTENCY REVIEW

Before finalizing `conventions.md`, verify:

- No convention contradicts `problemStatement.txt`.
- No convention contradicts `architecture.md`.

- No convention contradicts `interfaces.md`.
- Provider neutrality is preserved.
- Agent neutrality is preserved.
- Coding-agent integration is preserved.
- Optimization safety is preserved.
- Cost accounting is preserved.
- Quality gates are preserved.
- Security requirements are preserved.
- Multi-tenancy is preserved.
- Observability is mandatory.
- Every optimization remains measurable and reversible where required.
- The optimizer itself cannot become an uncontrolled source of cost.

At the end include:

1. Convention summary
2. Mandatory vs recommended conventions
3. Engineering invariants
4. Prohibited anti-patterns
5. Cross-document consistency matrix
6. Open questions
7. Recommended next document

Generate ONLY `conventions.md`. Do not implement code.

After that

Once `conventions.md` is generated, **send me the file/result before moving forward**. I can check whether Antigravity introduced contradictions or overly restrictive conventions.

Then we'll move to:

`edge-cases.md` → `optimization-catalog.md`

Those two are especially important for your project because they determine whether the eventual optimizer behaves intelligently under pathological conditions rather than just on ideal benchmark prompts.

User Prompt for Generate Problem Statement-3.pdf

1ff56d55-1dea-4937-81ad-bfd49fc58c4a.md

File

conventions.md file is generated. Now going for edge-cases.md file.

Worked for 6s

Perfect. I checked the generated `conventions.md` against the existing `problemStatement`, `architecture.md`, and `interfaces.md`. It is aligned with the intended control-plane design and, importantly, establishes the engineering rules around the optimization lifecycle, cost accounting, quality validation, provider neutrality, agent/coding-agent behavior, and traceability. `1ff56d55-1dea-4937-81ad-bfd49fc...`

`edge-cases.md` **is the correct next document.**

For this project, I would make it unusually comprehensive. Edge cases are where an aggressive optimizer can accidentally **increase cost, corrupt context, leak data, degrade quality, or create agent loops.**

Use this prompt in Antigravity:

 Edit



Analyze the following authoritative documents before generating this file:

1. `problemStatement.txt`
2. `architecture.md`
3. `interfaces.md`
4. `conventions.md`

Generate a comprehensive `edge-cases.md` for the **Enterprise Agent & LLM Inference Optimization Control Plane.**

IMPORTANT:

- Treat `problemStatement.txt` as the ultimate product requirements source.
- Treat `architecture.md` as the architectural source of truth.
- Treat `interfaces.md` as the contract source of truth.
- Treat `conventions.md` as the mandatory engineering rules.
- Do NOT redesign these documents.
- Do NOT implement code.
- Do NOT silently remove requirements.
- Do NOT assume an optimization is safe merely because it reduces tokens.

- Every edge case must define detection, expected behavior, safety implications, fallback/recovery, observability, and testing requirements where applicable.
- The optimizer must fail safely.
- The application must not become unavailable merely because an optimization fails unless explicitly required by policy.
- Never fabricate token, cost, quality, or savings measurements when provider data is unavailable.
- Preserve security, authorization, tenant isolation, correctness, freshness, provenance, and auditability.

The purpose of this document is to create the definitive catalogue of abnormal, adversarial, pathological, boundary, failure, and high-risk scenarios that the implementation and evaluation framework must handle.

For every edge case use this structure where applicable:

EDGE CASE ID

Category

Scenario

Trigger / Detection

Why It Matters

Expected Behavior

Optimization Behavior

Fallback / Recovery

Security Impact

Quality Impact

Cost Impact

Latency Impact

Observability

Test Requirement

Severity

Priority

Related Requirements

Related Interfaces

Related Components

Organize the document into the following sections.

1. EDGE-CASE PRINCIPLES

Define mandatory principles:

- Safety before savings
- Correctness before compression
- Security before reuse
- Freshness before cache hit
- Authorization before semantic similarity
- Net savings before optimization execution
- Bounded work
- Explicit fallback
- No silent degradation
- No fabricated measurements
- No uncontrolled recursion
- No uncontrolled retries
- No uncontrolled context growth
- No uncontrolled sub-agent spawning

2. REQUEST INGESTION EDGE CASES

Cover:

- empty request
- whitespace-only request
- extremely large request
- malformed request
- invalid schema
- missing required fields
- unknown request type
- unsupported capability
- conflicting constraints
- impossible token budget
- zero budget
- negative/invalid budget
- impossible latency requirement
- impossible quality requirement
- contradictory policy requirements
- missing tenant identity
- invalid tenant identity
- missing application identity
- duplicate request_id
- duplicate task_id

- replayed request
- concurrent duplicate requests
- cancellation during ingestion
- request mutation during processing

3. PROMPT / INSTRUCTION EDGE CASES

Cover:

- empty system prompt
- conflicting system/developer/user instructions
- duplicated instructions
- near-duplicate instructions
- contradictory instructions
- extremely long instructions
- security-critical instructions
- instructions that appear redundant but are actually required
- repeated policy text
- hidden delimiter problems
- malformed structured prompts
- prompt injection
- indirect prompt injection
- untrusted retrieved instructions
- untrusted tool output containing instructions
- instructions embedded in code
- instructions embedded in documents
- instruction hierarchy ambiguity

The optimizer must never remove or weaken higher-priority security/system/developer instructions.

4. CONTEXT EDGE CASES

Cover:

- context exactly at model limit
- context one token below limit
- context one token above limit
- context dramatically above limit
- empty context
- duplicated context
- near-duplicated context

- contradictory context
- stale context
- partially stale context
- low-relevance context
- high-relevance context with low token cost
- high-value large context
- low-value large context
- small but critical context
- delayed-relevance context
- context whose value cannot be determined
- context with missing provenance
- context with conflicting versions
- context with circular dependencies
- context dependency graph failure
- context ordering sensitivity
- position-sensitive information
- context with security-sensitive content
- context with tenant-specific data
- context with permission-sensitive data
- context with personally sensitive data
- context with binary/large artifacts
- context with malformed encoding

5. CONTEXT PRUNING EDGE CASES

Cover:

- pruning removes essential information
- pruning removes security constraints
- pruning removes authorization information
- pruning removes provenance
- pruning removes citations
- pruning removes dependencies
- pruning changes interpretation
- pruning creates contradictory context
- pruning causes downstream tool failure
- pruning causes schema failure
- pruning removes information that becomes relevant later
- pruning confidence is low
- pruning model unavailable
- pruning overhead exceeds expected savings

Expected behavior must include safe rollback.

6. COMPRESSION EDGE CASES

Cover:

- compression produces no savings
- compression produces negative savings
- compression overhead exceeds savings
- semantic drift
- factual drift
- loss of constraints
- loss of numbers
- loss of identifiers
- loss of code semantics
- loss of citations
- loss of provenance
- loss of authorization information
- compression of already-compressed context
- repeated compression
- recursive compression
- compression of structured data
- compression of JSON
- compression of code
- compression of logs
- compression of terminal output
- compression of test results
- compression model unavailable
- compression timeout
- compression output malformed
- compression validator failure
- reversible representation unavailable

7. RETRIEVAL EDGE CASES

Cover:

- zero results
- one result
- too many results
- irrelevant results
- duplicate results

- contradictory results
- stale results
- permission-filtered results
- retrieval timeout
- retrieval service unavailable
- reranker unavailable
- embedding service unavailable
- incorrect ranking
- low-confidence retrieval
- query rewriting makes query worse
- retrieval introduces malicious instructions
- retrieval result exceeds context budget
- retrieval result has missing provenance

8. CACHE EDGE CASES

Cover:

- exact cache hit
- semantic cache hit
- false semantic hit
- near semantic hit
- stale hit
- expired hit
- unauthorized hit
- cross-tenant hit
- policy-version mismatch
- model-version mismatch
- context-version mismatch
- dependency-version mismatch
- mutable data
- permission-sensitive data
- cache corruption
- cache unavailable
- cache timeout
- cache stampede
- concurrent writes
- duplicate writes
- cache fragmentation
- low reuse probability
- cache entry larger than request

- cache warming that costs more than it saves
- cache invalidation race
- invalidation dependency failure
- provider-native cache unavailable
- provider-native cache semantics change

A cache hit must never be treated as proof of correctness.

9. CACHE ECONOMICS EDGE CASES

Cover:

- cache write costs more than future savings
- low reuse probability
- short TTL
- high invalidation rate
- cache warming overhead
- cache storage cost exceeds savings
- semantic lookup cost exceeds saved inference
- provider cache minimum-prefix requirements
- cache breakpoint not reached
- unstable prefix
- dynamic content contaminating cacheable prefix
- cache fragmentation across tenants/workspaces/models
- cached tokens cheaper but not free
- cache hit changes model/provider economics

10. MODEL ROUTING EDGE CASES

Cover:

- no model satisfies requirements
- cheapest model fails capability requirement
- model unavailable
- model rate limited
- model quota exhausted
- model latency spike
- model quality regression
- provider outage
- provider capability mismatch
- context exceeds candidate model limit
- tool support mismatch

- structured output mismatch
- reasoning requirement mismatch
- compliance mismatch
- region mismatch
- model pricing changed
- model deprecated
- model version changed
- routing confidence low
- routing model unavailable
- routing overhead exceeds expected savings
- routing oscillation between models

11. REASONING-BUDGET EDGE CASES

Cover:

- reasoning budget too low
- reasoning budget too high
- task complexity misclassified
- reasoning controls unsupported
- reasoning cost unavailable
- model exposes different reasoning semantics
- adaptive reasoning causes quality regression
- reasoning escalation loop
- verifier repeatedly escalates reasoning
- reasoning reduction saves tokens but increases retries

12. OUTPUT OPTIMIZATION EDGE CASES

Cover:

- output exceeds requested limit
- output is truncated
- output schema invalid
- output contains unnecessary verbosity
- output compression changes meaning
- structured output is damaged
- code output becomes invalid
- citations are removed
- required fields removed

- downstream parser failure
- output budget too low
- output budget too high
- model ignores output constraints

13. TOOL OPTIMIZATION EDGE CASES

Cover:

- tool unnecessary
- tool call produces no useful information
- tool has negative expected ROI
- tool schema is huge
- tool result is huge
- tool result is irrelevant
- tool result duplicates existing context
- tool call fails
- tool times out
- tool rate limit
- tool authorization failure
- tool arguments invalid
- tool arguments contain sensitive data
- tool is non-idempotent
- retry could cause duplicate side effects
- tool has hidden cost
- parallel tool calls are not actually independent
- tool results arrive out of order
- tool result changes while being processed
- programmatic tool execution fails
- tool discovery fails
- dynamic tool loading fails
- tool catalog is too large

14. AGENT LOOP EDGE CASES

Cover:

- infinite loop
- repeated identical action
- oscillation between actions
- no-progress loop
- low-information iterations

- repeated tool calls
- repeated model calls
- repeated context reconstruction
- context grows every iteration
- token budget exhaustion
- cost budget exhaustion
- latency budget exhaustion
- agent stuck waiting for tool
- agent state corruption
- checkpoint failure
- recovery failure
- premature termination
- incorrect early termination
- stop detector false positive
- stop detector false negative

15. SUB-AGENT EDGE CASES

Cover:

- sub-agent unnecessary
- sub-agent cost exceeds expected benefit
- recursive spawning
- uncontrolled fan-out
- duplicate sub-agent tasks
- overlapping sub-agent tasks
- context duplication
- inconsistent sub-agent outputs
- conflicting results
- aggregation failure
- child timeout
- child cancellation
- child provider failure
- parent/child budget mismatch
- child inherits excessive context
- child receives insufficient context
- child leaks tenant data
- child continues after parent termination

16. DEVELOPER / CODING AGENT EDGE CASES

Cover:

- repository extremely large
- monorepo
- generated code
- vendored dependencies
- binary files
- lock files
- minified files
- deleted files
- renamed files
- uncommitted changes
- staged vs unstaged changes
- branch divergence
- merge conflicts
- stale repository index
- symbol resolution failure
- dependency graph failure
- circular dependencies
- generated build output
- huge terminal output
- huge test output
- compiler error flood
- repeated test failures
- flaky tests
- hidden test failures
- test output contains secrets
- repository contains secrets
- current task depends on seemingly unrelated file
- optimizer prunes a required dependency
- optimizer summarizes code incorrectly
- patch context becomes incomplete
- agent repeatedly reopens same files
- agent repeatedly runs same command
- coding task requires broad repository understanding

17. MCP / TOOL ECOSYSTEM EDGE CASES

Cover:

- MCP server unavailable
- MCP server slow
- MCP schema huge
- too many tools exposed
- duplicate tools
- conflicting tool names
- tool version mismatch
- resource unavailable
- resource changes during execution
- unauthorized resource
- malicious tool metadata
- malicious tool result
- tool injection
- dynamic discovery failure
- server capability mismatch

18. MEMORY EDGE CASES

Cover:

- memory unavailable
- stale memory
- contradictory memory
- irrelevant memory
- oversized memory
- duplicate memory
- sensitive memory
- permission-sensitive memory
- memory poisoning
- incorrect memory retrieval
- memory version conflict
- memory compaction loss
- memory deletion/recovery failure
- session boundary confusion
- tenant boundary confusion

19. RAG / KNOWLEDGE EDGE CASES

Cover:

- stale documents
- conflicting documents

- deleted documents
- permission changes
- document version changes
- duplicated documents
- retrieval drift
- index drift
- embedding drift
- chunk boundary problems
- citation loss
- citation mismatch
- unsupported claims after compression

20. SECURITY EDGE CASES

Cover:

- prompt injection
- indirect prompt injection
- data exfiltration
- cross-tenant cache reuse
- cross-user memory reuse
- unauthorized tool call
- privilege escalation
- sensitive data entering optimization logs
- sensitive data entering cache
- compression leaking sensitive information
- semantic cache leaking sensitive information
- malicious optimization policy
- compromised provider
- compromised tool
- compromised retrieval source
- policy bypass through optimization
- security instruction removal
- audit record tampering

Security violations must override optimization objectives.

21. MULTI-TENANT EDGE CASES

Cover:

- tenant ID missing

- tenant ID spoofed
- tenant configuration unavailable
- tenant policy conflict
- tenant-specific model allowlist
- tenant-specific cache
- tenant-specific memory
- tenant-specific budget
- tenant-specific quality requirements
- tenant data entering shared caches
- tenant data entering shared metrics
- tenant data entering training/learning systems

22. POLICY ENGINE EDGE CASES

Cover:

- no policy
- conflicting policies
- invalid policy
- expired policy
- policy version mismatch
- policy update during execution
- policy rollback
- emergency policy
- tenant policy conflicts with global policy
- quality policy conflicts with cost policy
- latency policy conflicts with cost policy
- security policy conflicts with optimization

Define deterministic precedence.

23. QUALITY-GATE EDGE CASES

Cover:

- validator unavailable
- validator timeout
- validator disagreement
- false positive
- false negative
- quality measurement unavailable
- quality degradation detected late

- quality threshold exactly met
- quality threshold barely missed
- quality metric noisy
- insufficient evaluation evidence
- model-based evaluator disagreement
- deterministic validator vs model evaluator disagreement

Never treat missing quality evidence as proof of success.

24. OBSERVABILITY EDGE CASES

Cover:

- telemetry unavailable
- metrics backend unavailable
- tracing backend unavailable
- high-cardinality explosion
- logging overload
- sensitive content in logs
- missing correlation ID
- broken trace propagation
- duplicate events
- out-of-order events
- incomplete cost data
- missing provider usage data
- missing token counts
- clock skew
- event version mismatch

The optimizer must remain safe when observability infrastructure is degraded.

25. COST ACCOUNTING EDGE CASES

Cover:

- unknown token count
- estimated token count
- provider reports delayed usage
- provider usage mismatch
- pricing changes
- pricing version mismatch

- cached-token pricing unavailable
- tool costs unavailable
- infrastructure cost unavailable
- optimizer cost unavailable
- currency mismatch
- negative/invalid cost
- cost attribution failure
- double counting
- missing optimization overhead

Never claim savings from incomplete accounting without clearly marking them as estimates.

26. FAILURE / FALLBACK EDGE CASES

Test every optimization stage when:

- dependency unavailable
- timeout
- malformed output
- invalid result
- quality failure
- security failure
- policy failure
- budget exceeded
- cancellation
- partial failure

Verify that the baseline path remains available.

27. CONCURRENCY EDGE CASES

Cover:

- duplicate concurrent optimization
- duplicate cache writes
- cache stampede
- simultaneous invalidation
- simultaneous policy changes
- simultaneous agent actions
- race between context retrieval and invalidation
- race between model routing and model outage

- race between cancellation and completion
- race between parent and child agent termination

28. RETRY / RATE LIMIT EDGE CASES

Cover:

- retry storms
- exponential backoff failure
- retry after partial side effect
- non-idempotent tool retry
- provider retry amplification
- nested retries
- optimizer retry + provider retry multiplication
- agent retry + tool retry multiplication
- rate-limit feedback loops

29. TIME / FRESHNESS EDGE CASES

Cover:

- clock skew
- stale timestamps
- timezone mismatch
- TTL boundary
- expiration during execution
- data changes during execution
- long-running agent crosses freshness boundary
- policy changes during execution
- model pricing changes during execution

30. INFERENCE / SERVING EDGE CASES

For P5 infrastructure-dependent capabilities cover:

- KV cache unavailable
- prefix cache miss
- cache fragmentation
- batching unavailable
- incompatible batch requests
- batch timeout
- speculative decoding unavailable
- speculative model mismatch

- quantization quality degradation
- GPU memory exhaustion
- GPU underutilization
- scheduler overload
- prefill bottleneck
- decode bottleneck
- model loading delay
- model eviction
- replica imbalance
- inference queue explosion

Keep these separate from prompt/context optimization measurements.

31. OPTIMIZER-ITSELF EDGE CASES

This section is mandatory.

Cover:

- optimizer overhead exceeds savings
- optimizer becomes latency bottleneck
- optimizer token usage explodes
- optimization recursion
- optimization module failure
- policy engine failure
- decision engine failure
- optimization decision oscillation
- conflicting optimization modules
- optimization ordering instability
- module dependency cycle
- stale optimization policy
- bad learned policy
- optimization regression
- optimization creates more tool calls
- optimization causes more retries
- optimization causes larger context
- optimization reduces cost but destroys quality
- optimization reduces tokens but increases infrastructure cost
- optimization creates cache fragmentation
- optimization increases operational complexity beyond value

32. LEARNING / ADAPTIVE POLICY EDGE CASES

Cover:

- insufficient training/evaluation data
- biased historical data
- workload distribution shift
- model distribution shift
- provider change
- optimization policy drift
- reward hacking
- learned policy exploits evaluation metric
- feedback loop
- catastrophic policy update
- unsafe automatic rollout
- regression after rollout
- noisy reward signal

Require shadow mode, validation, controlled rollout, and rollback.

33. DISASTER / INFRASTRUCTURE EDGE CASES

Cover:

- database unavailable
- cache cluster unavailable
- message broker unavailable
- configuration service unavailable
- secrets service unavailable
- provider outage
- regional outage
- network partition
- partial service outage
- storage corruption
- replica failure
- control-plane restart
- state recovery
- deployment rollback

34. PERFORMANCE / SCALE EDGE CASES

Cover:

- sudden traffic spike
- extreme concurrency
- huge tenant
- noisy neighbor
- high-cardinality workloads
- very long agent sessions
- extremely large repositories
- cache hot key
- cache cold start
- model hot spot
- provider rate limits
- telemetry overload
- optimization queue backlog

35. ADVERSARIAL / RED-TEAM SCENARIOS

Include adversarial scenarios designed specifically to defeat optimization:

- prompt injection that causes context retention
- prompt injection that causes unsafe cache reuse
- malicious content designed to survive pruning
- malicious content designed to exploit summarization
- semantic-cache collision attempts
- tenant isolation attacks
- tool-result injection
- memory poisoning
- policy manipulation
- cost amplification attacks
- agent loop amplification
- sub-agent fan-out attack
- retry amplification attack
- context explosion attack
- cache stampede attack

36. EDGE-CASE INTERACTION MATRIX

Do not only test individual failures.

Define compound scenarios such as:

- cache miss + provider outage
- compression failure + context overflow
- retrieval failure + low token budget
- tool timeout + agent retry
- model failure + sub-agent escalation
- quality failure + model escalation
- policy change + running agent
- tenant policy + semantic cache hit
- cache hit + stale data + authorization change
- optimizer failure + LLM failure
- telemetry failure + optimization failure
- context overflow + provider fallback
- cost budget exhaustion + quality requirement
- security violation + optimization opportunity

37. RECOVERY PRIORITY

Define the order of precedence:

Security

→ Authorization

→ Data integrity

→ Correctness

→ Task completion

→ Reliability

→ Quality

→ Freshness

→ Cost

→ Latency

Where appropriate, document exceptions explicitly.

38. TEST MATRIX

Create a test matrix mapping:

Edge Case

- Component
- Trigger
- Expected Behavior
- Fallback
- Metric
- Test Type
- Severity
- Requirement

Include:

- unit
- contract
- integration
- end-to-end
- load
- chaos
- security
- regression
- adversarial

39. SEVERITY CLASSIFICATION

Define:

P0 — catastrophic/security/data-integrity

P1 — major correctness/reliability failure

P2 — significant quality/cost/performance issue

P3 — recoverable operational issue

P4 — minor/diagnostic issue

40. IMPLEMENTATION REQUIREMENTS

For each high-severity edge case specify whether implementation requires:

- guard
- validator
- timeout
- circuit breaker
- fallback

- retry policy
- cache isolation
- budget
- circuit breaker
- feature flag
- quality gate
- audit event
- alert

41. DEFINITION OF DONE

An edge case is considered addressed only when:

- detection is defined
- expected behavior is defined
- fallback is defined
- security impact is understood
- quality impact is understood
- cost impact is understood
- observability is defined
- automated test exists or is explicitly planned
- requirement traceability exists

42. FINAL REVIEW

Before finalizing `edge-cases.md`, perform a completeness review against:

`problemStatement.txt`

`architecture.md`

`interfaces.md`

`conventions.md`

Verify coverage across:

- request processing
- context
- compression
- retrieval
- caching
- tools
- agents

- sub-agents
- coding agents
- MCP
- memory
- model routing
- reasoning
- output
- quality
- security
- tenancy
- policy
- cost
- observability
- concurrency
- retries
- infrastructure
- inference serving
- learning
- adversarial attacks
- optimizer self-failure

At the end provide:

1. Edge-case inventory
2. Highest-risk scenarios
3. Compound failure scenarios
4. Security-critical scenarios
5. Cost-amplification scenarios
6. Quality-degradation scenarios
7. Agent-loop scenarios
8. Optimizer-self-failure scenarios
9. Coverage matrix
10. Open questions
11. Recommended next documentation artifact

Generate ONLY `edge-cases.md`.

Do not implement code.

One important instruction

When Antigravity generates this, **don't let it make** `edge-cases.md` **just a list of generic errors.**

The most valuable part for your product is the **optimizer-specific failure space**:

“What happens when the optimization itself makes the system worse?”

For example:

Compression saves 50% tokens → but causes a 3% quality drop → retries increase → total cost becomes 12% higher.

That scenario is much more important to your product than something generic like *“database unavailable.”*

Your existing specification already explicitly requires every optimization to account for overhead, quality, fallback, and measurable net value, so `edge-cases.md` should turn those principles into concrete failure scenarios. `8f1ee3e4-b731-4468-866e-2bb6802...`

Once `edge-cases.md` is generated, **send it to me before moving to** `optimization-catalog.md`. That catalog is going to be one of the most important documents in the entire project, and we'll want to make sure it doesn't miss optimization families or accidentally treat experimental techniques as production-safe.

Sources

User Prompt for Generate Problem Statement-2.pdf

At this stage I would like to know the below scenarios are taken care in our control plane:

- 1) What if the organization doesn't have access to enable the outside GenAI products other than the internal implementation on top of the actual products?
- 2) What if the context is totally pruned or context is never taken while going through the pipeline?
- 3) What if the suggested models are not accessible?
- 4) What was the scenario for coding agent wants to generate something out of context?
- 5) What if the coding agent related developer wants to access something against the policies?
- 6) What's the case for negative scenarios like: only few things to be performed, just in-case files to be attached in-between, permissions to be added/removed while executing the tasks or steps?
- 7) What's the case if the user has made the context to wait for an unexpected longer time?
- 8) What's the case, if the user changes the context window while processing another result?
- 9) What's the case, if the user changes context or workflow while executing the tasks or steps?
- 10) What's the case if the execution flow is interrupted?
- 11) What's the scenario, can the context be retained by the agent itself or do we need to retain? If we have to retain what are the conditions?

and so on...

So, I want you to perform all such negative/positive scenarios to get passed and must be included in all the generated docs as of now. and show me the step process for this till now to get any such scenarios to be added.

Worked for 5s

Yes. And your 11 questions reveal an important gap in the current documentation strategy:

We have been documenting the optimizer as if it receives a reasonably stable request and then optimizes it. An enterprise control plane must instead treat the request, context, permissions, workflow, tools, model availability, and execution state as potentially changing while the workflow is running.

Your existing specification already has pieces of this—context invalidation, hierarchical memory, session phases, reversible transformations, agent-loop control, safe fallback, permission-aware caching, and cost/quality constraints—but the scenarios you listed need to be elevated into an explicit **Dynamic Execution & State Management** requirement across the documents.

problemstatement

c9ea3590-2236-47e6-8cb6-f0e1cd1...

Your scenarios — what we need to guarantee

#	Scenario	Required Control-Plane behavior
1	Organization cannot access external GenAI products directly	Must support gateway/proxy/internal-provider mode. The Control Plane optimizes requests before the organization's existing internal GenAI implementation, without requiring direct provider access.
2	Context gets totally pruned / no context reaches LLM	Must have context-integrity gates. If required context disappears, restore/retrieve/reject—not blindly execute.
3	Suggested models inaccessible	Capability-aware routing + fallback chain. Never select an inaccessible model.
4	Coding agent needs to generate something "out of context"	Intent/context expansion path. The optimizer must allow the agent to deliberately request broader context rather than assuming the current optimized context is sufficient.
5	Developer requests something against policy	Policy gate must override optimization. Block, redact, escalate, or require authorization.
6	Few actions / files attached / permissions change mid-task	Dynamic workflow/context mutation handling. Re-evaluate only the affected portions instead of blindly restarting everything.

#	Scenario	Required Control-Plane behavior
7	User waits unexpectedly long	Session lease/timeout/freshness management. Revalidate state before continuing.
8	User changes context window while another result is processing	Versioned execution state. Result must be associated with the context version it was generated against.
9	User changes context/workflow during execution	Workflow versioning + cancellation/reconciliation. Old execution must not blindly overwrite new intent.
10	Execution interrupted	Checkpoint + resumability + idempotency + recovery.
11	Agent retains context vs Control Plane	Hybrid ownership. Agent may retain working state, but Control Plane must own the authoritative optimization/audit/security state required for safe continuation.

And there are many more variants.

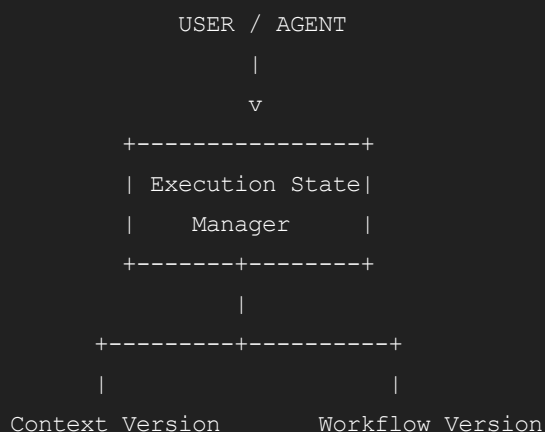
The bigger requirement I recommend adding

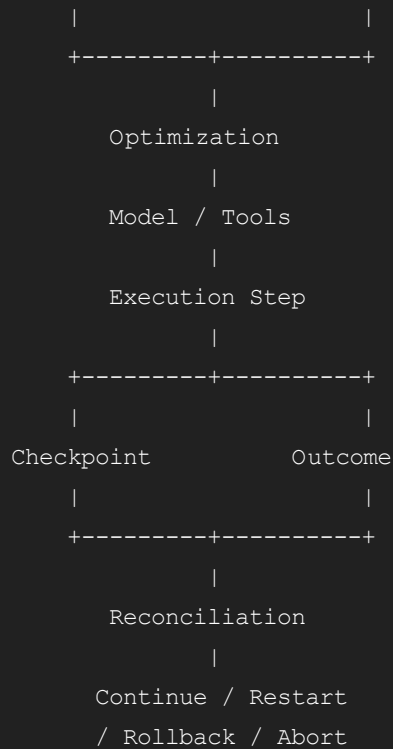
I would introduce a new first-class concept:

Dynamic Execution State & Context Lifecycle Management

The Control Plane should **not** treat a request as immutable.

Instead:





That changes the product from merely an **LLM optimizer** into a genuine **enterprise execution control plane**.

The documentation process I recommend from this point

Don't simply continue:

```
problemStatement → architecture → interfaces → conventions → edge-cases →  
optimization-catalog
```

Instead, introduce a **scenario-validation loop**.

Step 1 — Requirements

```
problemStatement.txt
```

Defines **what the platform must accomplish**.

Step 2 — Architecture

```
architecture.md
```

Defines **where capabilities live**.

Step 3 — Contracts

```
interfaces.md
```

Defines how components communicate.

Step 4 — Engineering laws

`conventions.md`

Defines how implementation must behave.

Step 5 — Scenario universe

`edge-cases.md`

Defines what can go wrong.

Step 6 — NEW: Scenario matrix

I strongly recommend adding:

`scenario-matrix.md`

This becomes the master validation document.

It should contain:

```
Scenario
  ↓
Trigger
  ↓
Affected state
  ↓
Affected context
  ↓
Affected permissions
  ↓
Affected workflow
  ↓
Affected model/tool
  ↓
Required decision
  ↓
Expected behavior
  ↓
Fallback
  ↓
Recovery
  ↓
Audit
  ↓
Metrics
```

Step 7 — Optimization catalog

Only after that:

```
optimization-catalog.md
```

Because then every optimization can be tested against the scenario matrix.

I would add these scenario families

Your 11 examples should actually expand into **15 major scenario families**.

A. Capability availability

- provider unavailable
- provider inaccessible
- model inaccessible
- feature unavailable
- internal gateway only
- provider credentials unavailable
- organization policy blocks provider
- provider temporarily unavailable

B. Context lifecycle

- context missing
- context empty
- context completely pruned
- context partially pruned
- context becomes stale
- context becomes invalid
- context changes
- context version changes
- context becomes larger
- context becomes smaller
- user adds context
- user removes context

C. Workflow lifecycle

- workflow changes
- task changes
- task cancelled

- task paused
- task resumed
- task expires
- task superseded
- workflow branch changes
- workflow partially completed

D. Permission lifecycle

This is particularly important.

```

Permission granted
    ↓
Task starts
    ↓
Permission revoked
    ↓
Cached result exists
    ↓
Tool already executing
    ↓
Agent wants next action
  
```

The Control Plane must decide **which state remains valid and which must be invalidated**.

Similarly:

```

Permission denied
    ↓
User requests access
    ↓
Permission granted
    ↓
Continue task
  
```

The system shouldn't necessarily restart the entire workflow.

E. Model lifecycle

- preferred model unavailable
- model becomes unavailable mid-task
- model capability changes
- model deprecated
- model price changes
- model context limit changes
- model quality regression

- fallback model has smaller context
- fallback model doesn't support tools
- fallback model doesn't support required structured output

F. User interaction lifecycle

- user changes request
- user adds instructions
- user removes instructions
- user attaches file
- user removes file
- user changes priority
- user changes budget
- user changes quality requirement
- user changes deadline
- user changes context window
- user pauses execution
- user resumes execution
- user abandons task

G. Long-running execution

- user disappears
- session expires
- task waits for hours
- external dependency takes too long
- underlying data changes while waiting
- permissions change while waiting
- model availability changes while waiting
- pricing changes while waiting
- context becomes stale while waiting

H. Interruptions

- network failure
- process crash
- agent crash
- Control Plane restart
- provider timeout
- tool timeout
- user cancellation
- machine restart
- deployment

- region outage

I. Resume/recovery

When resuming:

Do NOT simply replay the original prompt.

Instead:

```
Checkpoint
+
Current Context
+
Current Workflow Version
+
Current Permissions
+
Current Policies
+
Current Tool State
+
Current Model Availability
+
Current External State
+
Previous Outcomes
```

Then determine:

resume / recompute / rollback / restart / abort

J. Agent-owned context

This deserves explicit architecture.

There should be three concepts:

Agent Working State

What the agent currently believes/uses.

Control Plane State

What the platform needs to safely optimize, govern, audit, recover, and evaluate.

Authoritative External State

Repository, database, permissions, files, APIs, etc.

These must not be conflated.

K. Coding-agent "out-of-context" behavior

This is extremely important.

Suppose the optimizer gives Cursor/Claude Code only:

```
src/payment/service.py
```

Then the agent realizes:

"I need the database migration and authentication middleware."

The optimizer must **not interpret this as a failure of the original context**.

Instead:

```
Current Context
  ↓
Agent identifies missing dependency
  ↓
Context Expansion Request
  ↓
Dependency / Symbol Retrieval
  ↓
Security / Policy Check
  ↓
ROI evaluation
  ↓
Add required context
  ↓
Continue
```

This should be a formal interface.

L. Selective execution

Your example:

"only a few things need to be performed"

is important.

The optimizer should support **partial execution**.

For example:

Workflow:

1. Search repository
2. Modify code
3. Run tests
4. Generate documentation
5. Commit changes

User says:

"Only do 1 and 2."

The Control Plane must:

- cancel/suppress 3–5
- preserve completed work
- update workflow state
- prevent the agent from automatically continuing

M. Dynamic attachment

Example:

```
Task started
  ↓
User attaches requirements.pdf
  ↓
Existing context = version 4
New context = version 5
  ↓
Determine affected steps
  ↓
Invalidate affected decisions
  ↓
Retain unaffected work
  ↓
Continue from safe checkpoint
```

This is far more efficient than restarting the whole task.

N. Context-window changes

If the user/model changes:

500K → 128K

while execution is underway, the Control Plane must:

1. freeze affected execution
2. create new context budget

- 3. re-evaluate context
- 4. preserve mandatory context
- 5. recompute routing if necessary
- 6. reconstruct prompt
- 7. validate
- 8. resume

Conversely:

128K → 500K

must **not automatically cause the system to stuff more context back in**.

A larger window is an available capability—not a requirement to use it.

Agent context retention: who owns what?

I recommend this model:

State	Owner
Agent's transient reasoning/work state	Agent
Original user request	Control Plane
Optimized representation	Control Plane
Optimization decisions	Control Plane
Policy state	Control Plane
Cost ledger	Control Plane
Context provenance	Control Plane
Checkpoints	Control Plane
Workflow state	Shared/Control Plane authoritative
Repository/files	External source of truth
Permissions	External identity/policy source
Tool state	Tool/external system

State	Owner
Long-term memory	Configurable memory subsystem

The key principle:

The agent may own working state, but the Control Plane must own enough authoritative state to safely govern, audit, invalidate, recover, and reproduce the execution.

This fits very well with your existing requirements for context provenance, invalidation, reversible optimization, hierarchical memory, and session phases.

c9ea3590-2236-47e6-8cb6-f0e1cd1...

One change I'd make before you continue

I would **pause the sequential document generation for one step.**

Don't immediately ask Antigravity for another ordinary document.

First ask it to update the existing documents with this new concept:

Add to `problemStatement.txt`

Dynamic Execution State & Context Lifecycle Management

Add to `architecture.md`

Execution State Manager + Context/Workflow Versioning + Checkpoint/Recovery

Add to `interfaces.md`

New interfaces for:

- execution state
- context version
- workflow version
- context expansion
- context mutation
- permission change
- capability change
- checkpoint
- resume
- cancellation
- supersession

- reconciliation

Add to `conventions.md`

Rules for:

- immutable execution versions
- optimistic concurrency
- state transitions
- idempotency
- stale-result rejection
- authorization revalidation
- context provenance
- checkpointing

Add to `edge-cases.md`

All the scenarios above.

Then create

`scenario-matrix.md`

This becomes the **master scenario validation artifact**.

The final documentation pipeline should therefore become

```
01 problemStatement.txt
    ↓
02 architecture.md
    ↓
03 interfaces.md
    ↓
04 conventions.md
    ↓
05 edge-cases.md
    ↓
06 scenario-matrix.md      ← NEW
    ↓
07 optimization-catalog.md
    ↓
08 provider-matrix.md
    ↓
09 cache-strategy.md
    ↓
10 agent-optimization.md
    ↓
```

```
11 inference-optimization.md
    ↓
12 quality-gates.md
    ↓
13 eval.md
    ↓
14 security.md
    ↓
15 observability.md
    ↓
16 implementation-plan.md
    ↓
17 SCALING.md
    ↓
18 ADRs
```

And every document from #7 onward should be required to pass against `scenario-matrix.md`.

That gives you something much stronger than simply a large list of optimization techniques:

Every optimization has to prove that it remains safe when context, permissions, workflow, models, tools, users, and execution state change dynamically.

That is much closer to an actual **Enterprise Agent & LLM Inference Optimization Control Plane** than a prompt/token optimizer.

If you want, I can give you the **single Antigravity master prompt** that updates the four **existing documents** (`problemStatement.txt`, `architecture.md`, `interfaces.md`, `conventions.md`) and generates `scenario-matrix.md` **without losing anything already established**.

Sources

User Prompt for Generate Problem Statement-1.pdf

yes. please give me the single Antigravity master prompt that updates the four existing documents (problemStatement.txt, architecture.md****, interfaces.md****, conventions.md****) and generates scenario-matrix.md without losing anything already established.

Absolutely. I recommend doing this **before generating** `edge-cases.md`, because the new dynamic-execution requirements affect all four foundational documents.

The prompt below is deliberately written as a **controlled architecture evolution**, not a rewrite. It tells Antigravity to preserve everything already established, identify gaps, add the new requirements, maintain traceability, and then generate the new `scenario-matrix.md`.

 Edit



MASTER PROMPT

Enterprise Agent & LLM Inference Optimization Control Plane

Foundational Architecture Upgrade: Dynamic Execution State, Context Lifecycle & Scenario Validation

You are modifying an existing enterprise-grade software architecture and requirements specification.

The product is:

Enterprise Agent & LLM Inference Optimization Control Plane

The Control Plane must support:

- generic LLM applications
- enterprise AI applications
- autonomous agents
- multi-agent systems
- sub-agent systems
- developer/coding agents
- coding assistants
- IDE-integrated agents
- CLI-based coding agents
- agent frameworks
- MCP-enabled systems
- internal enterprise GenAI gateways
- externally hosted LLM providers
- self-hosted/private models

- hybrid inference environments

The Control Plane is intended to operate as an optimization, governance, policy, routing, context-management, cost-control, quality-control, observability, and execution-management layer.

0. CRITICAL INSTRUCTION — DO NOT LOSE EXISTING REQUIREMENTS

The following existing documents are authoritative working documents:

1. `problemStatement.txt`
2. `architecture.md`
3. `interfaces.md`
4. `conventions.md`

You MUST read and understand all four documents before making any modification.

DO NOT:

- rewrite them from scratch
- simplify them
- remove existing requirements
- replace existing optimization techniques
- discard existing architecture
- silently change established behavior
- remove provider-neutrality
- remove agent/coding-agent support
- remove enterprise requirements
- remove security requirements
- remove observability requirements
- remove evaluation requirements
- remove cost accounting
- remove quality gates
- remove caching requirements
- remove context optimization
- remove inference optimization
- remove memory requirements

- remove routing requirements
- remove fallback mechanisms
- remove auditability
- remove multi-tenancy
- remove existing interfaces
- rename existing concepts unless absolutely necessary

This is an **architecture evolution**, not a rewrite.

Preserve all existing content unless a genuine contradiction is discovered.

If a contradiction exists, explicitly identify it and resolve it using the source-of-truth hierarchy already defined by the project.

1. PRIMARY OBJECTIVE

Upgrade the existing architecture so that the Control Plane explicitly supports:

Dynamic execution

The request, context, workflow, permissions, tools, models, policies, files, and external state may change while a task is executing.

The Control Plane **MUST** therefore treat execution as a versioned, stateful lifecycle rather than a single immutable request/response transaction.

The system must safely support:

- context changes
- context expansion
- context reduction
- context invalidation
- workflow changes
- user instruction changes
- file attachment/removal
- permission changes
- policy changes
- model availability changes
- provider availability changes

- tool availability changes
 - model/context-window changes
 - user cancellation
 - task interruption
 - task pause
 - task resume
 - task supersession
 - partial execution
 - long-running execution
 - delayed execution
 - failed execution
 - recovery
 - checkpointing
 - reconciliation
 - rollback
 - retry
 - safe continuation
-

2. NEW FIRST-CLASS PRODUCT CAPABILITY

Introduce the following first-class capability throughout the architecture:

Dynamic Execution State & Context Lifecycle Management

The Control Plane must maintain enough authoritative execution state to safely:

- optimize
- govern
- audit
- evaluate
- invalidate
- recover
- resume
- reconcile
- cancel

- supersede
- reproduce

an execution.

The agent may maintain its own working state, but the Control Plane must retain authoritative state required for governance and safe execution.

3. HYBRID STATE OWNERSHIP MODEL

Explicitly define three state domains.

3.1 Agent Working State

State owned by the executing agent, such as:

- transient reasoning
- temporary plans
- local scratch state
- intermediate working representations
- agent-specific temporary state

The Control Plane must NOT unnecessarily duplicate transient agent state.

3.2 Control Plane Authoritative State

The Control Plane should own or durably track, where required:

- original request
- request version
- context version
- workflow version
- execution ID
- execution state
- optimization decisions
- optimization versions
- policy versions
- authorization decisions
- model/provider decision
- tool authorization

- budget state
- cost ledger
- quality-gate state
- checkpoint state
- recovery metadata
- provenance
- audit information
- relevant cache metadata
- dependency versions
- supersession relationships
- cancellation state
- reconciliation state

3.3 External Authoritative State

External systems remain authoritative for:

- repositories
- files
- source control
- databases
- identity
- permissions
- enterprise policy systems
- external APIs
- tool state
- external documents
- external business systems

The architecture **MUST NOT** incorrectly treat an optimization cache or agent memory as the authoritative source when the external system is authoritative.

4. ORGANIZATIONS WITHOUT DIRECT PROVIDER ACCESS

Explicitly support organizations that do **NOT** have direct access to external GenAI providers.

The Control Plane MUST support at least these deployment patterns:

Pattern A — Direct Provider

Control Plane → Provider

Pattern B — Enterprise Gateway

Control Plane → Enterprise GenAI Gateway → Provider

Pattern C — Internal AI Platform

Control Plane → Internal AI Platform → Provider/Model

Pattern D — Self-Hosted Model

Control Plane → Private Inference Infrastructure

Pattern E — Hybrid

Control Plane → Policy/Router → approved internal or external path

The optimization framework MUST NOT require direct provider credentials when the organization's architecture intentionally exposes only an internal AI gateway.

Provider-specific optimization capabilities should be abstracted through capability discovery.

Where the internal gateway hides provider details, the Control Plane must operate using the capabilities and usage metadata available through that gateway.

If token/cost/model information is unavailable, explicitly represent it as:

- unavailable
- estimated
- provider-reported
- gateway-reported
- unknown

Never fabricate provider information.

5. CONTEXT INTEGRITY REQUIREMENT

Introduce a mandatory **Context Integrity Gate**.

The system **MUST** detect situations where:

- all context was accidentally pruned
- required context was never collected
- required context was removed
- compression removed essential information
- context retrieval failed
- context became stale
- context became unauthorized
- context provenance was lost
- context dependencies disappeared

The Control Plane **MUST NOT** blindly send an empty or materially incomplete context to the LLM merely because token optimization succeeded.

The system must distinguish:

Valid zero-context execution

The task genuinely requires no additional context.

Invalid zero-context execution

Required context was lost, omitted, or incorrectly pruned.

The latter must trigger an appropriate:

- context recovery
- retrieval
- expansion
- fallback
- validation failure
- execution pause
- user clarification

depending on policy.

6. CONTEXT EXPANSION REQUIREMENT

Optimization must be reversible in both directions.

The Control Plane must support:

Context Reduction



Context Expansion

An agent must be able to report:

"The currently available context is insufficient for this task."

The Control Plane must then evaluate:

- what information is missing
- where it can be obtained
- whether the agent is authorized
- expected token cost
- expected quality benefit
- expected latency
- security implications
- dependency implications

Then retrieve/add the minimum sufficient context.

This is especially mandatory for coding agents.

7. CODING AGENT OUT-OF-CONTEXT SCENARIO

Explicitly support:

Optimized Context



Coding Agent Begins Work



Agent Detects Missing Dependency



Requests Additional Context



Context Expansion Evaluation



Authorization / Policy Check

↓

Targeted Retrieval

↓

Context Re-optimization

↓

Quality / Integrity Validation

↓

Continue Execution

Examples:

- agent has one source file but needs an interface definition
- agent has implementation but needs tests
- agent needs database schema
- agent needs configuration
- agent needs dependency implementation
- agent needs API contract
- agent needs repository-wide architectural context
- agent needs historical commit context
- agent needs external documentation

The optimizer **MUST NOT** interpret every context-expansion request as an optimization failure.

8. POLICY AND AUTHORIZATION DURING EXECUTION

Permissions are dynamic.

Authorization **MUST** be evaluated at the point of sensitive action, not only when the task begins.

Support:

- permission granted during execution
- permission revoked during execution
- permission expires
- role changes
- policy changes
- resource ownership changes
- file permissions change

- repository permissions change
- tool authorization changes

If authorization is revoked:

- stop affected action
- invalidate affected cached/context state
- prevent unauthorized continuation
- preserve unaffected work where safe
- record audit event
- reconcile execution state

Do NOT automatically restart the entire workflow unless required.

9. MODEL ACCESSIBILITY

A model suggested by the optimization/routing engine may not actually be accessible.

The architecture MUST distinguish:

- recommended model
- eligible model
- accessible model
- currently available model
- authorized model
- policy-approved model
- capability-compatible model

A model may be rejected because:

- organization does not have access
- tenant policy prohibits it
- provider unavailable
- credentials unavailable
- region restriction
- quota
- rate limit
- model retired
- capability mismatch
- context limit
- compliance restriction

Routing must select only models satisfying the effective policy and capability constraints.

10. DYNAMIC MODEL AVAILABILITY

Model availability may change during execution.

Example:

```
Task starts
  ↓
Model A selected
  ↓
Model A becomes unavailable
  ↓
Execution pauses/retries/falls back
  ↓
Re-evaluate:
policy
capability
context
budget
quality
latency
  ↓
Select Model B
```

The system **MUST NOT** blindly replay the original request against Model B if the request/context has changed.

11. DYNAMIC CONTEXT WINDOW

Explicitly support context-window changes while processing.

Examples:

```
128K → 32K
32K → 128K
128K → 1M
```

When the available context budget decreases:

- preserve mandatory context
- recompute context selection
- invalidate incompatible optimization decisions
- re-run relevant quality/integrity gates
- re-route if necessary

When the available context increases:

- do NOT automatically add unnecessary context

A larger context window is an available capability, not an instruction to consume more tokens.

12. USER CONTEXT MUTATION

Users may change context while an execution is in progress.

Examples:

- add file
- remove file
- add instruction
- remove instruction
- change requirement
- change task priority
- change budget
- change model preference
- change quality requirement
- change deadline
- change context window
- change workflow steps

The system must use versioning.

Every execution must be associated with:

- request version
- context version
- workflow version
- policy version
- authorization state

Results generated against stale versions must not blindly overwrite newer user intent.

13. WORKFLOW MUTATION

Support dynamic workflow changes.

Example:

Original:

1. Search
2. Modify
3. Test
4. Document
5. Commit

User changes:

"Only perform steps 1 and 2."

The Control Plane must:

- update workflow version
 - cancel/suppress future steps
 - preserve completed work
 - prevent automatic continuation
 - reconcile agent state
 - record the change
 - avoid unnecessary recomputation
-

14. PARTIAL EXECUTION

Explicitly support workflows where only some operations are required.

The system **MUST NOT** optimize the entire workflow if only a subset will execute.

Optimization should be aware of execution scope.

Examples:

- only inspect files
 - only generate code
 - don't run tests
 - don't commit
 - don't send email
 - don't call external tools
 - don't modify files
 - only produce a plan
-

15. DYNAMIC FILE ATTACHMENTS

Support:

- file added during execution
- file removed during execution
- file modified during execution
- file replaced
- file becomes inaccessible
- file permission changes
- file version changes

Use file version/provenance tracking.

Only affected context and workflow portions should be invalidated where safe.

16. LONG-WAIT EXECUTION

Tasks may wait much longer than expected.

Examples:

- user pauses
- approval pending
- tool waiting
- external API waiting
- asynchronous job
- human-in-the-loop

- enterprise approval workflow

Before resuming after a significant delay, revalidate:

- permissions
- policies
- context freshness
- external state
- model availability
- provider availability
- budgets
- task validity
- workflow version

Do not assume the world is unchanged.

17. USER CHANGES CONTEXT WHILE RESULT IS PROCESSING

If a model result is already being generated and the user changes context:

The Control Plane must support:

- cancellation if possible
- marking result as stale
- completion followed by discard
- result reconciliation
- re-execution against new context

The stale result must NOT automatically become the authoritative result.

18. EXECUTION INTERRUPTION

Support interruption from:

- user cancellation
- network failure
- provider failure

- tool failure
- Control Plane restart
- agent restart
- machine failure
- deployment
- region failure
- timeout
- budget exhaustion

Execution must transition through explicit states.

Recommended state model:

CREATED

VALIDATING

OPTIMIZING

READY

RUNNING

WAITING

PAUSED

CANCELLING

CANCELLED

INTERRUPTED

RECOVERING

RECONCILING

RESUMABLE

RESUMING

COMPLETED

FAILED

SUPERSEDED

ABORTED

19. CHECKPOINTING

Long-running or high-value executions should support checkpoints.

A checkpoint should capture sufficient state to safely determine:

- what completed
- what did not complete
- what assumptions were used
- which context version was used

- which workflow version was used
- which permissions were used
- which model/provider was used
- which tools were called
- which side effects occurred
- which budgets remain
- what can be safely resumed

Do not require full prompt/context duplication when references and immutable versions are sufficient.

20. RESUME DECISION

On resume, do NOT simply replay the original prompt.

Use:

```
Checkpoint
+
Current User Intent
+
Current Context
+
Current Workflow
+
Current Permissions
+
Current Policies
+
Current External State
+
Current Model Availability
+
Completed Work
+
Remaining Work
```

Then decide:

- RESUME
- RECOMPUTE
- EXPAND CONTEXT

- ROLLBACK
 - RESTART
 - ABORT
 - REQUEST USER INPUT
-

21. IDEMPOTENCY AND SIDE EFFECTS

Every action must be classified as:

- read-only
- idempotent
- conditionally idempotent
- non-idempotent
- irreversible

Never blindly replay non-idempotent operations.

Examples:

- file write
- git commit
- deployment
- email
- database mutation
- payment
- ticket creation
- external API mutation

must have explicit recovery semantics.

22. SUPERSESSION

A newer user instruction may supersede an older execution.

Example:

Task V1: Implement feature A

while running

Task V2: Don't implement A; implement B instead.

V1 must become:

SUPERSEDED

unless policy explicitly allows both.

The Control Plane must prevent stale execution from continuing to modify authoritative state.

23. CONTEXT RETENTION POLICY

Explicitly define when context must be retained by the Control Plane.

Retain enough information when needed for:

- security
- audit
- compliance
- recovery
- reproducibility
- optimization evaluation
- billing/cost attribution
- quality evaluation
- debugging
- workflow continuation

Do NOT retain all raw context indefinitely.

Support:

- configurable retention
 - minimization
 - redaction
 - pseudonymization
 - encryption
 - tenant-specific policies
 - legal/compliance retention
 - deletion
-

24. AGENT VS CONTROL-PLANE CONTEXT RETENTION

Document the principle:

The agent may retain working context, but the Control Plane must retain authoritative metadata and immutable/versioned references necessary to safely govern and recover execution.

The Control Plane must NOT assume an agent will always remember:

- previous context
 - previous policy
 - previous authorization
 - previous optimization decision
 - previous workflow
 - previous checkpoint
-

25. CONTEXT PROVENANCE

Every retained/reconstructed context item should, where applicable, identify:

- source
- version
- timestamp
- owner
- authorization scope
- sensitivity
- transformation history
- optimization history
- validity period

This is mandatory for safe reconstruction.

26. STALE RESULT PROTECTION

A result generated using:

Context V4
Workflow V7
Policy V3

must not automatically overwrite:

Context V5
Workflow V8
Policy V4

without reconciliation.

Introduce version checks and optimistic concurrency semantics.

27. OPTIMIZATION RE-EVALUATION

When material state changes, determine which optimization decisions remain valid.

Do NOT automatically rerun every optimization.

Use dependency-aware invalidation.

Example:

File A changed

↓

Only optimizations depending on A invalidated

↓

Unrelated cached/retrieval decisions remain valid

28. NEGATIVE OPTIMIZATION SCENARIOS

Explicitly support scenarios where optimization makes things worse.

Examples:

- token savings increase retries
- compression causes quality failure
- pruning causes tool failure

- routing selects cheaper but unsuitable model
- caching causes stale result
- semantic cache creates false hit
- tool suppression causes missing information
- context compression increases reasoning requirements
- reduced context causes additional retrieval
- optimization latency exceeds inference savings
- optimization compute costs more than inference saved

The system must calculate:

Net Optimization Value

rather than gross token savings.

29. NO-OPTIMIZATION DECISION

The optimizer must be allowed to explicitly decide:

`DO_NOT_OPTIMIZE`

Reasons may include:

- request already optimal
- insufficient confidence
- optimization risk too high
- savings too small
- optimizer overhead too high
- task too small
- context too sensitive
- quality uncertainty
- latency requirement too strict

This is a valid optimization outcome.

30. INTERNAL-ONLY DEPLOYMENT

Support organizations where the Control Plane is deployed entirely inside enterprise infrastructure.

Examples:

Coding Agent



Internal Control Plane



Internal AI Gateway



Approved Provider

The Control Plane must remain useful even when it cannot directly inspect provider-level internals.

31. SCENARIO VALIDATION AS A FIRST-CLASS ENGINEERING PROCESS

Create a new artifact:

```
scenario-matrix.md
```

This is the master validation matrix.

Every major requirement, architecture component, interface, optimization technique, and policy must eventually be traceable to relevant scenarios.

32. SCENARIO MATRIX STRUCTURE

For every scenario include:

- Scenario ID
- Scenario family
- Description
- Actor
- Trigger
- Preconditions
- Initial state
- State mutation
- Context mutation
- Workflow mutation
- Permission mutation
- Policy mutation

- Model/provider mutation
 - Tool mutation
 - Expected Control Plane decision
 - Expected agent behavior
 - Expected optimization behavior
 - Expected fallback
 - Expected recovery
 - Security implications
 - Quality implications
 - Cost implications
 - Latency implications
 - Data-retention implications
 - Audit requirements
 - Metrics
 - Traces
 - Alerts
 - Required tests
 - Severity
 - Priority
 - Related requirement
 - Related architecture component
 - Related interface
 - Related convention
 - Related edge case
 - Related optimization
 - Open questions
-

33. SCENARIO CATEGORIES

The scenario matrix **MUST** cover at minimum:

1. Provider access
2. Model availability
3. Context lifecycle
4. Context pruning
5. Context expansion
6. Context compression
7. Context invalidation
8. User mutation

9. Workflow mutation
10. File lifecycle
11. Permission lifecycle
12. Policy lifecycle
13. Agent lifecycle
14. Sub-agent lifecycle
15. Coding-agent lifecycle
16. Tool lifecycle
17. MCP lifecycle
18. Memory lifecycle
19. Retrieval lifecycle
20. Cache lifecycle
21. Long-running execution
22. Waiting
23. Pause/resume
24. Cancellation
25. Interruption
26. Recovery
27. Checkpointing
28. Supersession
29. Concurrency
30. Retry
31. Budget exhaustion
32. Quality failure
33. Security failure
34. Provider failure
35. Infrastructure failure
36. Cost-accounting failure
37. Observability failure
38. Optimizer self-failure
39. Adversarial behavior
40. Disaster recovery

34. POSITIVE SCENARIOS

Do NOT create only failure cases.

The matrix MUST also include successful scenarios such as:

- successful exact cache hit

- successful semantic cache hit
 - successful context pruning
 - successful context expansion
 - successful model routing
 - successful fallback
 - successful agent resume
 - successful partial workflow execution
 - successful permission grant
 - successful permission revocation handling
 - successful dynamic file attachment
 - successful context-window increase
 - successful context-window reduction
 - successful tool suppression
 - successful sub-agent avoidance
 - successful cost reduction
 - successful quality-preserving compression
 - successful coding-agent repository expansion
 - successful long-running task continuation
-

35. COMPOUND SCENARIOS

The matrix **MUST** include multi-event scenarios.

Examples:

Scenario A

Context changed

+

permission revoked

+

model unavailable

Scenario B

Compression

+

quality failure

+

retry

+
cost budget nearly exhausted

Scenario C

User changes workflow
+
agent is executing
+
non-idempotent tool is running

Scenario D

File attached
+
context window reduced
+
existing result becomes stale

Scenario E

Provider unavailable
+
internal gateway available
+
fallback model has smaller context

Scenario F

Long wait
+
policy update
+
permission change
+
context becomes stale

36. CODING-AGENT SCENARIO SUITE

Create a dedicated section for coding agents.

Include:

- targeted file context
 - repository-wide context expansion
 - symbol dependency discovery
 - build output
 - test output
 - terminal output
 - Git state
 - branch changes
 - uncommitted changes
 - file changes
 - permission changes
 - generated files
 - dependency changes
 - user instruction changes
 - task cancellation
 - agent retry
 - tool retry
 - context expansion
 - context reduction
 - model fallback
 - model capability mismatch
 - stale patch
 - conflicting patch
 - external repository changes
-

37. GOVERNANCE SCENARIOS

Include:

- policy blocks optimization
- policy blocks model
- policy blocks tool
- policy blocks context access
- permission revoked
- approval required
- emergency policy
- policy version changed
- tenant policy changed
- compliance requirement changes mid-task

38. ECONOMIC SCENARIOS

Include:

- optimization saves tokens
 - optimization saves money
 - optimization costs more than it saves
 - cache saves money
 - cache costs more than it saves
 - cheaper model causes retries
 - expensive model avoids retries
 - optimizer latency reduces business value
 - tool suppression increases downstream cost
 - sub-agent spawning increases total cost
 - context compression increases inference quality risk
-

39. QUALITY SCENARIOS

Include:

- quality preserved
 - quality improved
 - quality degraded
 - validator unavailable
 - evaluator disagreement
 - quality evidence insufficient
 - quality threshold barely passed
 - quality threshold failed
 - regression after optimization
 - regression after model fallback
-

40. SECURITY SCENARIOS

Include:

- unauthorized context access
- revoked permission
- cross-tenant cache attempt

- sensitive context compression
- sensitive data in logs
- prompt injection
- tool injection
- memory poisoning
- malicious retrieved context
- malicious file
- policy bypass attempt
- stale authorization
- cached result after authorization revocation

Security MUST override cost optimization.

41. OBSERVABILITY SCENARIOS

Include:

- telemetry available
 - telemetry unavailable
 - partial telemetry
 - missing token counts
 - missing cost data
 - trace propagation failure
 - audit service unavailable
 - metrics backend unavailable
 - sensitive data detected in telemetry
-

42. SCENARIO-TO-DOCUMENT TRACEABILITY

Create explicit traceability links:

Problem

↓

Requirement

↓

Architecture

↓

Interface

↓
Convention
↓
Scenario
↓
Optimization
↓
Quality Gate
↓
Evaluation
↓
Implementation
↓
Observability

Every high-risk scenario must eventually have:

- design support
- interface support
- test support
- evaluation support
- observability support

43. DOCUMENT MODIFICATION REQUIREMENTS

Update the existing documents as follows.

```
problemStatement.txt
```

Add:

- Dynamic Execution State & Context Lifecycle Management
- Hybrid state ownership
- Context integrity
- Context expansion
- dynamic permissions
- dynamic model availability
- dynamic workflow mutation
- user mutation
- interruptions
- checkpoints

- recovery
- resumability
- supersession
- internal-only deployment
- provider gateway deployment
- scenario validation
- positive + negative scenario coverage

Do NOT remove existing requirements.

`architecture.md`

Add/update architecture for:

- Execution State Manager
- Context Version Manager
- Workflow Version Manager
- Checkpoint Manager
- Reconciliation Engine
- Context Integrity Gate
- Context Expansion Controller
- Permission Revalidation
- Dynamic Policy Evaluation
- Capability/Availability Resolver
- Stale Result Protection
- Supersession Manager
- Recovery Coordinator

Show how these interact with existing components.

Do NOT duplicate existing components unnecessarily.

If an existing component already performs the responsibility, extend it instead of creating a redundant component.

`interfaces.md`

Add contracts for:

- ExecutionState
- ExecutionVersion

- ContextVersion
- WorkflowVersion
- ContextMutation
- ContextExpansionRequest
- ContextInvalidation
- PermissionChange
- PolicyChange
- CapabilityChange
- ModelAvailability
- Checkpoint
- ResumeRequest
- CancelRequest
- Supersession
- Reconciliation
- StaleResult
- RecoveryDecision
- ContextIntegrityResult

For each contract define:

- purpose
- input
- output
- invariants
- errors
- idempotency
- authorization
- versioning
- observability
- compatibility

Do NOT break existing interfaces.

`conventions.md`

Add mandatory conventions for:

- versioning
- immutable execution identity
- optimistic concurrency
- state transitions

- stale result rejection
 - context provenance
 - permission revalidation
 - checkpointing
 - resumability
 - cancellation
 - supersession
 - idempotency
 - side-effect classification
 - dynamic configuration
 - context ownership
 - state ownership
 - recovery
 - scenario traceability
-

44. BACKWARD COMPATIBILITY

Existing clients must continue to work.

If dynamic execution functionality is unavailable, the system should safely operate in a simpler request/response mode where architecture permits.

Do not make every existing integration require complex session management unless necessary.

Use capability negotiation.

45. PERFORMANCE REQUIREMENT

Dynamic execution management must not introduce excessive overhead.

Do not:

- re-optimize the entire workflow after every minor mutation
- re-tokenize/recompute unchanged context unnecessarily
- invalidate unrelated caches
- restart entire workflows unnecessarily
- duplicate large context payloads

- persist redundant state

Prefer:

- immutable references
 - incremental updates
 - dependency-aware invalidation
 - delta state
 - checkpoints
 - selective recomputation
 - event-driven updates
-

46. SECURITY REQUIREMENT

Never sacrifice authorization, tenant isolation, or data security for optimization.

Examples:

A cache hit is NOT sufficient if authorization changed.

A compressed context is NOT sufficient if provenance/security metadata was lost.

A checkpoint is NOT safe if it contains data the current principal is no longer authorized to access.

A resumed workflow MUST revalidate authorization where required.

47. COST REQUIREMENT

Dynamic state management must be economically aware.

For every recovery/recompute decision, consider:

- tokens
- inference cost
- tool cost
- optimization cost
- infrastructure cost
- latency

- expected quality
- expected task value

Prefer selective recomputation over full restart when safe.

48. QUALITY REQUIREMENT

A resumed or dynamically modified execution must not assume that previous quality validation remains valid if material inputs changed.

Re-run only the necessary quality gates based on dependency impact.

49. FINAL CONSISTENCY AUDIT

After modifying the documents, perform an explicit consistency audit.

Verify:

Requirements

No existing requirement was lost.

Architecture

Every new requirement has an architectural home.

Interfaces

Every new architectural responsibility has an interface/contract where appropriate.

Conventions

Every important behavior has an engineering convention.

Scenarios

Every major requirement has positive and negative scenarios.

Security

Every dynamic state transition respects authorization and tenant isolation.

Cost

Every optimization/recovery decision accounts for net economics.

Quality

Every material context/model/workflow change can trigger appropriate revalidation.

Recovery

Every interruptible operation has explicit recovery semantics.

Agent support

Autonomous agents and sub-agents remain supported.

Coding agents

Developer/coding agents can dynamically request additional context.

Enterprise deployment

Organizations without direct external-provider access remain fully supported.

Provider neutrality

No architecture decision requires one provider.

50. REQUIRED OUTPUTS

Modify these existing files:

1. `problemStatement.txt`
2. `architecture.md`
3. `interfaces.md`
4. `conventions.md`

Create:

5. `scenario-matrix.md`

Do NOT create implementation code.

Do NOT create:

- `edge-cases.md`
- `optimization-catalog.md`
- `implementation-plan.md`
- `eval.md`
- `security.md`
- `observability.md`
- `SCALING.md`

Those will be generated later.

51. CHANGE LOG

At the end of each modified document, add a concise:

Change Impact Summary

containing:

- what was added
- what existing sections were affected
- why the change was necessary
- compatibility impact
- architectural impact
- unresolved questions

Do NOT duplicate the entire document.

52. SCENARIO MATRIX COMPLETENESS GATE

Before declaring the work complete, verify that `scenario-matrix.md` includes at least:

- 100+ distinct scenarios

- positive scenarios
- negative scenarios
- boundary scenarios
- dynamic mutation scenarios
- compound scenarios
- security scenarios
- coding-agent scenarios
- long-running scenarios
- interruption/recovery scenarios
- provider-access scenarios
- model-access scenarios
- context lifecycle scenarios
- permission lifecycle scenarios
- workflow lifecycle scenarios
- optimizer-self-failure scenarios

Do not inflate the count with trivial duplicates.

Prefer meaningful scenario diversity.

53. FINAL OUTPUT FORMAT

After generating the files, provide a concise implementation report containing:

Files Updated

- `problemStatement.txt`
- `architecture.md`
- `interfaces.md`
- `conventions.md`

File Created

- `scenario-matrix.md`

Requirements Added

List the major newly introduced capabilities.

Architectural Changes

List newly introduced or extended components.

Interface Changes

List newly introduced contracts.

Scenario Coverage

Report:

- total scenarios
- positive scenarios
- negative scenarios
- compound scenarios
- security scenarios
- coding-agent scenarios
- recovery scenarios
- dynamic mutation scenarios

Compatibility Assessment

Identify any backward-compatibility concerns.

Contradictions Found

List any contradictions with existing documents.

If none:

None found.

Open Questions

List unresolved architectural questions.

Recommended Next Step

Recommend the next document to generate based on dependency order.

Do not generate implementation code.

Why I recommend this change now

This gives your project a crucial distinction:

Static optimization

“Here is the prompt; make it cheaper.”

versus

Control-plane optimization

“Here is a living enterprise execution. Its context, permissions, workflow, files, models, tools, policies and external state can change at any point. Continuously determine what remains valid, what must be invalidated, what can be reused, what must be recomputed, and what is safe, useful and economically justified.”

That second definition is much closer to the product you are actually building.

After Antigravity completes this

I would **not immediately generate the next document**.

First, bring me the generated:

- `problemStatement.txt`
- `architecture.md`
- `interfaces.md`
- `conventions.md`
- `scenario-matrix.md`

Then we should perform a **cross-document consistency audit**.

If that passes, the next sequence should be:

`edge-cases.md` → `optimization-catalog.md` → `provider-matrix.md` → `cache-strategy.md` → `agent-optimization.md` → `inference-optimization.md` → `quality-gates.md` → `eval.md` → `security.md` → `observability.md` → `SCALING.md` → `implementation-plan.md` → **ADRs**.

That ordering will prevent the optimization catalog from being designed in isolation from the dynamic execution and failure model.

Today 3:51 AM



4a1c3115-10a1-4489-a...
File

This file is generated before all these additions. So, i wanted include all these steps for this file as well with proper edge cases for the new scenarios as well. Also, consider If the user is going beyond the context scenario

Worked for 9s

Yes. In that case I would **change the master prompt slightly from the previous version.**

The important distinction is that your current `problemStatement.txt` is already a substantial specification with optimization domains, acceptance criteria, failure/fallback concepts, agent/coding-agent optimization, context lifecycle, and external validation. It already explicitly recognizes that context can exceed the model limit after optimization and has a failure/fallback model.

Enterprise_Agent_LLM_Inference_...

So we should **not tell Antigravity to “add some new sections.”** We should tell it to perform a **requirements-preserving second-pass architecture hardening** over the entire existing specification.

And yes, **“user goes beyond the context window” needs to be much broader than simply truncating the prompt.** The Control Plane should distinguish *logical task context* from *the currently admitted model context*. That is a very important architectural principle.

Here is the master prompt I recommend using.

Edit



MASTER PROMPT

Enterprise Agent & LLM Inference Optimization Control Plane

Requirements Hardening — Dynamic Execution, Context Overflow, Recovery & Scenario Completeness

You are performing a SECOND-PASS HARDENING and REQUIREMENTS COMPLETENESS REVIEW of an already-developed enterprise architecture specification.

The product is:

Enterprise Agent & LLM Inference Optimization Control Plane

The existing specification was created before a new set of dynamic execution, context lifecycle, context-overflow, agent, coding-agent, permission, workflow, recovery, and enterprise deployment scenarios were identified.

Your job is to incorporate these new requirements into the EXISTING specification without losing, weakening, contradicting, or silently changing anything that has already been established.

This is NOT a rewrite.

This is NOT a simplification.

This is a controlled requirements expansion and consistency-hardening exercise.

1. AUTHORITATIVE EXISTING ARTIFACT

The existing `problemStatement.txt` is the baseline source document.

READ THE ENTIRE FILE before modifying it.

Do not rely only on section headings or search snippets.

Understand:

- existing objectives

- optimization domains
- optimization techniques
- provider/model strategy
- context strategy
- agent optimization
- coding-agent optimization
- caching
- retrieval
- inference optimization
- quality gates
- security
- governance
- observability
- evaluation
- cost accounting
- failure/fallback behavior
- deployment models
- maturity model
- acceptance criteria
- external validation references
- existing edge cases
- existing terminology
- existing identifiers

Preserve all valid existing requirements.

2. ABSOLUTE PRESERVATION RULE

The existing specification contains substantial prior work.

DO NOT:

- rewrite from scratch
- remove existing optimization techniques
- remove existing optimization domains
- remove existing agent requirements
- remove coding-agent requirements
- remove MCP requirements
- remove provider neutrality
- remove enterprise requirements

- remove security requirements
- remove quality requirements
- remove cost requirements
- remove observability requirements
- remove evaluation requirements
- remove caching
- remove context optimization
- remove inference optimization
- remove routing
- remove fallback
- remove model cascade
- remove reasoning controls
- remove tool optimization
- remove memory optimization
- remove acceptance criteria
- remove maturity levels
- remove external validation notes
- replace detailed requirements with vague statements

Do not shorten the specification merely because new requirements are being added.

If an existing section already addresses a new scenario, ENHANCE that section instead of creating unnecessary duplication.

3. NEW PRODUCT MATURITY REQUIREMENT

The Control Plane must now be treated as more than a request optimizer.

It is an:

Enterprise Agent & LLM Inference Optimization Control Plane

that governs a continuously changing execution lifecycle.

The system must optimize not only:

PROMPT

CONTEXT

MODEL

TOOLS
OUTPUT

but also:

EXECUTION STATE
CONTEXT STATE
WORKFLOW STATE
PERMISSION STATE
POLICY STATE
MODEL STATE
PROVIDER STATE
TOOL STATE
MEMORY STATE
CACHE STATE
EXTERNAL STATE

The system must be capable of determining:

What is the cheapest SAFE execution strategy that can still achieve the required outcome under the current context, authorization, quality, latency, reliability, freshness, compliance, and availability constraints?

4. NEW FIRST-CLASS CAPABILITY

Add this as a first-class requirement:

Dynamic Execution State & Context Lifecycle Management

A request **MUST NOT** be assumed to be immutable throughout execution.

Any of the following may change:

- user intent
- context
- files
- workflow
- permissions
- policies

- model availability
- provider availability
- tool availability
- MCP capabilities
- context-window size
- budget
- latency requirement
- quality requirement
- external data
- repository state
- memory state
- cache validity

The Control Plane must detect and safely handle these mutations.

5. CRITICAL CONCEPT — LOGICAL CONTEXT VS MODEL-ADMITTED CONTEXT

Introduce this architectural distinction.

5.1 Logical Task Context

The complete set of information that may be relevant to the task.

This can include:

- user instructions
- conversation history
- files
- repository information
- retrieved documents
- tool results
- memory
- prior decisions
- external data
- code dependencies
- test results
- system policies
- workflow state

- agent state

Logical context may be MUCH larger than the context window of any individual model invocation.

5.2 Model-Admitted Context

The subset of logical context actually supplied to a particular LLM invocation.

The Control Plane MUST NOT equate:

Context Window = Total Task Context

Instead:

Logical Task Context
↓
Relevance / Dependency / Freshness / Security
↓
Optimization
↓
Model Context Budget
↓
Model-Admitted Context

This distinction is mandatory.

6. USER GOES BEYOND THE CONTEXT WINDOW

Explicitly support the scenario where the user provides more context than the selected model can accept.

Examples:

- huge document
- many files
- complete repository
- large log files
- long conversation
- multiple attachments
- massive tool output

- multi-agent accumulated context
- very large codebase
- user explicitly asks to "include everything"

The system MUST NOT simply:

- truncate arbitrarily
- silently drop information
- reject the request immediately
- assume the oldest information is least relevant
- assume the newest information is most relevant
- blindly compress everything

Instead, it must determine the appropriate strategy.

Possible strategies include:

- relevance-based selection
- dependency-aware selection
- hierarchical summarization
- progressive compaction
- retrieval-on-demand
- context paging
- context windowing
- rolling summaries
- structured state extraction
- code-aware context selection
- tool-result reduction
- semantic compression
- external memory
- staged reasoning
- multi-pass execution
- model routing to larger-context models
- task decomposition
- subtask-specific contexts
- user clarification
- controlled refusal when required information cannot safely fit

7. CONTEXT OVERFLOW DECISION HIERARCHY

When:

Required Context > Available Model Context

the Control Plane must evaluate, in an appropriate policy-driven order:

1. Can irrelevant context be removed?
2. Can duplicate context be removed?
3. Can stale context be removed?
4. Can context be compressed safely?
5. Can context be summarized hierarchically?
6. Can context be retrieved on demand?
7. Can context be split into task-specific windows?
8. Can the task be decomposed?
9. Can a larger-context eligible model be used?
10. Can intermediate state be persisted externally?
11. Can the workflow execute in multiple stages?
12. Is user clarification required?
13. Is execution unsafe without the omitted information?

Do NOT prescribe one universal strategy.

The optimizer must make a context-admission decision based on:

- task intent
- dependency graph
- information utility
- freshness
- sensitivity
- model capability
- budget
- quality threshold
- latency SLO
- expected token savings
- optimization overhead
- execution risk

**8. NEVER CONFUSE TOKEN REDUCTION
WITH SUCCESS**

A smaller prompt is NOT automatically better.

The system must explicitly detect:

Token reduction

BUT

Information loss

and:

Token reduction

BUT

Additional retrieval/retry cost

and:

Token reduction

BUT

Quality degradation

and:

Token reduction

BUT

Task failure

Optimization must therefore maximize:

Net Safe Task Value

rather than raw token reduction.

9. CONTEXT PRESERVATION TIERS

Introduce a context classification model.

At minimum:

Tier 0 — Mandatory

Information that **MUST** be retained.

Examples:

- user constraints
- security policies

- explicit prohibitions
- critical task facts
- required interfaces
- safety requirements
- mandatory output requirements

Tier 1 — High Value

Strongly relevant information.

Tier 2 — Conditional

Relevant only for particular subtasks.

Tier 3 — Recoverable

Can be retrieved again.

Tier 4 — Redundant

Can safely be removed.

Tier 5 — Stale/Invalid

Must not be used.

The optimizer must not remove Tier 0 information simply to satisfy a token budget.

10. CONTEXT OVERFLOW WITH NO SAFE COMPRESSION

If the context exceeds the model limit and:

- required information cannot be compressed safely
- required information cannot be retrieved later
- larger models are unavailable
- decomposition is not possible
- task quality would be compromised

the system must NOT fabricate a solution.

Possible outcomes:

- pause
- request user clarification
- ask the user to narrow scope
- switch to an eligible larger-context model
- decompose the task
- execute a safe subset
- fail explicitly with a machine-readable reason

The system must explain what constraint caused the failure where appropriate.

11. CONTEXT OVERFLOW WITH MULTIPLE MODELS

If Model A cannot accommodate the required context but Model B can:

the routing engine may evaluate Model B.

But Model B must still satisfy:

- tenant policy
- authorization
- compliance
- region
- capability
- budget
- quality
- latency
- provider availability

Never route solely because Model B has a larger context window.

12. CONTEXT WINDOW IS NOT STATIC

Context limits may differ by:

- provider
- model
- deployment

- API mode
- tool mode
- structured-output mode
- reasoning mode
- system configuration

The Control Plane must maintain effective context capacity rather than assuming a universal context limit.

13. EFFECTIVE CONTEXT BUDGET

Define:

$$\begin{aligned} \text{Effective Context Budget} &= \\ &\text{Model Context Limit} \\ &- \\ &\text{System Prompt} \\ &- \\ &\text{Policy Overhead} \\ &- \\ &\text{Tool Definitions} \\ &- \\ &\text{Required Output Budget} \\ &- \\ &\text{Reasoning/Inference Reservation} \\ &- \\ &\text{Protocol Overhead} \\ &- \\ &\text{Safety Margin} \end{aligned}$$

The optimizer must use the effective available budget rather than blindly targeting the published context window.

Where provider behavior differs, use provider/model capability metadata.

14. CONTEXT WINDOW CHANGES DURING EXECUTION

Support:

128K → 32K

32K → 128K

128K → 1M

If context capacity decreases:

- preserve mandatory context
- recompute admission
- invalidate incompatible optimization decisions
- re-evaluate model routing
- re-run relevant quality gates

If context capacity increases:

- do NOT automatically expand the prompt
- retain the smallest sufficient context unless evidence shows additional context is useful

15. CONTEXT IS TOTALLY PRUNED

Support the scenario where optimization accidentally removes all useful context.

Detect:

- zero-context anomaly
- mandatory-context loss
- entity loss
- dependency loss
- instruction loss
- provenance loss
- task-state loss

If required information is missing:

- restore
- retrieve
- expand
- reconstruct
- pause
- request clarification

Never blindly continue.

16. CONTEXT WAS NEVER PROVIDED

Distinguish:

Legitimate no-context task

Example:

"What is 2 + 2?"

from:

Missing-context task

Example:

"Fix the bug in the attached project"

when the project is unavailable.

Do not manufacture missing context.

17. CODING AGENT CONTEXT EXPANSION

A coding agent may begin with a minimal context.

During execution it may discover that it needs:

- another source file
- interface definition
- dependency implementation
- configuration
- database schema
- tests
- build output
- compiler error
- runtime logs

- Git history
- API contract
- repository architecture
- documentation

The agent must be able to request context expansion.

The Control Plane must:

1. identify requested information
2. evaluate relevance
3. evaluate authorization
4. evaluate sensitivity
5. retrieve selectively
6. optimize the new context
7. preserve provenance
8. validate context integrity
9. continue execution

18. CODING AGENT OUT-OF-CONTEXT GENERATION

Explicitly support a coding agent requesting something that is not currently present in context.

Do NOT automatically classify this as:

- hallucination
- failure
- prompt error
- optimization failure

It may be a legitimate context-discovery event.

The system should determine whether the request is:

- valid dependency discovery
- repository exploration
- tool discovery
- documentation lookup
- missing requirement
- unauthorized access

- irrelevant request
- prompt injection
- malicious instruction

19. ORGANIZATION WITHOUT EXTERNAL PROVIDER ACCESS

The Control Plane MUST work when an organization cannot directly access external GenAI products.

Support:

```
Coding Agent
    ↓
Enterprise Control Plane
    ↓
Internal AI Gateway
    ↓
Approved Provider
```

and:

```
Agent
    ↓
Control Plane
    ↓
Internal AI Platform
    ↓
Private Model
```

and hybrid deployment.

The Control Plane must not require direct provider credentials when the enterprise gateway intentionally abstracts them.

20. PROVIDER CAPABILITY ABSTRACTION

Where the internal gateway hides provider information:

represent capability information as:

- provider-reported
- gateway-reported
- internally configured
- estimated
- unavailable
- unknown

Never invent token counts, prices, context limits, or model capabilities.

21. MODEL SUGGESTED BUT INACCESSIBLE

The optimizer may identify an ideal model that cannot be used.

Separate:

- discovered model
- recommended model
- eligible model
- authorized model
- accessible model
- available model
- capability-compatible model

Routing must only select the intersection of all required conditions.

22. DYNAMIC PERMISSIONS

Permissions may change during execution.

Examples:

- access granted
- access revoked
- role changed
- repository permission changed
- file permission changed
- tool permission changed
- policy changed

- approval expires

Sensitive operations **MUST** revalidate authorization at the point of execution.

Do not rely indefinitely on authorization established at task start.

23. POLICY CHANGES DURING EXECUTION

If policy changes:

- identify affected execution steps
 - invalidate affected decisions
 - stop prohibited operations
 - preserve unaffected work where safe
 - re-evaluate routing
 - re-evaluate context access
 - re-evaluate tools
 - re-evaluate caching
 - re-evaluate retention
 - record an audit event
-

24. DYNAMIC WORKFLOW CHANGES

Users may change workflows during execution.

Examples:

"Only do steps 1 and 2."

"Don't modify files."

"Don't run tests."

"Generate a plan only."

"Stop before committing."

"Now include the new requirements file."

The system must create a new workflow version and safely reconcile the running execution.

25. USER ATTACHES FILES MID-EXECUTION

Support:

- new file
- modified file
- deleted file
- replaced file
- inaccessible file
- permission change

The Control Plane must determine whether the change affects:

- current context
- previous decisions
- current workflow
- cached results
- quality gates
- pending tool calls

Invalidate only affected state when safe.

26. USER REMOVES CONTEXT MID-EXECUTION

If the user removes context:

- update context version
- identify dependencies
- invalidate affected decisions

- prevent stale results from becoming authoritative
 - continue unaffected work where safe
 - ask for clarification when required
-

27. USER CHANGES CONTEXT WINDOW MID-EXECUTION

If the user changes the desired context budget:

the Control Plane must treat it as a policy/configuration mutation.

It must NOT blindly:

- restart the entire workflow
- repack every context item
- discard valid results

Use dependency-aware recomputation.

28. LONG-WAIT SCENARIOS

A task may remain:

- waiting for user
- waiting for approval
- waiting for tool
- waiting for provider
- waiting for external system
- paused for hours/days

On resume, revalidate relevant:

- context
- permissions
- policies
- model availability
- provider availability
- external state
- cache validity
- workflow version

- budget
- task intent

Do not assume the world is unchanged.

29. EXECUTION INTERRUPTION

Support:

- network failure
- model failure
- provider timeout
- tool timeout
- agent crash
- Control Plane crash
- machine failure
- deployment
- user cancellation
- infrastructure outage
- budget exhaustion

Execution must have explicit state transitions.

30. CHECKPOINTING

For long-running/high-value workflows support checkpoints.

A checkpoint must identify enough information to establish:

- completed steps
- pending steps
- context version
- workflow version
- policy version
- authorization state
- model/provider
- tool operations
- side effects
- cost
- budget

- quality state
- recovery options

Use references/immutable state where possible rather than duplicating huge prompts.

31. RESUME MUST NOT MEAN REPLAY

Never define resume as:

Replay original prompt

Instead:

Checkpoint
+
Current User Intent
+
Current Context
+
Current Workflow
+
Current Authorization
+
Current Policy
+
Current External State
+
Current Model Availability
+
Completed Work
+
Remaining Work

Then decide:

- RESUME
- RECOMPUTE
- EXPAND CONTEXT
- ROLLBACK
- RESTART
- ABORT
- REQUEST USER INPUT

32. NON-IDEMPOTENT ACTIONS

Classify operations as:

- read-only
- idempotent
- conditionally idempotent
- non-idempotent
- irreversible

Never blindly replay:

- database writes
 - deployments
 - payments
 - emails
 - commits
 - ticket creation
 - external mutations
 - destructive operations
-

33. STALE RESULT PROTECTION

Every significant result must be associated with the relevant versions.

Example:

Result generated against:

Request V4

Context V7

Workflow V3

Policy V8

must not automatically overwrite:

Request V5

Context V8

Workflow V4

Policy V9

Implement stale-result detection and reconciliation.

34. SUPERSESION

If a new user request supersedes an existing execution:

Execution V1

↓

User changes requirement

↓

Execution V2

V1 should transition to:

SUPERSEDED

unless policy explicitly permits both executions.

Stale execution must not continue making unauthorized or unwanted side effects.

35. PARTIAL EXECUTION

Support workflows where only selected actions should execute.

Example:

1. Inspect
2. Modify
3. Test
4. Document
5. Commit

User:

"Only perform 1 and 2."

The Control Plane must suppress 3–5 and preserve completed work.

36. AGENT-OWNED VS CONTROL-PLANE-OWNED CONTEXT

Define three ownership domains.

Agent working state

- transient reasoning
- scratch state
- temporary plans
- agent-specific intermediate state

Control Plane state

- request
- execution state
- context versions
- workflow versions
- policies
- authorization decisions
- optimization decisions
- checkpoints
- provenance
- cost
- quality state
- recovery metadata
- audit metadata

External authoritative state

- repository
- files
- database
- identity
- permissions
- external APIs
- business systems

The Control Plane must not assume agent memory is durable or authoritative.

37. CONTEXT RETENTION

Do not retain everything by default.

Retain information according to:

- security
- compliance
- audit
- recovery
- reproducibility
- optimization evaluation
- cost attribution
- quality evaluation
- debugging
- workflow continuation

Support:

- TTL
 - configurable retention
 - encryption
 - redaction
 - minimization
 - tenant-specific policies
 - deletion
 - legal/compliance holds
-

38. CONTEXT PROVENANCE

For retained or reconstructed context, track where practical:

- source
- source version
- timestamp
- owner
- authorization scope
- sensitivity
- transformation history
- compression history
- retrieval history

- optimization history
 - validity/freshness
-

39. CACHE VALIDITY UNDER DYNAMIC STATE

A cached result must not be reused merely because the query is semantically similar.

Evaluate:

- authorization
- tenant
- policy
- context version
- source freshness
- dependency versions
- workflow
- model requirements
- task intent

A cached result invalidated by a relevant state change must not be served.

40. NEGATIVE OPTIMIZATION SCENARIOS

Explicitly support:

- compression increases retries
- pruning causes failure
- cheaper model lowers quality
- cache returns stale information
- semantic cache produces false hit
- tool suppression removes required information
- routing causes additional calls
- optimization latency exceeds savings
- optimization compute exceeds inference savings
- context reduction causes excessive retrieval

- model downgrade causes longer output
- sub-agent avoidance increases task complexity

The optimizer must be allowed to select:

`DO_NOT_OPTIMIZE`

41. OPTIMIZER FAILURE

The optimizer itself may fail.

Examples:

- tokenizer unavailable
- classifier unavailable
- retrieval unavailable
- compression model unavailable
- routing model unavailable
- cache unavailable
- policy service unavailable
- cost data unavailable
- evaluator unavailable

The underlying task should continue safely where policy permits.

Optimization must be fail-safe.

42. FAIL-OPEN VS FAIL-CLOSED

Explicitly define which components fail:

Fail closed

- authorization
- security policy
- tenant isolation
- sensitive-data controls
- prohibited tool execution

Potentially fail open / fallback

- optimization
- compression
- caching
- routing intelligence
- cost estimation

provided doing so does not violate policy or safety.

Do not use one global fail-open/fail-closed rule.

43. COMPOUND FAILURE SCENARIOS

The specification must explicitly consider simultaneous failures.

Examples:

A

Context exceeds model limit

+

larger model unavailable

+

required context cannot be safely compressed.

B

Permission revoked

+

agent has pending tool call

+

cached result exists.

C

User changes workflow

+

model is generating

+

non-idempotent action is pending.

D

File attached

+

context window reduced

+

existing optimization becomes stale.

E

Provider unavailable

+

fallback model has smaller context

+

budget is nearly exhausted.

F

Long wait

+

policy changes

+

context becomes stale

+

permission changes.

44. POSITIVE SCENARIOS ARE MANDATORY

Do not document only failures.

Include successful cases:

- context successfully compressed
- context successfully expanded
- context successfully retrieved on demand
- larger model selected
- smaller model selected safely
- cache successfully reused
- stale cache correctly rejected
- permission granted mid-task
- permission revoked safely
- user adds file successfully

- user removes file successfully
 - workflow partially executed
 - task resumed from checkpoint
 - coding agent expands repository context
 - model fallback succeeds
 - optimizer correctly chooses DO_NOT_OPTIMIZE
 - task continues safely after optimizer failure
-

45. NEW SCENARIO MATRIX REQUIREMENT

Create a new file:

```
scenario-matrix.md
```

This becomes the master validation artifact.

It must be generated AFTER understanding the entire hardened

```
problemStatement.txt .
```

46. SCENARIO MATRIX MINIMUM STRUCTURE

Each scenario must contain:

- Scenario ID
- Scenario family
- Scenario title
- Positive/negative/boundary/compound classification
- Actor
- Trigger
- Preconditions
- Initial state
- State mutation
- Context mutation
- Workflow mutation
- Permission mutation
- Policy mutation
- Model/provider mutation

- Tool mutation
 - Expected Control Plane behavior
 - Expected optimizer behavior
 - Expected agent behavior
 - Expected result
 - Fallback
 - Recovery
 - Rollback
 - Security implications
 - Quality implications
 - Cost implications
 - Latency implications
 - Freshness implications
 - Audit requirements
 - Observability requirements
 - Metrics
 - Traces
 - Alerts
 - Required test
 - Severity
 - Priority
 - Related requirement
 - Related architecture component
 - Related interface
 - Related convention
 - Related optimization
 - Open questions
-

47. SCENARIO FAMILIES

At minimum include:

1. Request lifecycle
2. Context lifecycle
3. Context overflow
4. Context compression
5. Context pruning
6. Context expansion
7. Context retrieval

8. Context invalidation
9. Context versioning
10. Context-window changes
11. User mutation
12. Workflow mutation
13. File lifecycle
14. Permission lifecycle
15. Policy lifecycle
16. Model availability
17. Provider availability
18. Internal gateway deployment
19. Tool lifecycle
20. MCP lifecycle
21. Agent lifecycle
22. Sub-agent lifecycle
23. Coding-agent lifecycle
24. Memory lifecycle
25. Cache lifecycle
26. Long-running execution
27. Waiting
28. Pause/resume
29. Cancellation
30. Interruption
31. Checkpointing
32. Recovery
33. Supersession
34. Concurrency
35. Retry
36. Budget exhaustion
37. Quality degradation
38. Security failure
39. Optimizer failure
40. Observability failure
41. Disaster recovery
42. Adversarial behavior

48. CONTEXT OVERFLOW SCENARIO SUITE

The scenario matrix MUST specifically include:

- user input larger than context limit
- attachments larger than context limit
- repository larger than context limit
- conversation larger than context limit
- tool results larger than context limit
- combined context larger than context limit
- context exceeds limit after optimization
- context exceeds limit after output reservation
- context exceeds limit after tool definitions
- context exceeds limit because system prompt grows
- context exceeds limit because reasoning budget changes
- no model with sufficient context exists
- larger-context model exists but is inaccessible
- larger-context model exists but violates policy
- larger-context model exists but is too expensive
- compression loses required information
- retrieval-on-demand succeeds
- retrieval-on-demand fails
- decomposition succeeds
- decomposition causes quality degradation
- user insists on including everything
- mandatory context alone exceeds model capacity
- multiple mandatory sources conflict
- context window shrinks during execution
- context window increases during execution
- context overflow after workflow mutation
- context overflow after file attachment
- context overflow after tool output
- context overflow after agent-generated context
- context overflow in multi-agent orchestration

49. CODING-AGENT SCENARIO SUITE

Include at minimum:

- targeted file optimization
- repository-wide context request
- dependency discovery

- symbol discovery
 - test discovery
 - build-error retrieval
 - runtime-log retrieval
 - Git state retrieval
 - branch changes
 - uncommitted changes
 - user adds file
 - user removes file
 - file changes during execution
 - permission changes
 - context expansion
 - context overflow
 - model fallback
 - model capability mismatch
 - stale patch
 - conflicting patch
 - external repository change
 - agent requests unauthorized file
 - agent requests prohibited tool
 - agent requests external resource
 - agent requests context outside policy
 - agent wants to continue despite superseded workflow
-

50. ENTERPRISE DEPLOYMENT SCENARIO SUITE

Include:

- direct provider
- enterprise gateway
- internal AI platform
- private model
- hybrid provider
- provider hidden behind gateway
- token usage unavailable
- cost data unavailable
- model capabilities unavailable
- gateway timeout

- gateway policy rejection
 - provider unavailable behind gateway
 - tenant-specific provider restrictions
-

51. SECURITY SCENARIO SUITE

Include:

- unauthorized context
 - revoked permission
 - stale authorization
 - cross-tenant cache
 - sensitive context in logs
 - sensitive context in checkpoints
 - sensitive context in telemetry
 - prompt injection
 - tool injection
 - malicious file
 - memory poisoning
 - malicious retrieved document
 - cache poisoning
 - policy bypass
 - stale result after permission revocation
-

52. ECONOMIC SCENARIO SUITE

Include:

- gross token savings
- net token savings
- optimization overhead
- negative ROI optimization
- cache ROI
- compression ROI
- routing ROI
- model downgrade ROI
- model upgrade ROI
- retry cost
- recovery cost

- tool-call savings
 - avoided LLM calls
 - avoided sub-agent calls
-

53. QUALITY SCENARIO SUITE

Include:

- quality preserved
 - quality improved
 - quality degraded
 - compression failure
 - pruning failure
 - routing failure
 - evaluator unavailable
 - validator unavailable
 - insufficient quality evidence
 - quality regression after fallback
 - quality regression after resume
 - quality regression after context mutation
-

54. TRACEABILITY REQUIREMENT

Every major requirement must eventually map to:

Requirement

↓

Architecture

↓

Interface

↓

Convention

↓

Scenario

↓

Test

↓

Evaluation

↓

Observability

The current `problemStatement.txt` should therefore become the source of truth for downstream documents.

55. ACCEPTANCE CRITERIA

Add acceptance criteria for:

Context overflow

The system must never silently lose mandatory context solely to fit a token limit.

Dynamic context

The system must support context expansion and reduction.

Dynamic workflow

The system must support workflow mutation.

Dynamic permissions

The system must revalidate authorization where required.

Model accessibility

The system must never route to an inaccessible model.

Recovery

Interrupted tasks must have explicit recovery semantics.

Supersession

Newer user intent must be protected from stale execution.

Partial execution

Users must be able to restrict workflow scope.

Context retention

Retention must be policy-controlled.

Coding agents

Coding agents must be able to request additional context.

Enterprise gateways

Direct external provider access must not be mandatory.

Optimization safety

Optimization must be allowed to return:

```
DO_NOT_OPTIMIZE
```

Quality

Context reduction must not be considered successful merely because tokens decreased.

56. COMPLETENESS REVIEW OF EXISTING EDGE CASES

The existing problem statement already contains edge/failure requirements.

Do NOT duplicate them blindly.

Instead:

1. identify existing coverage
 2. identify gaps
 3. strengthen existing sections
 4. add only missing scenarios
 5. cross-reference related existing requirements
-

57. INTERNAL CONSISTENCY REVIEW

After modification, check for contradictions between:

- context optimization
- model routing
- caching
- memory

- agent optimization
- coding-agent optimization
- security
- quality gates
- cost accounting
- fallback
- inference optimization
- provider abstraction
- deployment requirements

Example contradiction to detect:

"Always minimize context"

versus:

"Preserve all mandatory information."

The correct rule must be:

Minimize context subject to mandatory information, quality, security, freshness, and task-completion constraints.

58. PERFORMANCE REQUIREMENT

Dynamic control-plane logic must itself be optimized.

Do not:

- re-optimize unchanged context
- rebuild entire workflows after tiny changes
- invalidate unrelated caches
- recompute unrelated dependencies
- persist huge duplicate state
- invoke LLMs unnecessarily for state management

Prefer:

- versioning
- deltas

- dependency graphs
 - incremental recomputation
 - immutable references
 - checkpoints
 - selective invalidation
 - local deterministic logic where possible
-

59. SCENARIO COUNT

`scenario-matrix.md` must contain at least:

150 meaningful scenarios

Do not satisfy the number using trivial wording variations.

The scenarios should represent genuinely different behaviors, risks, states, or recovery paths.

At minimum include:

- 40 positive
- 40 negative
- 20 boundary
- 20 dynamic mutation
- 15 compound
- 15 coding-agent
- 15 context-overflow
- 15 security
- 10 recovery/interruption

Categories may overlap.

60. FINAL VALIDATION

Before completing the work, perform a final audit.

Verify:

Existing requirements

No valid requirement was removed.

New scenarios

All newly identified scenarios are represented.

Context

Context overflow is handled explicitly.

Context ownership

Agent and Control Plane ownership is clearly defined.

Context retention

Retention is governed.

Context integrity

Mandatory context cannot silently disappear.

Agents

Autonomous agents remain supported.

Coding agents

Developer/coding agents remain first-class.

Providers

Internal gateway and inaccessible-provider scenarios are supported.

Models

Inaccessible models cannot be selected.

Permissions

Dynamic authorization is handled.

Workflow

Dynamic workflow changes are handled.

Files

Dynamic file lifecycle is handled.

Recovery

Interruption/resume is handled.

Side effects

Non-idempotent operations are protected.

Cost

Optimization overhead is accounted for.

Quality

Token reduction alone is not treated as success.

Security

Security overrides optimization.

Observability

Dynamic state changes are observable.

Traceability

Requirements map to scenarios.

61. REQUIRED OUTPUT FILES

MODIFY:

```
problemStatement.txt
```

CREATE:

```
scenario-matrix.md
```

Do NOT create the downstream architecture documents yet.

Do NOT modify:

- ```
architecture.md
```
- ```
interfaces.md
```
- ```
conventions.md
```

- `edge-cases.md`
- `optimization-catalog.md`
- `provider-matrix.md`
- `cache-strategy.md`
- `agent-optimization.md`
- `inference-optimization.md`
- `quality-gates.md`
- `eval.md`
- `security.md`
- `observability.md`
- `SCALING.md`
- `implementation-plan.md`
- ADRs

Those will be regenerated or reconciled later against this hardened source.

---

## 62. CHANGE MANAGEMENT

At the end of `problemStatement.txt`, add:

### Second-Pass Hardening Summary

Document:

- newly added capabilities
- newly added scenario families
- context-overflow enhancements
- dynamic execution enhancements
- coding-agent enhancements
- enterprise gateway enhancements
- recovery enhancements
- security enhancements
- cost/quality enhancements

Do not repeat the whole specification.

---

## 63. FINAL REPORT

After generating the files, report:

## **Existing Requirements Preserved**

List major existing domains confirmed preserved.

## **New Requirements Added**

List all major new capability groups.

## **Context Overflow Coverage**

Report the number of context-overflow scenarios.

## **Dynamic Execution Coverage**

Report scenarios covering:

- context mutation
- workflow mutation
- permission mutation
- policy mutation
- model mutation
- provider mutation
- tool mutation
- file mutation
- user mutation

## **Coding-Agent Coverage**

Report coding-agent scenario count.

## **Recovery Coverage**

Report interruption/checkpoint/resume/recovery scenarios.

## **Security Coverage**

Report security scenario count.

## **Compound Scenarios**

Report compound scenario count.

## **Contradictions Found**



List contradictions found and how they were resolved.

If none:

None found.

## Open Questions

List unresolved architecture questions.

## Recommended Next Step

Recommend the next document based on dependency order.

Do NOT generate implementation code.

## One particularly important addition

I would pay special attention to the **Logical Task Context vs Model-Admitted Context** distinction.

For example, suppose a developer gives the coding agent a **2-million-token repository**, while the selected model can only accept 128K tokens.

The Control Plane shouldn't conclude:

“The context is too large, therefore we must truncate it.”

Instead:

2M-token Logical Task Context

|  
├─ User intent  
├─ Repository  
├─ Dependencies  
├─ Tests  
├─ Git state  
├─ Documentation  
└─ Tool results

|

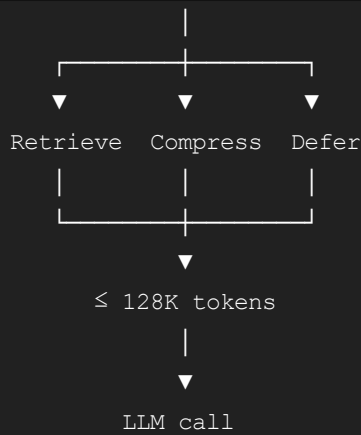
▼

Dependency / Relevance  
/ Freshness / Security

|

▼

Context Admission



And if the agent subsequently says:

“I need `PaymentRepository.java` and the database migration.”

the Control Plane should be able to **expand the logical context and admit those specific pieces**, rather than having to start the entire task again.

This is the architectural behavior that will make the product substantially more powerful for **Cursor/Claude Code/Copilot/Antigravity-style coding agents**, not merely ordinary chat applications.

Also, your existing specification already has the right philosophical foundation: the optimizer should optimize the *complete inference lifecycle*, rather than merely maximize token reduction. `Enterprise_Agent_LLM_Inference_...`

I would run this prompt against the existing `problemStatement.txt` first. Once Antigravity produces the hardened version plus `scenario-matrix.md`, we should use those as the new baseline before touching `architecture.md`, `interfaces.md`, `conventions.md`, or `edge-cases.md`.

I would give the previous prompt first then we will chnge the edge-cases.md file accorind to that later we will add the scenarios-matrix.md file then we will modify the future genrated docs. suggest me so that I can give the previous prompt for modifying only \*

[Ent\_Agent\_LLM\_Inference\_Opt\_Control\_Plane\_problems tatement.txt]

(file;file:///d%3A/GenAI/Practice/Tok\_Agent/docs/Ent\_Agent\_LLM\_Inference\_Opt\_Control\_Plane\_problemstatement.txt)

Show more

Yes — that is the better sequence.

I recommend **not introducing** `scenario-matrix.md` **yet**. First establish the new requirements consistently across the existing baseline documents, then harden `edge-cases.md`, then create the scenario matrix from the stabilized requirements.

Your sequence should be:

```
1. problemStatement.txt
 ↓
2. Engineering Spec.md
 ↓
3. architecture.md
 ↓
4. interfaces.md
 ↓
5. conventions.md
 ↓
6. edge-cases.md
 ↓
7. scenario-matrix.md
 ↓
8. Future documents
```

The important thing is that **step 1–6 should be one controlled consistency pass**, rather than letting Antigravity independently invent interpretations in each document.

## What I recommend you do now

Give Antigravity the **previous prompt**, but add a very strict scope instruction at the beginning/end:

**For this pass, modify ONLY these six existing files. Do not create `scenario-matrix.md` yet and do not modify any other files.**

The six files are:

- `Ent_Agent_LLM_Inference_Opt_Control_Plane_problemstatement.txt`
- `Ent_Agent_LLM_Inference_Opt_Control_Plane_Engineering_Spec.md`
- `architecture.md`
- `interfaces.md`
- `conventions.md`

- `edge-cases.md`

And importantly, tell it that `edge-cases.md` **must be updated only after reconciling the first five documents**, so it doesn't become an independent interpretation.

I would append this to the previous prompt:

 Edit



## SCOPE-CONTROL ADDENDUM — CURRENT PASS

### IMPORTANT: THIS IS A CONTROLLED REQUIREMENTS-RECONCILIATION PASS

The previous master prompt defines the new requirements and scenarios that must be incorporated.

For THIS PASS ONLY, modify and reconcile ONLY the following six existing files:

1. `Ent_Agent_LLM_Inference_Opt_Control_Plane_problemstatement.txt`
2. `Ent_Agent_LLM_Inference_Opt_Control_Plane_Engineering_Spec.md`
3. `architecture.md`
4. `interfaces.md`
5. `conventions.md`
6. `edge-cases.md`

These six files are the ONLY files that may be modified in this pass.

---

## DO NOT CREATE OR MODIFY ANY OTHER FILE

Do NOT create:

- `scenario-matrix.md`
- `implementation-plan.md`
- `eval.md`

- SCALING.md
- security.md
- observability.md
- optimization-catalog.md
- provider-matrix.md
- cache-strategy.md
- agent-optimization.md
- inference-optimization.md
- quality-gates.md
- ADR files
- implementation code
- configuration files
- source code

Do NOT modify any other existing documentation.

The scenario matrix will be created in a later controlled pass.

---

## DOCUMENT AUTHORITY AND ORDER

Treat the documents in this order of conceptual authority:

1. problemStatement.txt
2. Engineering\_Spec.md
3. architecture.md
4. interfaces.md
5. conventions.md
6. edge-cases.md

The first document establishes WHAT the product must achieve.

The Engineering Specification establishes the detailed engineering interpretation.

The Architecture establishes HOW the system is structured.

Interfaces establish HOW components communicate.

Conventions establish HOW implementations must behave consistently.

Edge Cases establishes HOW exceptional, negative, boundary, dynamic, and failure conditions are handled.

Do NOT allow a lower-level document to contradict a higher-level requirement.

---

## **REQUIRED PROCESS**

Before editing anything:

### **STEP 1 — READ ALL SIX FILES COMPLETELY**

Do not edit after reading only one file.

Understand the complete existing system first.

Create an internal inventory of:

- existing requirements
  - existing architecture decisions
  - existing interfaces
  - existing conventions
  - existing edge cases
  - optimization techniques
  - agent requirements
  - coding-agent requirements
  - context management
  - model routing
  - provider abstraction
  - caching
  - inference optimization
  - security
  - quality
  - cost
  - observability
  - failure handling
- 

### **STEP 2 — PRESERVE EVERYTHING VALID**

This is a requirements expansion and consistency-hardening operation.

It is NOT a rewrite.

Do not remove existing valid requirements.

Do not replace detailed requirements with generic language.

Do not simplify away previously established optimization techniques.

Do not delete existing edge cases merely because new edge cases are being introduced.

Where an existing requirement already covers a new scenario:

- strengthen it
- clarify it
- cross-reference it
- avoid unnecessary duplication

---

## STEP 3 — INCORPORATE THE NEW DYNAMIC EXECUTION MODEL

Ensure all six documents consistently understand that:

Request

Context

Workflow

Permissions

Policies

Models

Providers

Tools

Files

Memory

Cache

External State

may change during execution.

The Control Plane must therefore be stateful, version-aware, mutation-aware, and recovery-aware.

---

## STEP 4 — CONTEXT MODEL MUST BE HARDENED

All six documents must consistently distinguish:

### Logical Task Context

The complete information potentially relevant to the task.

from:

### Model-Admitted Context

The subset actually sent to a particular LLM invocation.

Never assume:

`Logical Context == Model Context`

The optimizer must be able to dynamically:

- retrieve
- prune
- compress
- summarize
- defer
- page
- decompose
- reconstruct
- expand
- reduce
- route to another eligible model

while protecting mandatory information.

---

## STEP 5 — EXPLICITLY HANDLE CONTEXT OVERFLOW



All six documents must account for:

Required Context > Effective Model Context Budget

including when the user:

- provides too many files
- provides a huge repository
- provides a long conversation
- attaches data mid-task
- requests "include everything"
- generates large tool results
- generates large agent state
- triggers multi-agent context accumulation

The system must not silently truncate mandatory information.

---

## STEP 6 — CONTEXT OVERFLOW DECISION MODEL

Document the decision process conceptually:

```
Context Required
 ↓
Mandatory Information?
 ↓
Remove Redundancy
 ↓
Remove Irrelevant Information
 ↓
Compress
 ↓
Summarize
 ↓
Retrieve On Demand
 ↓
Partition / Decompose
 ↓
Use Larger Eligible Model
 ↓
Persist External State
 ↓
```

Ask User

↓

Safe Failure

This is a policy-driven decision tree, not a mandatory fixed sequence.

---

## STEP 7 — EFFECTIVE CONTEXT BUDGET

Ensure the documents consistently recognize that the usable context budget may be less than the published model context window.

Conceptually:

Effective Context Budget

=

Model Context Limit

–

System Prompt

–

Policies

–

Tool Definitions

–

Output Reservation

–

Reasoning Reservation

–

Protocol Overhead

–

Safety Margin

Provider/model-specific behavior must be represented through capability metadata.

---

## STEP 8 — DYNAMIC MUTATION REQUIREMENTS

Ensure all relevant documents handle:

### **User mutation**

- changed instructions
- changed scope
- cancellation
- partial execution

### **Context mutation**

- context added
- context removed
- context changed
- context becomes stale

### **File mutation**

- file attached
- file removed
- file modified
- repository changed

### **Workflow mutation**

- steps added
- steps removed
- steps reordered
- workflow superseded

### **Permission mutation**

- permission granted
- permission revoked
- role changed

### **Policy mutation**

- policy changed
- policy becomes stricter
- previously permitted operation becomes prohibited

### **Model mutation**

- model unavailable

- model becomes available
- context capacity changes
- model capability changes

### **Provider mutation**

- provider unavailable
- gateway unavailable
- internal gateway policy changes

### **Tool mutation**

- tool unavailable
- tool permission revoked
- tool schema changes

---

## **STEP 9 — LONG-WAIT AND RESUME**

Ensure all relevant documents support:

- waiting for user
- waiting for approval
- waiting for tools
- waiting for provider
- waiting for external systems
- hours/days between execution stages

Upon resume, revalidate relevant:

- context
- workflow
- authorization
- policies
- model/provider
- external state
- cache
- budget
- task intent

Do not blindly replay the original request.

---

## STEP 10 — INTERRUPTION AND RECOVERY

The architecture and interfaces must support explicit execution states such as:

- CREATED
- RUNNING
- WAITING
- PAUSED
- BLOCKED
- CANCELLED
- INTERRUPTED
- RESUMABLE
- SUPERSEDED
- COMPLETED
- FAILED
- ABORTED

Use the terminology already present in the documents where equivalent states exist; do not create unnecessary duplicate state names.

---

## STEP 11 — CHECKPOINTING

For long-running workflows, define sufficient state to safely resume.

Checkpoint/recovery information should account for:

- completed work
- pending work
- context version
- workflow version
- policy version
- authorization state
- model/provider
- tool operations
- side effects
- cost
- budget

- quality state
- recovery state

Avoid storing huge duplicated prompts when references, deltas, or immutable state are sufficient.

---

## **STEP 12 — DO NOT REPLAY NON-IDEMPOTENT OPERATIONS**

Explicitly distinguish:

- read-only
- idempotent
- conditionally idempotent
- non-idempotent
- irreversible

Protect operations such as:

- commits
- deployments
- database writes
- emails
- ticket creation
- external mutations
- destructive actions

from accidental replay after retry/resume.

---

## **STEP 13 — CODING AGENT REQUIREMENTS**

Coding agents must be treated as first-class consumers.

Support:

- repository exploration
- dependency discovery
- symbol discovery
- targeted context retrieval

- test retrieval
- build-error retrieval
- runtime-log retrieval
- Git state
- context expansion
- context overflow
- user-added files
- user-removed files
- changing repository state
- permission changes
- tool restrictions
- policy restrictions
- stale patches
- conflicting patches
- superseded tasks

A coding agent requesting information not currently in context must NOT automatically be treated as hallucinating.

Determine whether it is:

- legitimate context discovery
- missing requirement
- unauthorized request
- prohibited operation
- prompt injection
- malicious behavior
- irrelevant request

---

## STEP 14 — ENTERPRISE INTERNAL-GATEWAY DEPLOYMENT

Ensure the architecture works when the organization does NOT have direct access to external GenAI providers.

Support:

Agent

↓

Control Plane

↓

Enterprise AI Gateway

↓

Approved Model Provider

and:

Agent

↓

Control Plane

↓

Internal AI Platform

↓

Private Model

The Control Plane must not require direct external provider credentials when the enterprise architecture intentionally abstracts them.

---

## STEP 15 — INACCESSIBLE MODEL

Never select a model merely because it is theoretically optimal.

Distinguish:

- discovered
- recommended
- eligible
- authorized
- accessible
- available
- capability-compatible

Only an eligible, authorized, accessible and available model may be selected.

---

## STEP 16 — SECURITY OVERRIDES OPTIMIZATION

Ensure all documents consistently enforce:



Security / Authorization / Compliance

>

Optimization

Examples:

- unauthorized context must never enter the prompt
  - revoked permission must not be bypassed through cache
  - sensitive information must not leak through summaries
  - stale authorization must not authorize new actions
  - cross-tenant cache reuse is prohibited
  - optimization cannot weaken policy
- 

## STEP 17 — NEGATIVE OPTIMIZATION

All optimization layers must be allowed to return:

`DO_NOT_OPTIMIZE`

when:

- savings are negligible
  - optimization overhead is greater than savings
  - quality risk is high
  - latency risk is high
  - context is already sufficiently compact
  - compression is unsafe
  - routing provides no economic benefit
  - cache confidence is insufficient
- 

## STEP 18 — OPTIMIZATION SUCCESS MUST BE MULTI-DIMENSIONAL

Do not define optimization success as:

`tokens_before > tokens_after`

Instead consider:

- token savings
- total inference cost

- optimization overhead
  - quality
  - task success
  - latency
  - retries
  - tool calls
  - model calls
  - cache effectiveness
  - retrieval overhead
  - failure/recovery cost
- 

## STEP 19 — UPDATE EDGE-CASES.MD LAST

After reconciling:

1. problemStatement.txt
2. Engineering\_Spec.md
3. architecture.md
4. interfaces.md
5. conventions.md

THEN update:

`edge-cases.md`

The updated `edge-cases.md` must become the consolidated operational treatment of:

- positive scenarios
- negative scenarios
- boundary scenarios
- context overflow
- context mutation
- workflow mutation
- permission mutation
- policy mutation
- provider/model failures
- tool failures
- coding-agent edge cases

- interruptions
- recovery
- long waits
- checkpointing
- stale state
- supersession
- partial execution
- compound failures
- optimizer failures
- security failures

Do not make `edge-cases.md` contradictory to the architecture or interfaces.

---

## STEP 20 — CROSS-DOCUMENT TRACEABILITY

Before finishing, perform a consistency check.

For each new major requirement verify:

```
Problem Statement
 ↓
Engineering Specification
 ↓
Architecture
 ↓
Interfaces
 ↓
Conventions
 ↓
Edge Cases
```

If a requirement is not applicable to a particular document, state that explicitly rather than inventing unnecessary content.

---

## STEP 21 — DO NOT CREATE SCENARIO-MATRIX.MD YET

This is extremely important.

Do NOT create:

```
scenario-matrix.md
```

in this pass.

The scenario matrix will be generated only after these six documents have been stabilized.

The later scenario-matrix generation must consume the hardened versions of these documents as its inputs.

---

## STEP 22 — FINAL VALIDATION REPORT

After modification, provide a concise report containing:

### Files Modified

List exactly the six files.

### Files Not Modified

Confirm that all other documents remain untouched.

### Existing Requirements Preserved

Summarize major preserved domains.

### New Requirements Added

Summarize:

- dynamic context
- context overflow
- context expansion
- context reduction
- logical vs model-admitted context
- dynamic workflow
- dynamic permissions
- dynamic policies
- model/provider availability
- enterprise gateway
- coding agents
- interruptions

- checkpointing
- recovery
- supersession
- partial execution
- long waits
- cache invalidation
- security
- optimization failure
- negative optimization

## Cross-Document Consistency

Report whether the six documents agree.

## Contradictions Found

List any contradictions and how they were resolved.

## Unresolved Questions

List only genuinely unresolved questions.

## Scenario-Matrix Readiness

State whether the six documents are now sufficiently stable to serve as the source material for generating `scenario-matrix.md`.

Do NOT generate the scenario matrix in this pass.

## Then your next stages should be

I would now follow this exact sequence:

### Pass 1 — what you're about to do

```
Existing 6 documents
 ↓
Requirements hardening
 ↓
Cross-document reconciliation
 ↓
Updated edge-cases.md
```

### Pass 2

```
Six stabilized documents
↓
scenario-matrix.md
↓
150+ scenarios
↓
Traceability IDs
```

### Pass 3

Then modify the remaining documents based on the **stabilized scenario matrix**, rather than independently guessing edge cases:

```
scenario-matrix.md
↓
optimization-catalog.md
provider-matrix.md
cache-strategy.md
agent-optimization.md
inference-optimization.md
quality-gates.md
security.md
observability.md
eval.md
SCALING.md
implementation-plan.md
ADRs
```

That ordering is significantly safer for an enterprise system because `scenario-matrix.md` becomes the **verification layer between requirements and implementation design**, rather than becoming another document that contains its own interpretation of the requirements.

And I would specifically **not ask Antigravity to generate all 15 documents in one shot**. Your current one-document-at-a-time approach is much better for maintaining consistency and catching contradictions early.



**architecture(1).md**  
File



**conventions(1).md**  
File



**edge-cases(1).md**  
File



**Ent\_Agent\_LLM\_Infere...**  
File



Ent\_Agent\_LLM\_Infere...  
Document



interfaces(1).md  
File

so now, I have all these 6 six docs in-place. Now I have all these questions:

At this stage I would like to know the below scenarios are taken care in our control plane:

- 1) What if the organization doesn't have access to enable the outside GenAI products otherthan the internal implementation on top of the actual products?
- 2) What if the context is totally pruned or context is never taken while going throught the pipeline?
- 3) What if the suggested models are not accessible?

Show more

Worked for 7s

Yes. Since you now have the **six documents in place**, and you have **not modified anything after** `conventions.md` and `edge-cases.md`, this is actually the right moment to pause and do a **scenario-completeness pass before generating any more documents**.

I checked the current artifacts available to me. Your existing specification already has substantial coverage: developer-agent optimization, dynamic context-related optimization, context invalidation, code-aware context handling, model routing, cache economics, outcome-based optimization, and progressive compaction are already part of the baseline.

`problemstatement(6)(1)` Your architecture also already treats context lifecycle and agent/workflow optimization as first-class layers. `architecture(1)` And the interfaces already support multiple agent frameworks, including Cursor, Claude Code, Copilot and Antigravity, while defining versioning, cancellation, retry, tenant isolation and policy boundaries. `interfaces`

However, **your 12 questions expose a deeper requirement**: the Control Plane cannot be modeled as a static `request → optimize → LLM → response` pipeline. It needs to behave like a **stateful execution control plane**.

**My recommendation: don't modify the six documents manually**

Use Antigravity to perform another controlled reconciliation pass over exactly these six:

```
problemStatement.txt
Engineering_Spec.md
```

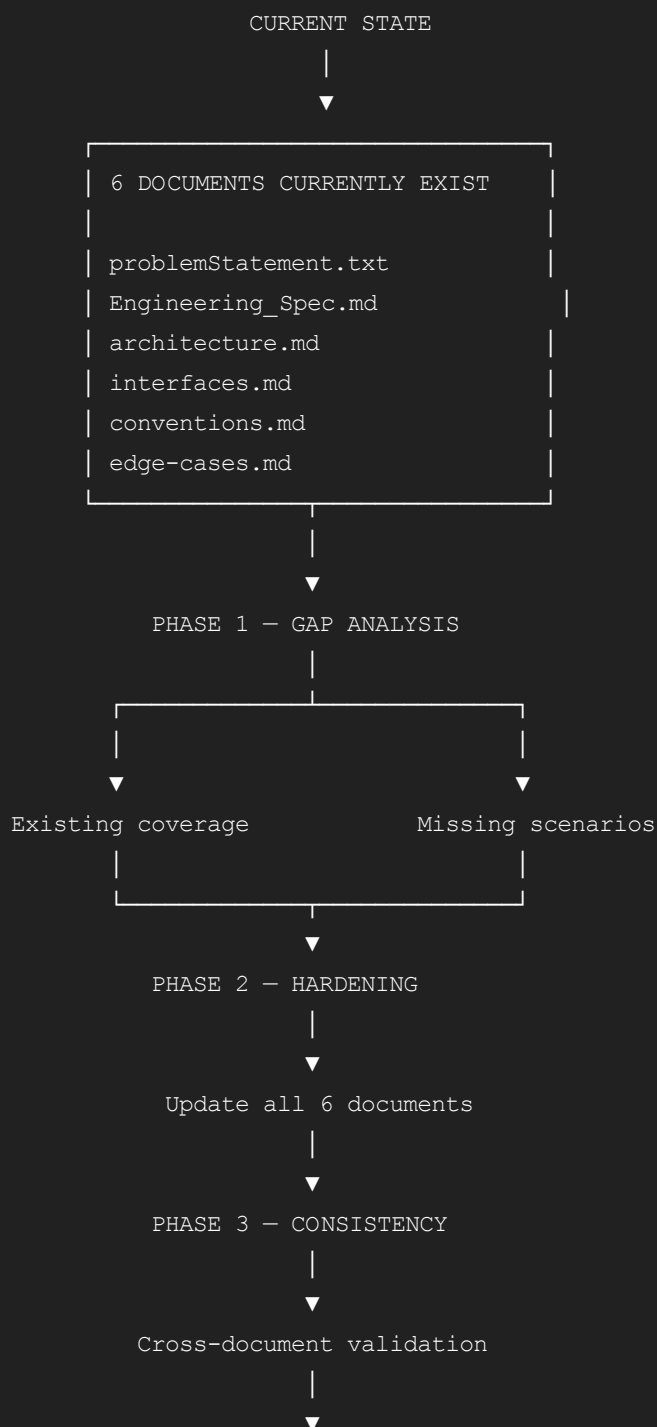
```
architecture.md
interfaces.md
conventions.md
edge-cases.md
```

Then **stop**.

Do not create `scenario-matrix.md` yet.

After that pass, we create the scenario matrix from the stabilized six documents.

## The complete process I recommend from this point





PHASE 4 – SCENARIO MATRIX



150+ real scenarios



PHASE 5 – DOCUMENTATION



optimization-catalog

provider-matrix

cache-strategy

agent-optimization

inference-optimization

quality-gates

security

observability

eval

SCALING

implementation-plan

ADRs

The crucial part is **Phase 1 → Phase 2 → Phase 3**.

## Your 12 questions reveal 6 major missing dimensions

I would explicitly add these to the current hardening pass.

### 1. Deployment independence

Your Control Plane must work with:

External Provider  
Internal AI Gateway  
Private Model  
Hybrid Provider  
Enterprise Proxy  
On-prem inference

The provider itself should therefore be replaceable behind an abstraction.

Your existing interface already establishes provider neutrality.

interfaces

### 2. Context lifecycle

You need to formally model:

```
CONTEXT_CREATED
CONTEXT_ADMITTED
CONTEXT_OPTIMIZED
CONTEXT_PARTIALLY_PRUNED
CONTEXT_COMPRESSED
CONTEXT_EXPANDED
CONTEXT_INVALIDATED
CONTEXT_STALE
CONTEXT_RECONSTRUCTED
CONTEXT_ARCHIVED
CONTEXT_DELETED
```

This is more powerful than simply saying "context pruning."

---

### 3. Execution lifecycle

Likewise:

```
CREATED
RUNNING
WAITING
PAUSED
BLOCKED
INTERRUPTED
CANCELLED
SUPERSEDED
RESUMABLE
COMPLETED
FAILED
ABORTED
```

That is what handles your questions around:

- long waits
  - user changes
  - interruptions
  - permissions
  - partial execution
  - resumption
- 

### 4. Dynamic mutation

This is probably the **most important missing conceptual layer**.

During a task, any of these can change:

```
User
Context
Files
Workflow
Permissions
Policies
Model
Provider
Tools
MCP
Repository
Memory
Cache
External state
Budget
Quality requirements
Latency requirements
```

The Control Plane therefore needs **version-aware execution**.

For example:

```
Task V1
Context V3
Policy V4
Workflow V2
Model A
```

Then the user changes something:

```
Task V2
Context V4
Policy V5
Workflow V3
Model B
```

The system needs to determine which previous decisions remain valid.

---

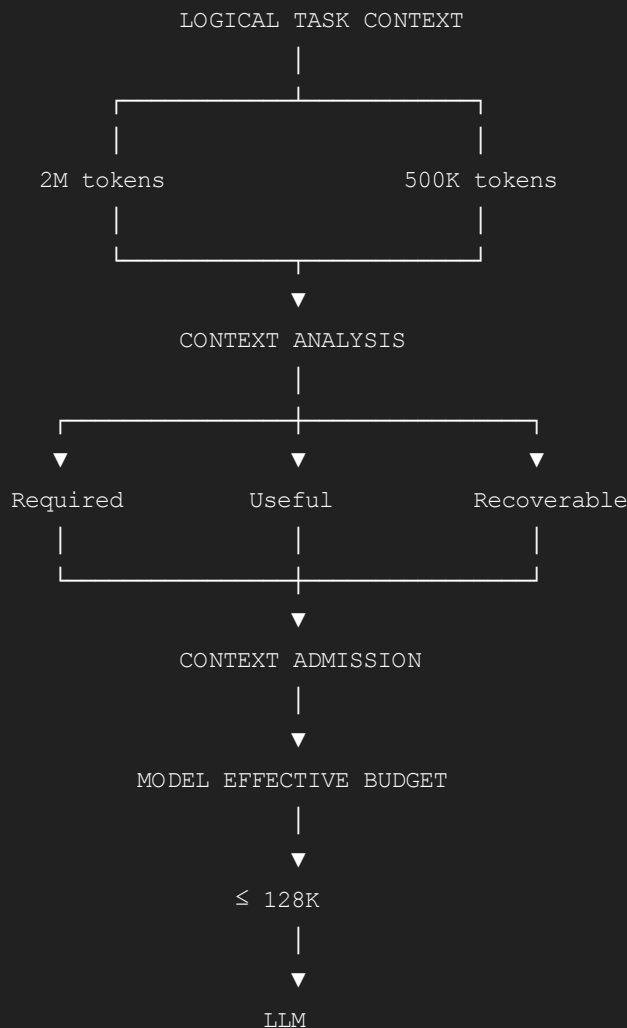
## 5. Context overflow must become a first-class subsystem

This is particularly important for your product.

Your existing specification already recognizes unbounded context growth and soft-reset/context budgeting.

problemstatement

But I would now make the model more explicit:



The optimizer should **never** equate "context optimization" with "delete tokens."

## 6. Agent-owned memory vs Control-Plane state

Your question #11 is important enough to become an explicit architecture decision.

I recommend this principle:

**The agent may own working memory, but the Control Plane owns execution truth.**

For example:

### Agent can own

- scratchpad
- temporary reasoning state
- temporary plan
- local working context

- transient observations

### **Control Plane should own**

- request identity
- execution state
- context versions
- workflow versions
- policy decisions
- authorization decisions
- optimization decisions
- checkpoints
- provenance
- cost
- quality state
- recovery metadata
- audit metadata

### **External systems own**

- repository
- files
- database
- identity
- permissions
- business-system state

That separation will become extremely valuable when you implement recovery.

---

## **What about your #12: out-of-context questions?**

This deserves its own classification.

Suppose the coding agent has:

"Fix the authentication bug."

Then it asks:

"What is the company's HR leave policy?"

That's not necessarily a context overflow problem.

It could be:

IRRELEVANT  
MISSING CONTEXT  
LEGITIMATE DISCOVERY  
UNAUTHORIZED REQUEST  
POLICY VIOLATION  
PROMPT INJECTION  
MALICIOUS REQUEST  
USER INTENT CHANGE

The Control Plane should classify the request before retrieving or exposing additional information.

That's especially important for enterprise environments.

# One more thing I would add

You said:

"and so on..."

I would take that literally.

Don't limit the scenario exercise to the 12 examples.

Have Antigravity systematically generate scenarios across these dimensions:

| Dimension  | Examples                              |
|------------|---------------------------------------|
| User       | new request, correction, cancellation |
| Context    | add/remove/change/overflow            |
| Files      | attach/delete/modify                  |
| Workflow   | add/remove/reorder                    |
| Permission | grant/revoke/expire                   |
| Policy     | change/strengthen                     |
| Model      | unavailable/capability change         |
| Provider   | unavailable/gateway failure           |
| Tool       | unavailable/changed                   |

| Dimension      | Examples                   |
|----------------|----------------------------|
| MCP            | tool discovery/change      |
| Agent          | loop/retry/stop            |
| Sub-agent      | delegate/reject/return     |
| Memory         | stale/corrupt/missing      |
| Cache          | hit/miss/stale/poisoned    |
| Repository     | branch/change/conflict     |
| Execution      | pause/resume/interruption  |
| Budget         | exhausted/changed          |
| Quality        | degraded/failed            |
| Security       | unauthorized/injection     |
| Recovery       | checkpoint/rollback/replay |
| Concurrency    | simultaneous changes       |
| External state | changed while waiting      |

That gives you a **real control-plane test universe**, rather than a collection of ordinary edge cases.

## The next Antigravity prompt

Since you already have the six files, I would **not reuse the earlier prompt verbatim**.

Use the following as the next master instruction. It is specifically designed for your current state.

 Edit



# ENTERPRISE AGENT & LLM INFERENCE OPTIMIZATION CONTROL PLANE

## SIX-DOCUMENT SCENARIO COMPLETENESS & DYNAMIC EXECUTION HARDENING

You are now performing a comprehensive requirements-gap, scenario-completeness, and cross-document consistency pass over the CURRENT six-document baseline.

The objective is to ensure that the Control Plane is robust against both normal and abnormal real-world enterprise execution conditions before the scenario matrix and remaining engineering documents are generated.

This is a HARDENING PASS.

It is NOT a rewrite.

It is NOT an implementation task.

It is NOT permission to redesign the product arbitrarily.

---

### 1. FILES IN SCOPE

Read and reconcile ONLY these six files:

1. `Ent_Agent_LLM_Inference_Opt_Control_Plane_problemstatement.txt`
2. `Ent_Agent_LLM_Inference_Opt_Control_Plane_Engineering_Spec.md`
3. `architecture.md`
4. `interfaces.md`
5. `conventions.md`
6. `edge-cases.md`

These six files are the current authoritative baseline.

---



## 2. DO NOT MODIFY OTHER FILES

Do NOT create or modify:

- `scenario-matrix.md`
- `optimization-catalog.md`
- `provider-matrix.md`
- `cache-strategy.md`
- `agent-optimization.md`
- `inference-optimization.md`
- `quality-gates.md`
- `security.md`
- `observability.md`
- `eval.md`
- `SCALING.md`
- `implementation-plan.md`
- ADRs
- source code
- configuration
- deployment manifests

Only the six files listed above may be modified.

---

## 3. FIRST READ ALL SIX FILES COMPLETELY

Before changing anything:

- read the entire problem statement
- read the entire engineering specification
- read the entire architecture
- read the entire interfaces specification
- read the entire conventions document
- read the entire edge-cases document

Build an internal understanding of all existing requirements.

DO NOT assume that a section is missing simply because it is not visible in a search result.

---

## 4. PRESERVATION RULE

Preserve every valid existing requirement.

Do not remove:

- optimization techniques
- agent optimization
- coding-agent optimization
- context optimization
- retrieval
- caching
- routing
- inference optimization
- reasoning optimization
- tool optimization
- memory optimization
- quality controls
- security
- governance
- observability
- cost accounting
- fallback
- acceptance criteria
- existing edge cases
- provider abstraction
- enterprise deployment support

If existing content already covers a scenario, strengthen or cross-reference it rather than duplicating it.

---

## 5. NEW CORE ARCHITECTURAL PRINCIPLE

The Control Plane must NOT be modeled as a static:

Request → Optimize → LLM → Response

pipeline.

It is a STATEFUL EXECUTION CONTROL PLANE.

The following may mutate during execution:

- user intent
- request
- context
- files
- workflow
- permissions
- policies
- models
- providers
- tools
- MCP
- memory
- cache
- repository
- external state
- budget
- quality requirements
- latency requirements

The architecture must explicitly account for these mutations.

---

## 6. LOGICAL CONTEXT VS MODEL-ADMITTED CONTEXT

Define and consistently use:

### **Logical Task Context**

Everything potentially relevant to the task.

### **Model-Admitted Context**

The subset actually supplied to a specific model invocation.

Never assume:

Logical Task Context = Model Context Window

The Control Plane must support:

- context discovery
  - context retrieval
  - context ranking
  - context pruning
  - context deduplication
  - context compression
  - context summarization
  - context paging
  - context partitioning
  - context reconstruction
  - context expansion
  - context reduction
  - deferred retrieval
  - externalized state
- 

## 7. CONTEXT OVERFLOW

Explicitly handle:

Required Context > Effective Model Context Budget

Scenarios include:

- huge user input
- huge attachment
- huge repository
- huge conversation
- huge tool output
- huge multi-agent state
- system prompt growth
- policy growth
- tool-definition growth
- reasoning-budget growth
- output reservation
- context growth during execution

Never silently discard mandatory information.

---

## 8. CONTEXT OVERFLOW DECISION ENGINE

The architecture must support policy-driven selection among:

- remove redundancy
- remove irrelevant context
- remove stale context
- compress
- summarize
- retrieve on demand
- partition
- decompose
- use larger eligible model
- externalize state
- pause
- ask user
- safe failure

Do not require one universal ordering.

---

## 9. MANDATORY CONTEXT PROTECTION

Introduce context criticality tiers where appropriate:

- mandatory
- high-value
- conditional
- recoverable
- redundant
- stale/invalid

Optimization must never silently delete mandatory information.

---

## 10. EFFECTIVE CONTEXT BUDGET

Do not treat published context-window size as automatically usable.

Account conceptually for:

- system instructions
- developer instructions
- policies
- tool definitions
- protocol overhead
- output reservation
- reasoning reservation
- safety margin

Provider/model differences must be represented through capability metadata.

---

## 11. CONTEXT COMPLETELY PRUNED

Handle:

- all useful context removed
- mandatory context removed
- instruction removed
- dependency removed
- entity removed
- provenance removed
- task state removed

The system must detect and recover rather than blindly execute.

---

## 12. NO CONTEXT PROVIDED

Distinguish:

**Legitimate no-context task**

from:

**Task requiring unavailable context**

Never fabricate missing information.

---

## 13. OUT-OF-CONTEXT USER REQUESTS

Classify out-of-context requests into appropriate categories such as:

- legitimate context expansion
- unrelated question
- new user intent
- missing requirement
- unauthorized information request
- prohibited request
- prompt injection
- malicious request
- irrelevant request

Do not automatically retrieve sensitive information merely because an agent asks for it.

---

## 14. CODING AGENT CONTEXT DISCOVERY

Coding agents must be able to dynamically request:

- files
- symbols
- dependencies
- tests
- build output
- runtime logs
- configuration
- API contracts
- Git state
- repository metadata
- documentation

Context discovery must pass:

- relevance
- authorization
- security
- policy
- freshness
- budget
- quality

checks.

---

## 15. CODING AGENT OUT-OF-CONTEXT GENERATION

Do not automatically classify generation outside the currently admitted context as hallucination.

Determine whether it is:

- legitimate inference
  - missing-context discovery
  - repository exploration
  - dependency discovery
  - unsupported assumption
  - hallucination
  - policy violation
  - unauthorized access
  - prompt injection
- 

## 16. ENTERPRISE PROVIDER ACCESS

Support organizations where:

- external provider access is unavailable
- external credentials are prohibited
- providers are accessible only through an enterprise gateway
- models are private
- models are internally hosted
- provider information is abstracted

Examples:

Agent → Control Plane → Enterprise AI Gateway → Provider

Agent → Control Plane → Internal AI Platform → Private Model

The Control Plane must remain provider-neutral.

---



## 17. INACCESSIBLE MODEL

Distinguish:

- discovered model
- recommended model
- eligible model
- authorized model
- accessible model
- available model
- capability-compatible model

Never select an inaccessible model.

---

## 18. DYNAMIC PERMISSIONS

Handle:

- permission granted during execution
- permission revoked during execution
- permission expired
- role changed
- repository permission changed
- tool permission changed

Authorization must be revalidated for sensitive actions where required.

---

## 19. DYNAMIC POLICIES

If policy changes while execution is active:

- identify affected steps
- invalidate affected decisions
- block newly prohibited operations
- preserve unaffected work
- re-evaluate context
- re-evaluate tools
- re-evaluate model/provider
- record audit information

## 20. PARTIAL EXECUTION

Support requests such as:

"Only perform steps 1 and 2."

"Do not modify files."

"Plan only."

"Do not run tests."

"Stop before commit."

The system must enforce workflow scope without unnecessarily restarting unaffected work.

---

## 21. FILES ATTACHED MID-EXECUTION

Handle:

- new attachment
- replacement
- deletion
- modification
- inaccessible attachment

Determine which context, cache, decisions and quality gates are affected.

Invalidate selectively.

---

## 22. USER CHANGES CONTEXT DURING EXECUTION

Handle:

- additions
- deletions
- corrections

- changed constraints
- changed requirements
- changed priority

Do not allow stale context to silently remain authoritative.

---

## **23. USER CHANGES WORKFLOW DURING EXECUTION**

Handle:

- steps added
- steps removed
- steps reordered
- workflow scope reduced
- workflow scope expanded
- workflow superseded
- execution cancelled

Use workflow versioning where appropriate.

---

## **24. USER CHANGES CONTEXT WINDOW DURING EXECUTION**

If available context capacity changes:

- recompute effective context budget
- invalidate affected admission decisions
- preserve mandatory information
- avoid unnecessary reprocessing
- use incremental recomputation

If capacity increases, do not automatically increase prompt size.

---

## **25. LONG WAIT**

Handle execution waiting for:

- user

- approval
- tool
- provider
- external system
- scheduled event

After an unexpectedly long wait, revalidate relevant state before resuming.

---

## 26. EXECUTION INTERRUPTION

Handle:

- network failure
- provider failure
- timeout
- tool failure
- process crash
- Control Plane failure
- infrastructure failure
- user cancellation
- deployment
- budget exhaustion

Define safe recovery semantics.

---

## 27. CHECKPOINTING

For long-running or high-value workflows support checkpoints containing sufficient state to recover:

- completed work
- pending work
- context version
- workflow version
- policy version
- authorization state
- model/provider
- tool operations

- side effects
- cost
- budget
- quality
- recovery state

Do not duplicate enormous context unnecessarily.

---

## 28. RESUME ≠ REPLAY

Resume must reconcile:

- checkpoint
- current user intent
- current context
- current workflow
- current permissions
- current policies
- current external state
- completed work
- remaining work

Then choose appropriately among:

- resume
  - recompute
  - retrieve
  - expand context
  - rollback
  - restart
  - abort
  - ask user
- 

## 29. NON-IDEMPOTENT OPERATIONS

Explicitly protect operations such as:

- commits
- deployments
- database writes

- emails
- ticket creation
- destructive actions
- external mutations

from accidental replay.

---

## 30. SUPERSESION

If a newer user request changes the task:

Old execution → SUPERSEDED

unless policy explicitly allows concurrent execution.

A superseded execution must not continue making unwanted side effects.

---

## 31. AGENT MEMORY VS CONTROL-PLANE STATE

Define clear ownership.

Agent may own:

- scratch state
- transient reasoning
- temporary plans
- temporary observations

Control Plane should own authoritative execution state such as:

- request
- execution state
- context versions
- workflow versions
- policy decisions
- authorization decisions
- optimization decisions
- checkpoints
- provenance

- cost
- quality
- recovery metadata
- audit metadata

External systems remain authoritative for external state.

---

## 32. CONTEXT RETENTION

Do not retain everything automatically.

Retention must be governed by:

- security
- compliance
- audit
- recovery
- reproducibility
- evaluation
- debugging
- cost attribution
- optimization learning

Support where required:

- TTL
  - deletion
  - encryption
  - minimization
  - redaction
  - tenant policies
  - legal/compliance requirements
- 

## 33. CONTEXT PROVENANCE

Track where practical:

- source
- source version
- timestamp

- authorization scope
  - sensitivity
  - transformation history
  - compression
  - retrieval
  - optimization
  - validity
- 

## 34. CACHE INVALIDATION

Cache reuse must consider:

- tenant
- authorization
- policy
- context version
- source freshness
- dependency versions
- workflow
- task intent

Never reuse a stale or unauthorized cached result.

---

## 35. NEGATIVE OPTIMIZATION

Every optimization must be able to return:

`DO_NOT_OPTIMIZE`

when optimization overhead or risk exceeds expected value.

Include:

- compression overhead
- retrieval overhead
- retry overhead
- routing overhead
- cache overhead
- evaluation overhead
- quality degradation



- latency degradation
- 

## 36. OPTIMIZER FAILURE

The optimizer itself may fail.

Examples:

- tokenizer unavailable
- classifier unavailable
- ranker unavailable
- compressor unavailable
- cache unavailable
- policy service unavailable
- model profile unavailable
- evaluator unavailable

Optimization failure must not unnecessarily become task failure.

Security-critical failures may fail closed.

---

## 37. COMPOUND FAILURES

Explicitly handle combinations such as:

- context overflow + larger model unavailable
  - permission revoked + pending tool call
  - workflow changed + non-idempotent action pending
  - new file attached + smaller context window
  - provider unavailable + budget nearly exhausted
  - long wait + policy changed + permission changed
  - context changed + stale cache
  - model changed + previous optimization invalid
  - agent resumed + repository changed
- 

## 38. POSITIVE SCENARIOS

Do not document only failures.

Include successful behavior for:

- dynamic context expansion
  - safe pruning
  - safe compression
  - context recovery
  - model fallback
  - provider fallback
  - internal gateway execution
  - permission grant
  - permission revocation
  - workflow reduction
  - workflow expansion
  - mid-task file attachment
  - successful resume
  - successful checkpoint recovery
  - coding-agent context discovery
  - cache reuse
  - safe cache invalidation
  - DO\_NOT\_OPTIMIZE
  - optimizer failure with successful task continuation
- 

## 39. SCENARIO COVERAGE DIMENSIONS

Ensure the hardened six documents collectively cover scenarios across:

1. User
2. Request
3. Context
4. Files
5. Workflow
6. Permissions
7. Policies
8. Models
9. Providers
10. Enterprise gateways
11. Tools
12. MCP
13. Agents

14. Sub-agents
  15. Coding agents
  16. Memory
  17. Cache
  18. Repository
  19. External systems
  20. Budget
  21. Quality
  22. Security
  23. Execution
  24. Recovery
  25. Concurrency
- 

## 40. CROSS-DOCUMENT TRACEABILITY

For every newly introduced major behavior verify:

Problem Statement

↓

Engineering Specification

↓

Architecture

↓

Interfaces

↓

Conventions

↓

Edge Cases

Do not allow lower-level documents to contradict higher-level requirements.

---

## 41. INTERFACE HARDENING

Where new dynamic behavior requires an interface change, update

`interfaces.md`.

Consider fields for:

- context version
- workflow version
- policy version
- authorization version/state
- execution state
- checkpoint
- supersession
- mutation
- effective context budget
- model capability
- accessibility
- provenance
- idempotency
- side-effect classification
- recovery state

Do not add fields merely for theoretical completeness; add them where they are required for deterministic control-plane behavior.

---

## 42. CONVENTION HARDENING

Update `conventions.md` so implementation teams have consistent rules for:

- versioning
- mutation handling
- invalidation
- stale state
- retries
- idempotency
- checkpointing
- resume
- cancellation
- supersession
- authorization
- policy changes
- context retention
- provenance
- logging
- sensitive data

- optimization failure
- 

## 43. EDGE-CASE HARDENING

Update `edge-cases.md` into the authoritative exceptional-behavior reference.

Include:

- positive
- negative
- boundary
- dynamic mutation
- compound failure
- context overflow
- context underflow/no context
- coding-agent discovery
- security
- provider/model
- permission
- workflow
- recovery
- interruption
- long wait
- cache
- memory
- concurrency

Do not remove useful existing cases.

---

## 44. SCENARIO MATRIX IS NOT TO BE GENERATED YET

Do NOT create `scenario-matrix.md`.

That will be generated ONLY after this six-document baseline has been stabilized.

---

## 45. FINAL COMPLETENESS AUDIT

Before finishing, verify:

- no valid existing requirement was removed
  - context overflow is explicitly handled
  - context can expand dynamically
  - context can contract safely
  - logical context is separated from model-admitted context
  - no-context tasks are distinguished from missing-context tasks
  - out-of-context requests are classified
  - coding-agent discovery is supported
  - unauthorized context cannot be retrieved
  - inaccessible models cannot be selected
  - internal enterprise gateways are supported
  - dynamic permissions are handled
  - dynamic policies are handled
  - dynamic workflows are handled
  - mid-task files are handled
  - context-window changes are handled
  - long waits are handled
  - interruptions are handled
  - checkpointing is handled
  - resume is not blind replay
  - non-idempotent actions are protected
  - superseded tasks are controlled
  - agent memory vs Control Plane state is defined
  - retention rules exist
  - provenance exists
  - cache invalidation exists
  - negative optimization exists
  - optimizer failure exists
  - compound failures exist
  - positive scenarios exist
  - security overrides optimization
  - quality overrides token savings
  - cost includes optimization overhead
  - all six documents remain consistent
-

## 46. FINAL REPORT

Report:

### **Files Modified**

Exactly the six files.

### **Requirements Preserved**

Major existing domains retained.

### **New Requirements Added**

Major newly hardened capabilities.

### **Existing Coverage**

Which of the user's identified scenarios were already covered.

### **New Coverage**

Which scenarios required additions.

### **Cross-Document Changes**

What changed in each of the six documents.

### **Contradictions Found**

List and resolve them.

### **Remaining Gaps**

List genuine unresolved issues.

### **Scenario-Matrix Readiness**

State:

READY

or:

NOT READY — explain why.

Do not create the scenario matrix yet.

## After this pass

Once Antigravity finishes this, **don't immediately ask it to create the next random document.**

The next logical artifact is:

```
scenario-matrix.md
```

And I would make that a **very serious validation artifact**, not merely a table of edge cases.

It should become the bridge:

```
Requirements
 ↓
Architecture
 ↓
Interfaces
 ↓
Conventions
 ↓
Edge Cases
 ↓
★ SCENARIO MATRIX ★
 ↓
Tests / Evaluations
 ↓
Implementation
```

Only after that would I start updating:

```
optimization-catalog.md
provider-matrix.md
cache-strategy.md
agent-optimization.md
inference-optimization.md
quality-gates.md
security.md
observability.md
eval.md
SCALING.md
implementation-plan.md
ADRs
```



That way, when you eventually implement this in Antigravity, you're not simply building a **token optimizer**. You're building a **state-aware enterprise control plane that can safely optimize an LLM/agent execution even when context, models, permissions, workflows, tools, providers, and user intent change while the task is running**.

That is the level I would target for the product you're describing.

Sources

Today 9:21 AM



**architecture(2).md**  
File



**conventions(2).md**  
File



**edge-cases(2).md**  
File



**Ent\_Agent\_LLM\_Infere...**  
File



**Ent\_Agent\_LLM\_Infere...**  
Document



**interfaces(2).md**  
File

I have uploaded the modified files here. Please verify so that I will start next logical document scenario-matrix.md file

Worked for 7s

Yes. I've reviewed the modified baseline, and **I would proceed to** `scenario-matrix.md` — **but with one important qualification**: the documents are now sufficiently hardened to generate the scenario matrix, but the matrix itself should be treated as the **next validation gate**, not merely another documentation file.

The current architecture is especially strong: the hardening pass added an **Execution State Manager, Context Version Manager, Workflow Version Manager, Checkpoint Manager, Reconciliation Engine, Context Integrity Gate, Context Expansion Controller, Permission Revalidation, Dynamic Policy Evaluation, Capability/Availability Resolver, Stale Result Protection, Supersession Manager, and Recovery Coordinator**.

architecture(2)

Your interfaces also have the right foundations: versioned contracts, idempotency, cancellation, retries, tenant isolation, policy versioning, security boundaries, and provider/framework neutrality.

interfaces

# My verdict

 **READY for** `scenario-matrix.md`

I would **not modify the six documents again before generating the matrix**, unless Antigravity's final validation report identifies an actual contradiction.

The architecture itself claims that all existing objectives, acceptance criteria, security requirements, modules, NFRs, optimization domains and anti-patterns are covered, with no requirements silently removed. `architecture(2)`

More importantly, the new dynamic architecture explicitly establishes the distinction we needed:

**Logical Task Context is authoritative and persistent, while ModelAdmittedContext is assembled for each inference call.**

It also requires versioning of context mutations and protection against destructive eviction. `architecture(2)`

That directly addresses one of your most important concerns: **"What if context gets completely pruned?"**

## How your original questions map now

| # | Scenario                                               | Status                                                                                        |
|---|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1 | Enterprise has no direct external GenAI access         |  Covered |
| 2 | Context completely pruned                              |  Covered |
| 3 | Recommended model inaccessible                         |  Covered |
| 4 | Coding agent needs information outside current context |  Covered |
| 5 | Coding agent requests policy-prohibited resource       |  Covered |
| 6 | Partial task / files added / permissions changed       |  Covered |

| #                        | Scenario                                  | Status    |
|--------------------------|-------------------------------------------|-----------|
| 7                        | User waits unexpectedly long              | ● Covered |
| 8                        | Context window changes during processing  | ● Covered |
| 9                        | Context/workflow changes during execution | ● Covered |
| 10                       | Execution interrupted                     | ● Covered |
| 11                       | Agent memory vs Control Plane state       | ● Covered |
| 12                       | User asks out-of-context question         | ● Covered |
| Context overflow         | Beyond effective model window             | ● Covered |
| Context expansion        | Need more information mid-task            | ● Covered |
| Context invalidation     | Previously valid context becomes stale    | ● Covered |
| Permission revocation    | Permission disappears mid-task            | ● Covered |
| Policy change            | Policy changes during execution           | ● Covered |
| Provider failure         | Provider becomes unavailable              | ● Covered |
| Model capability change  | Model no longer suitable                  | ● Covered |
| Tool/MCP failure         | Tool unavailable/changed                  | ● Covered |
| Long-running task        | Resume after long delay                   | ● Covered |
| Crash/recovery           | Control Plane interruption                | ● Covered |
| Retry                    | Safe vs unsafe retry                      | ● Covered |
| Non-idempotent operation | Avoid accidental replay                   | ● Covered |
| Superseded task          | New request replaces old one              | ● Covered |
| Stale cache              | Cache no longer valid                     | ● Covered |
| Compound failures        | Multiple failures simultaneously          | ● Covered |

That's the right direction for an **enterprise control plane**, rather than simply an optimization middleware.

## One particularly important architectural decision is now present

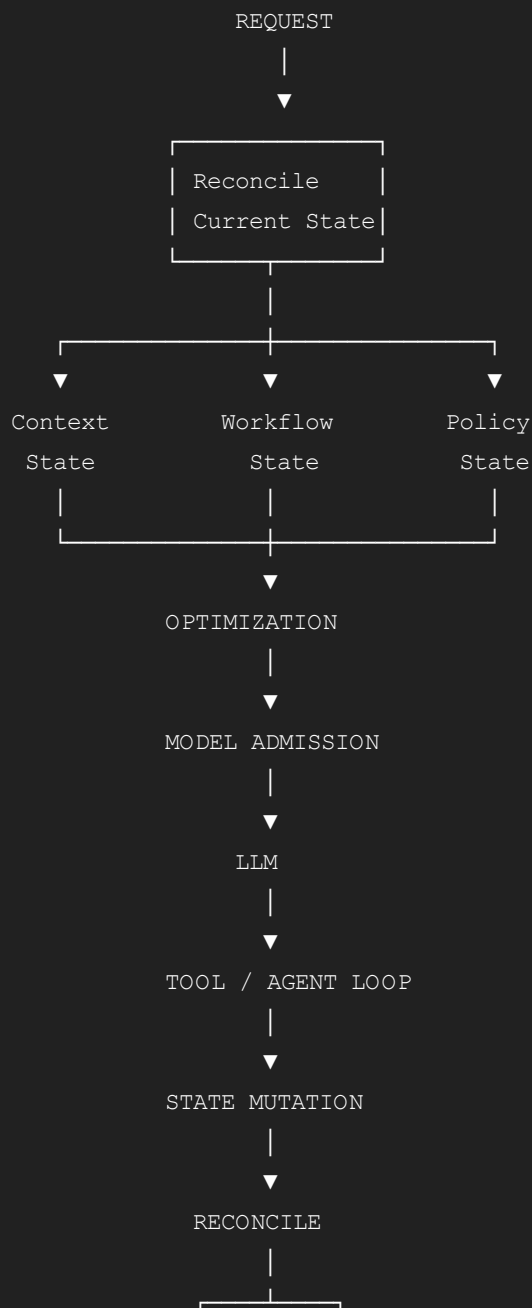
Your architecture says:

**No optimization decision may proceed without calling `reconcile()`.**

`architecture(2)`

That's excellent.

It means the future pipeline conceptually becomes:



That is much more robust than simply putting a tokenizer/pruner in front of the LLM.

What `scenario-matrix.md` should accomplish

Now I would make the scenario matrix **very comprehensive**.

Don't let Antigravity generate a document that merely says:

| Scenario | Expected Result |
|----------|-----------------|
|----------|-----------------|

Instead, make every scenario executable against the architecture.

I recommend each scenario have at least:

```

Scenario ID
Category
Subcategory
Initial State
Trigger
User Intent
Context State
Context Version
Workflow State
Workflow Version
Permission State
Policy State
Model State
Provider State
Tool State
Cache State
Expected Control Plane Decision
Expected Optimization Behavior
Expected Security Behavior
Expected Quality Behavior
Expected Cost Behavior
Expected State Transition
Expected Recovery
Expected Side Effects
Idempotency Requirement
Audit Requirement
Observability Requirement
Failure Classification
Acceptance Criteria
Traceability

```

# And don't make it only "negative scenarios"

This is extremely important.

You asked for **positive + negative + unexpected + boundary** scenarios.

I would organize the matrix into:

## A. Happy-path scenarios

Normal successful optimization.

## B. Context scenarios

- no context
- minimal context
- large context
- overflow
- complete pruning
- context expansion
- context contraction
- stale context
- conflicting context
- context mutation

## C. User mutation

- correction
- cancellation
- scope reduction
- scope expansion
- new requirement

## D. Agent scenarios

- agent loop
- early completion
- repeated request
- hallucinated assumption
- missing information
- legitimate discovery

## E. Coding-agent scenarios

This should be a **large section**.

For example:

Repository exploration  
Symbol lookup  
Dependency lookup  
Large repository  
Huge git diff  
Large terminal output  
Repeated compiler errors  
Test failure  
Runtime failure  
MCP discovery  
Tool restriction  
Unauthorized file  
Permission revocation  
Branch changed  
File modified externally  
Patch becomes stale  
Concurrent developer modification

## F. Security scenarios

Especially:

Unauthorized context  
Unauthorized tool  
Revoked permission  
Cross-tenant cache  
Prompt injection  
Malicious retrieved content  
Sensitive data in summary  
Sensitive data in cache  
Policy changes mid-execution

## G. Model/provider scenarios

Recommended model unavailable  
Model inaccessible  
Model context too small  
Provider unavailable  
Gateway unavailable  
Model capability mismatch  
Fallback model available  
No suitable model available

## H. Execution lifecycle

WAIT  
PAUSE  
RESUME

```
INTERRUPT
CANCEL
TIMEOUT
CRASH
RETRY
RECOVERY
SUPERSESSION
```

## I. Cache/memory

```
cache hit
cache miss
stale cache
invalid cache
cache poisoning
permission-sensitive cache
context-version mismatch
memory missing
memory stale
memory conflict
```

## J. Compound scenarios

This is where the matrix becomes **really valuable**.

For example:

User attaches a new restricted file while the agent is waiting for a provider; meanwhile the model's context budget changes and the user's permission to access another repository is revoked.

That's much closer to the conditions a real enterprise control plane will encounter.

---

## One more recommendation before you generate it

Tell Antigravity explicitly:

**Do not treat the scenario matrix as an independent design.**

It should be derived from:

```
problemStatement.txt
+
Engineering_Spec.md
+
architecture.md
```



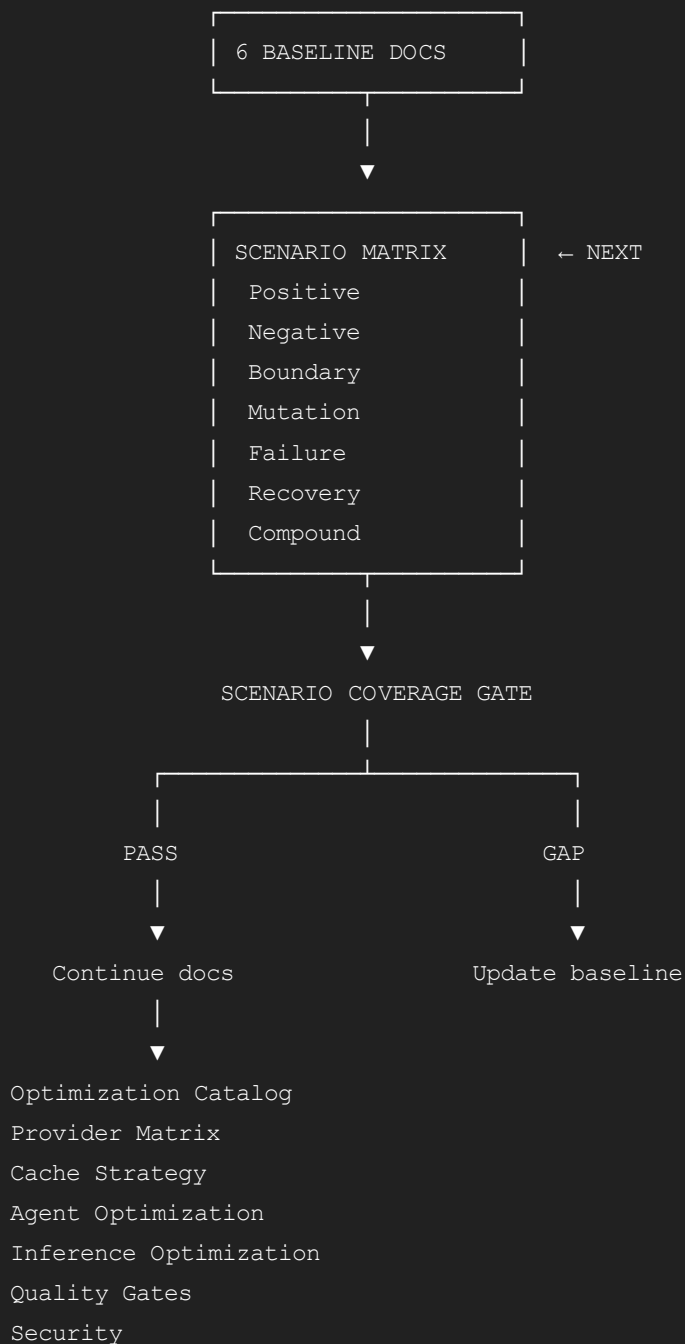
```
+
interfaces.md
+
conventions.md
+
edge-cases.md
```

And if the matrix discovers a requirement that is genuinely absent from those six documents, it should **flag it as a gap instead of silently inventing a new requirement**.

That distinction is extremely important.

## Your documentation workflow from here

I would now lock the process as:



Observability  
Evaluation  
Scaling  
Implementation Plan  
ADRs

**So, yes: go ahead with `scenario-matrix.md` now.**

I would consider your current six-document baseline **stable enough to enter the scenario-validation phase**. The scenario matrix should now become your **formal completeness gate** before you allow Antigravity to generate the implementation-oriented documents.

If you want, I can give you the **single master Antigravity prompt for generating an enterprise-grade `scenario-matrix.md` (including a target of 200+ scenarios, cross-product combinations, coding-agent cases, context-overflow cases, security cases, recovery cases, and traceability back to all six documents)**.

Sources

yes please give me the **single master Antigravity prompt for generating an enterprise-grade `scenario-matrix.md` (including a target of 200+ scenarios, cross-product combinations, coding-agent cases, context-overflow cases, security cases, recovery cases, and traceability back to all six documents)**.

Absolutely. At this point, I would make `scenario-matrix.md` a **formal system-validation artifact**, not just a list of edge cases.

The prompt below is designed to make Antigravity derive the matrix from your **six stabilized documents**, target **200+ scenarios**, include **cross-dimensional/compound scenarios**, and maintain traceability without silently inventing requirements.

 Edit



**MASTER PROMPT**

# ENTERPRISE AGENT & LLM INFERENCE OPTIMIZATION CONTROL PLANE

## SCENARIO MATRIX GENERATION

You are generating the next authoritative engineering artifact for the:

**Enterprise Agent & LLM Inference Optimization Control Plane**

The required output is:

```
scenario-matrix.md
```

This document must become the system's comprehensive scenario-validation and traceability matrix.

It must be derived from the CURRENT SIX-DOCUMENT BASELINE and must NOT silently introduce requirements that are unsupported by that baseline.

---

## 1. AUTHORITATIVE SOURCE DOCUMENTS

Before generating anything, read the following six documents completely:

1. `Ent_Agent_LLM_Inference_Opt_Control_Plane_problemstatement.txt`
2. `Ent_Agent_LLM_Inference_Opt_Control_Plane_Engineering_Spec.md`
3. `architecture.md`
4. `interfaces.md`
5. `conventions.md`
6. `edge-cases.md`

These six documents are the authoritative source material for this scenario matrix.

Do NOT rely on snippets.

Do NOT assume missing content.

Do NOT silently replace source terminology with generic terminology.

Preserve the terminology and architecture already established in these documents.

---

## 2. OUTPUT SCOPE

Create ONLY:

```
scenario-matrix.md
```

Do not modify any of the six source documents.

Do not create:

- implementation-plan.md
- eval.md
- security.md
- observability.md
- SCALING.md
- optimization-catalog.md
- provider-matrix.md
- cache-strategy.md
- agent-optimization.md
- inference-optimization.md
- quality-gates.md
- ADRs
- source code

Those documents will be generated later.

---

## 3. PRIMARY OBJECTIVE

The scenario matrix must answer:

"Under what conditions can this Control Plane operate correctly, safely, economically, and predictably across normal, abnormal, dynamic, adversarial, and compound execution scenarios?"

The matrix must validate:

- functional correctness
  - optimization correctness
  - context correctness
  - security
  - authorization
  - policy enforcement
  - quality
  - cost
  - latency
  - reliability
  - recovery
  - observability
  - auditability
  - multi-tenancy
  - provider independence
  - agent compatibility
  - coding-agent compatibility
  - state consistency
  - cache correctness
  - memory correctness
  - workflow correctness
  - dynamic mutation handling
- 

## 4. MINIMUM SCENARIO TARGET

Generate a minimum of:

**200 distinct scenarios**

Preferably generate substantially more if required for complete coverage.

Do NOT artificially create trivial variations merely to reach 200.

Every scenario must test a meaningful behavior, boundary, failure mode, recovery path, or interaction.

Target approximately:

- 30–40% normal/positive scenarios

- 30–40% negative/failure scenarios
- 15–20% boundary/extreme scenarios
- 10–20% compound/cross-dimensional scenarios

These percentages are guidance, not rigid constraints.

---

## 5. SCENARIO QUALITY REQUIREMENT

Every scenario must have a meaningful:

- trigger
- state
- expected decision
- expected outcome
- validation condition

Avoid vague scenarios such as:

"Provider fails."

Instead:

"Provider becomes unavailable after model selection but before inference dispatch."

Then define exactly what the Control Plane must do.

---

## 6. SCENARIO ID SYSTEM

Create deterministic scenario IDs.

Recommended format:

SCN-<DOMAIN>--<NUMBER>

Examples:

- SCN-CTX-001
- SCN-AGENT-021
- SCN-CODE-015
- SCN-SEC-034
- SCN-REC-012

- SCN-CMP-008

Compound scenarios may use:

SCN-CMP-001

Do not reuse IDs.

IDs must remain stable if the document is regenerated unless the underlying scenario is materially changed.

---

## 7. REQUIRED SCENARIO DOMAINS

The matrix must cover all of the following domains.

### DOMAIN A — REQUEST / USER

Cover:

- normal request
  - empty request
  - malformed request
  - ambiguous request
  - changed request
  - corrected request
  - cancellation
  - partial request
  - expanded request
  - conflicting request
  - duplicate request
  - repeated request
  - superseding request
  - out-of-context request
  - user intent change
- 

### 8. DOMAIN B — CONTEXT

Cover:

- no context
- minimal context

- normal context
  - large context
  - oversized context
  - context overflow
  - context underflow
  - completely pruned context
  - mandatory context accidentally removed
  - stale context
  - contradictory context
  - duplicate context
  - irrelevant context
  - sensitive context
  - context expansion
  - context reduction
  - context replacement
  - context mutation
  - context version mismatch
  - context reconstruction
  - context externalization
  - context retrieval failure
  - context retrieval timeout
  - context compression
  - context summarization
  - compression quality failure
  - retrieval quality failure
- 

## 9. DOMAIN C — EFFECTIVE CONTEXT BUDGET

Test:

- model context limit
- effective context budget
- system prompt overhead
- developer instructions
- policy overhead
- tool-definition overhead
- protocol overhead
- output reservation



- reasoning reservation
  - safety margin
  - context budget changes during execution
  - budget too small for mandatory context
  - larger model available
  - larger model unavailable
  - no model satisfies context requirement
- 

## 10. DOMAIN D — CONTEXT ADMISSION

Test:

- mandatory context admitted
  - irrelevant context rejected
  - sensitive context rejected
  - stale context rejected
  - unauthorized context rejected
  - duplicate context eliminated
  - context priority conflicts
  - context admission failure
  - admission recomputation
  - incremental admission after mutation
- 

## 11. DOMAIN E — AGENTS

Cover:

- single agent
- autonomous agent
- multi-agent system
- sub-agent
- agent delegation
- agent loop
- agent retry
- agent termination
- agent asks for missing context
- agent asks for additional context
- agent requests prohibited information
- agent requests unavailable tool

- agent receives stale context
  - agent receives superseded workflow
  - agent resumes from checkpoint
  - agent state conflicts with Control Plane state
- 

## 12. DOMAIN F — CODING / DEVELOPER AGENTS

This is a FIRST-CLASS domain.

Cover coding agents integrated with environments such as:

- Antigravity
- Cursor
- GitHub Copilot
- Claude Code
- other compatible developer-agent environments

Do not assume any particular vendor API unless supported by the source documents.

Test:

- repository exploration
- file discovery
- symbol discovery
- dependency discovery
- test discovery
- build-error discovery
- runtime-log discovery
- Git state
- branch state
- large repository
- huge diff
- generated files
- binary files
- ignored files
- deleted files
- modified files
- file changed externally
- repository changed during execution

- stale patch
  - conflicting patch
  - agent generates code outside current admitted context
  - agent legitimately discovers missing context
  - agent makes unsupported assumption
  - agent requests unauthorized file
  - agent requests prohibited operation
  - agent attempts to bypass policy
  - tool permission revoked
  - repository permission revoked
  - test execution denied
  - commit denied
  - deployment denied
  - MCP tool unavailable
  - MCP tool schema changed
  - build/test output causes context overflow
- 

## 13. DOMAIN G — WORKFLOW

Test:

- normal workflow
  - workflow with optional steps
  - workflow with mandatory steps
  - workflow reduction
  - workflow expansion
  - workflow reordering
  - workflow cancellation
  - workflow mutation
  - workflow version change
  - workflow supersession
  - workflow partially completed
  - workflow blocked
  - workflow resumed
  - workflow restarted
- 

## 14. DOMAIN H — FILE / ARTIFACT MUTATION

Test:

- file added
  - file removed
  - file replaced
  - file modified
  - file becomes inaccessible
  - file permissions changed
  - new file attached during execution
  - file removed while being processed
  - file changes after caching
  - file version mismatch
  - artifact becomes stale
  - large attachment causes overflow
  - sensitive attachment
  - unauthorized attachment
- 

## 15. DOMAIN I — PERMISSIONS

Test:

- permission exists
  - permission missing
  - permission granted during execution
  - permission revoked during execution
  - permission expires
  - role changes
  - repository permission changes
  - tool permission changes
  - model access changes
  - provider access changes
  - permission differs between initial request and resume
  - permission differs between cache creation and cache reuse
- 

## 16. DOMAIN J — POLICY

Test:

- normal policy

- policy denial
  - policy changed during execution
  - stricter policy
  - relaxed policy
  - policy version mismatch
  - policy service unavailable
  - policy evaluation timeout
  - cached decision invalidated
  - previously permitted action becomes prohibited
  - optimization conflicts with policy
  - context optimization attempts to expose restricted information
- 

## **17. DOMAIN K — MODEL SELECTION**

Test:

- optimal model available
  - optimal model unavailable
  - recommended model inaccessible
  - model unauthorized
  - model capability mismatch
  - insufficient context window
  - excessive latency
  - excessive cost
  - model quota exhausted
  - model rate limited
  - model temporarily unavailable
  - model permanently unavailable
  - model becomes unavailable after selection
  - model becomes available during execution
  - model capability metadata stale
  - no eligible model
- 

## **18. DOMAIN L — PROVIDER / ENTERPRISE GATEWAY**

Test:

- direct provider access
- enterprise gateway
- internal AI platform
- private model
- on-premise model
- hybrid architecture
- gateway unavailable
- provider unavailable
- gateway authentication failure
- provider timeout
- provider rate limit
- provider capability mismatch
- provider changes routing
- organization prohibits direct external provider access
- provider credentials unavailable to the Control Plane

The Control Plane must remain provider-neutral.

---

## 19. DOMAIN M — TOOLS / MCP

Test:

- tool available
  - tool unavailable
  - tool timeout
  - tool failure
  - tool schema change
  - tool permission revoked
  - tool returns huge result
  - tool returns malformed result
  - tool result becomes stale
  - MCP discovery failure
  - MCP tool authorization failure
  - tool result causes context overflow
  - tool result conflicts with existing context
- 

## 20. DOMAIN N — MEMORY

Test:

- no memory
  - memory available
  - stale memory
  - corrupted memory
  - unauthorized memory
  - memory conflict
  - memory version mismatch
  - memory retrieval failure
  - memory retention expiry
  - memory deletion
  - agent-owned memory
  - Control-Plane-owned execution state
  - memory differs from authoritative external state
- 

## 21. DOMAIN O — CACHE

Test:

- cache hit
  - cache miss
  - partial hit
  - stale cache
  - invalid cache
  - cache poisoning
  - authorization mismatch
  - policy mismatch
  - tenant mismatch
  - context version mismatch
  - workflow mismatch
  - task-intent mismatch
  - source-version mismatch
  - cache invalidation
  - cache unavailable
  - cache lookup overhead exceeds savings
  - negative optimization decision
- 

## 22. DOMAIN P — OPTIMIZATION

Test every applicable optimization category established in the source documents.

Examples may include:

- prompt optimization
- context pruning
- deduplication
- compression
- summarization
- retrieval
- caching
- model routing
- model selection
- batching
- request shaping
- tool optimization
- agent-loop optimization
- memory optimization
- inference optimization
- reasoning optimization
- output optimization

Do NOT add optimization techniques merely because they are common in the industry if they are not established by the source documents.

Instead mark genuinely unsupported candidates as:

SOURCE-GAP

---

## 23. DOMAIN Q — NEGATIVE OPTIMIZATION

Explicitly test:

DO\_NOT\_OPTIMIZE

when:

- optimization overhead exceeds expected savings
- quality risk is excessive



- latency increases
  - context is already efficient
  - compression is unsafe
  - retrieval costs exceed savings
  - cache provides insufficient benefit
  - model routing provides negligible benefit
  - optimization confidence is low
- 

## 24. DOMAIN R — QUALITY

Test:

- quality preserved
- quality improved
- quality degraded
- semantic drift
- lost requirements
- lost constraints
- lost dependencies
- wrong model selection
- incorrect summarization
- incorrect retrieval
- incorrect context pruning
- optimization causes task failure
- token savings achieved but task quality fails

Quality must take precedence over raw token reduction.

---

## 25. DOMAIN S — COST

Test:

- optimization saves tokens
- optimization saves total cost
- token reduction but higher total cost
- optimization overhead exceeds savings
- retries increase cost
- fallback increases cost
- context reconstruction increases cost

- evaluation increases cost
  - cache miss increases cost
  - model switch changes cost
  - budget exhausted
  - budget changes mid-execution
- 

## 26. DOMAIN T — LATENCY

Test:

- optimization improves latency
  - optimization increases latency
  - provider slow
  - retrieval slow
  - compression slow
  - cache slow
  - policy evaluation slow
  - long user wait
  - long external wait
  - timeout
  - deadline exceeded
  - latency budget changes
- 

## 27. DOMAIN U — EXECUTION STATE

Cover all applicable execution states established in the source documents.

At minimum evaluate:

- created
- running
- waiting
- paused
- blocked
- interrupted
- cancelled
- superseded
- resumable

- completed
- failed
- aborted

Test valid and invalid transitions.

---

## 28. DOMAIN V — LONG WAIT

Test:

- waiting for user
- waiting for approval
- waiting for tool
- waiting for provider
- waiting for external system
- hours-long wait
- days-long wait
- state changes while waiting
- context changes while waiting
- policy changes while waiting
- permission changes while waiting
- model changes while waiting
- workflow changes while waiting

Resume must reconcile current state rather than blindly replaying stale state.

---

## 29. DOMAIN W — INTERRUPTION / RECOVERY

Test:

- network interruption
- Control Plane crash
- agent crash
- provider failure
- tool failure
- timeout
- process restart

- infrastructure restart
  - user cancellation
  - partial completion
  - checkpoint available
  - checkpoint unavailable
  - checkpoint corrupted
  - stale checkpoint
  - resume
  - recompute
  - rollback
  - restart
  - safe abort
- 

## 30. DOMAIN X — IDEMPOTENCY / SIDE EFFECTS

Classify operations as:

- read-only
- idempotent
- conditionally idempotent
- non-idempotent
- irreversible

Test protection against duplicate execution for:

- commits
  - deployments
  - database writes
  - emails
  - tickets
  - external mutations
  - destructive operations
- 

## 31. DOMAIN Y — SECURITY

Test:

- authentication failure

- authorization failure
- privilege escalation
- unauthorized context
- unauthorized file
- unauthorized tool
- unauthorized model
- unauthorized provider
- cross-tenant access
- cache leakage
- memory leakage
- prompt injection
- malicious retrieved content
- malicious tool output
- sensitive information in summaries
- sensitive information in logs
- sensitive information in cache
- revoked permission
- policy bypass attempt

Security must override optimization.

---

## 32. DOMAIN Z — MULTI-TENANCY

Test:

- tenant isolation
  - context isolation
  - cache isolation
  - memory isolation
  - policy isolation
  - model access isolation
  - provider access isolation
  - quota isolation
  - observability isolation
  - cross-tenant cache attempt
  - cross-tenant context retrieval attempt
- 

## 33. DOMAIN AA — CONCURRENCY

Test:

- simultaneous requests
  - simultaneous context mutations
  - simultaneous file mutation
  - permission change during execution
  - policy change during execution
  - concurrent workflow modification
  - concurrent agent/sub-agent operations
  - concurrent cache update
  - concurrent checkpoint
  - concurrent resume
  - race conditions
  - stale-write protection
  - optimistic concurrency conflict
- 

## **34. DOMAIN AB — SUPERSESSION**

Test:

- newer request replaces old request
  - workflow superseded
  - context superseded
  - agent execution superseded
  - checkpoint superseded
  - pending side effect belongs to superseded execution
  - superseded task attempts to continue
  - superseded cache result
  - superseded optimization decision
- 

## **35. DOMAIN AC — OBSERVABILITY / AUDIT**

Test whether the system can explain:

- why a context item was removed
- why a model was selected
- why a model was rejected
- why a provider was rejected

- why an optimization was applied
- why optimization was skipped
- why cache was invalidated
- why execution was paused
- why execution failed
- why authorization was denied
- why policy blocked an operation
- why recovery chose recomputation
- what state version was active

Audit data must not itself create unauthorized information exposure.

---

## 36. DOMAIN AD — COMPOUND SCENARIOS

This is mandatory.

Generate meaningful combinations of multiple independent state changes.

Examples:

### Compound 1

User adds a sensitive file while:

- context is near overflow
- model becomes unavailable
- policy becomes stricter

Expected behavior must be explicitly defined.

### Compound 2

Coding agent is waiting for a tool while:

- repository changes
- permission is revoked
- workflow is modified

### Compound 3

Execution resumes after a long wait while:

- policy version changed
- context changed
- provider became unavailable
- checkpoint is stale

### Compound 4

Agent requests additional context while:

- requested file is unauthorized
- current model context budget is insufficient
- cache contains a stale copy

### Compound 5

A non-idempotent operation is pending while:

- execution is interrupted
- user supersedes the task
- external state changes

Generate many such compound scenarios.

Do not limit compound scenarios to these examples.

---

## 37. CROSS-PRODUCT / CROSS-DIMENSION COVERAGE

Do not merely test isolated features.

Explicitly test intersections such as:

Context × Model

Context × Provider

Context × Policy

Context × Permission

Context × Cache

Context × Memory

Context × Coding Agent

Context × Workflow

Agent × Tool

Agent × Permission



Agent × Policy  
Agent × Memory  
Agent × Context

Coding Agent × Repository  
Coding Agent × MCP  
Coding Agent × Permission  
Coding Agent × Policy  
Coding Agent × Context  
Coding Agent × Recovery

Workflow × Permission  
Workflow × Policy  
Workflow × Context  
Workflow × Recovery

Cache × Permission  
Cache × Policy  
Cache × Context Version  
Cache × Tenant

Model × Provider  
Model × Context  
Model × Cost  
Model × Latency

Recovery × Idempotency  
Recovery × Supersession  
Recovery × Policy  
Recovery × Permission

Also create higher-order combinations where meaningful.

---

## 38. SCENARIO RECORD FORMAT

Every scenario **MUST** use a consistent structure.

Use:

### SCN-XXX-NNN — Scenario Title

\*\*Category:\*\*

\*\*Subcategory:\*\*



**\*\*Expected Optimization Behavior:\*\***

...

**\*\*Expected Security Behavior:\*\***

...

**\*\*Expected Quality Behavior:\*\***

...

**\*\*Expected Cost Behavior:\*\***

...

**\*\*Expected Latency Behavior:\*\***

...

**\*\*Expected State Transition:\*\***

...

**\*\*Expected Recovery:\*\***

...

**\*\*Side-Effect Requirements:\*\***

...

**\*\*Idempotency Requirement:\*\***

...

**\*\*Audit/Observability Requirement:\*\***

...

**\*\*Failure Classification:\*\***

...

**\*\*Acceptance Criteria:\*\***

...

**\*\*Traceability:\*\***

- Problem Statement: ...
- Engineering Specification: ...
- Architecture: ...
- Interfaces: ...

- Conventions: ...
- Edge Cases: ...

Do not populate fields with meaningless placeholders.

If a field genuinely does not apply, write:

N/A – not applicable to this scenario.

---

## 39. TRACEABILITY REQUIREMENT

Every scenario must trace back to one or more source documents.

Prefer exact:

- section
- heading
- requirement identifier
- interface
- architecture component
- convention
- edge-case identifier

when available.

Do NOT invent requirement IDs that do not exist.

If the source documents lack stable IDs, reference the exact document and section heading.

---

## 40. SOURCE GAP IDENTIFICATION

If the scenario-generation exercise identifies a behavior that is necessary for a robust enterprise Control Plane but is NOT represented in the six source documents:

DO NOT silently add it as an established requirement.

Mark it:

SOURCE-GAP

and create a separate section:

# SOURCE GAPS DISCOVERED

For every gap record:

- Gap ID
- Scenario(s) affected
- Missing requirement
- Why it matters
- Which source document should eventually be updated
- Recommended disposition

The scenario matrix must distinguish:

SOURCE-SUPPORTED

from:

SOURCE-GAP

---

## 41. CONTRADICTION DETECTION

If source documents contradict each other:

DO NOT silently resolve the contradiction.

Create:

## SOURCE CONTRADICTIONS

Record:

- contradiction ID
- documents involved
- conflicting statements
- affected scenarios
- recommended resolution

The scenario may be marked:

BLOCKED-BY-CONTRADICTION

where appropriate.

---

## 42. COVERAGE MATRIX

After the scenarios, create a coverage summary.

Include:

| Domain | Positive | Negative | Boundary | Failure | Recovery | Compound |
|--------|----------|----------|----------|---------|----------|----------|
|--------|----------|----------|----------|---------|----------|----------|

Also provide coverage for:

- context
- agents
- coding agents
- security
- model/provider
- workflow
- permissions
- cache
- memory
- recovery
- concurrency
- cost
- quality
- latency

## 43. CROSS-DOCUMENT COVERAGE MATRIX

Create:

| Source Document   | Scenarios Referencing It | Coverage |
|-------------------|--------------------------|----------|
| Problem Statement | ...                      | ...      |
| Engineering Spec  | ...                      | ...      |
| Architecture      | ...                      | ...      |
| Interfaces        | ...                      | ...      |
| Conventions       | ...                      | ...      |

Identify areas where a source document has little or no scenario coverage.

---

## 44. REQUIREMENT → SCENARIO TRACEABILITY

Create a reverse traceability section:

Requirement / Section

↓

Scenario IDs

↓

Expected Behavior

↓

Validation

This should allow an engineer to start from a requirement and identify the scenarios that validate it.

---

## 45. SCENARIO → ARCHITECTURE TRACEABILITY

Where practical, map scenarios to architecture components such as:

- Execution State Manager
- Context Version Manager
- Workflow Version Manager
- Checkpoint Manager
- Reconciliation Engine
- Context Integrity Gate
- Context Expansion Controller
- Permission Revalidation
- Dynamic Policy Evaluation
- Capability / Availability Resolver
- Stale Result Protection

- Supersession Manager
- Recovery Coordinator
- optimization components
- cache components
- provider abstraction
- agent integration layer

Use the actual terminology from the architecture document.

Do not invent components.

---

## 46. SCENARIO PRIORITY

Assign:

- P0 — catastrophic/security/correctness
- P1 — critical enterprise behavior
- P2 — important behavior
- P3 — lower-priority behavior

P0 must include scenarios involving:

- unauthorized access
  - cross-tenant leakage
  - policy bypass
  - destructive replay
  - corrupted recovery
  - mandatory context loss
  - severe security failures
- 

## 47. AUTOMATION CANDIDATE

Mark whether a scenario should eventually become:

- unit test
- integration test
- contract test
- end-to-end test
- chaos test
- security test



- performance test
- load test
- evaluation
- manual validation

Do not write implementation code.

---

## 48. DETERMINISM

For every scenario, define the expected invariant.

Example:

Given:

Permission revoked before tool execution

Then:

Tool execution MUST NOT occur

Avoid acceptance criteria such as:

"System should behave correctly."

Use testable statements.

---

## 49. INVARIANTS

Create a dedicated section:

### CONTROL PLANE INVARIANTS

Include only invariants supported by the source documents.

Examples of the type of invariant to capture:

- unauthorized context must never be admitted
- revoked permission must not authorize new operations
- optimization cannot override policy
- quality cannot be sacrificed solely for token savings
- stale state must not silently become authoritative
- inaccessible models cannot be selected
- non-idempotent operations must not be blindly replayed

- superseded executions must not continue unauthorized side effects
- logical task context must not be confused with model-admitted context
- optimization failure must not automatically become task failure unless required by policy/safety

Use source terminology and verify each invariant against the documents.

---

## 50. FAILURE TAXONOMY

Create a failure classification covering, where supported:

- user error
- validation failure
- authorization failure
- policy failure
- context failure
- optimization failure
- model failure
- provider failure
- tool failure
- memory failure
- cache failure
- execution failure
- infrastructure failure
- state conflict
- stale state
- timeout
- budget failure
- quality failure
- security failure
- unrecoverable failure

Do not invent categories merely for completeness if unsupported.

---

## 51. RECOVERY TAXONOMY

Where applicable, classify recovery as:

- retry
  - retry with same state
  - retry after reconciliation
  - recompute
  - retrieve missing context
  - expand context
  - reduce context
  - switch model
  - switch provider
  - invalidate cache
  - restore checkpoint
  - rollback
  - restart
  - pause
  - ask user
  - abort safely
- 

## 52. NO BLIND FALLBACK

Do not assume every failure should trigger fallback.

For each relevant scenario distinguish:

Fallback Allowed

Fallback Not Allowed

Fallback Requires Reconciliation

Fallback Requires User Approval

Fallback Violates Policy

Fallback Would Reduce Quality

Fallback Would Increase Cost Excessively

---

## 53. NO SILENT DATA LOSS

Explicitly identify scenarios where optimization could cause:

- lost requirements
- lost constraints
- lost context

- lost provenance
- lost permissions
- lost policy state
- lost workflow state
- lost recovery state

The expected behavior must prevent silent data loss.

---

## **54. NO SILENT POLICY BYPASS**

Any optimization path must be subject to the same:

- authorization
- policy
- tenant isolation
- security
- compliance

requirements as the non-optimized path.

---

## **55. NO SILENT CONTEXT FABRICATION**

When required context is unavailable:

The system must not fabricate it.

The scenario must explicitly define whether the expected response is:

- retrieve
  - ask user
  - defer
  - fail safely
- 

## **56. SCENARIO DEDUPLICATION**

Before finalizing:

- remove duplicate scenarios

- merge semantically identical scenarios
- retain distinct boundary conditions
- retain meaningful compound interactions

Do not inflate the scenario count with superficial wording changes.

---

## 57. COMPLETENESS REVIEW

Before declaring the document complete, perform a systematic review across:

### Lifecycle

Create

→ Admit

→ Optimize

→ Execute

→ Mutate

→ Reconcile

→ Continue

→ Pause

→ Resume

→ Complete

and failure paths:

Fail

→ Detect

→ Classify

→ Recover

→ Retry/Recompute/Fallback

→ Reconcile

→ Resume/Abort

---

## 58. FINAL SCENARIO COVERAGE TARGET

The final document should contain at least:

**200 meaningful scenarios**

with substantial coverage of:

- positive
- negative
- boundary
- dynamic mutation
- security
- recovery
- coding agents
- context overflow
- provider/model failures
- cache/memory
- workflow
- permission
- policy
- compound scenarios

If 200 scenarios are insufficient for complete coverage, generate more.

Completeness is more important than the number 200.

---

## 59. FINAL REPORT

At the end of `scenario-matrix.md`, include:

### Scenario Count

Total number of scenarios.

### Domain Coverage

Counts by domain.

### Positive / Negative / Boundary / Failure / Recovery / Compound Distribution

Provide counts.

### P0 / P1 / P2 / P3 Distribution

Provide counts.

## Source Coverage

How many scenarios trace to each of the six documents.

## Architecture Coverage

Which architecture components are exercised.

## Interface Coverage

Which interfaces/contracts are exercised.

## Source Gaps

List all `SOURCE-GAP` findings.

## Source Contradictions

List all contradictions.

## Highest-Risk Scenarios

List P0 scenarios.

## Recovery-Critical Scenarios

List scenarios requiring checkpoint/recovery semantics.

## Security-Critical Scenarios

List security-critical scenarios.

## Coding-Agent-Critical Scenarios

List the highest-risk coding-agent scenarios.

## Context-Critical Scenarios

List the highest-risk context scenarios.

---

## 60. FINAL READINESS ASSESSMENT

End with:

## SCENARIO MATRIX READINESS

Choose exactly one:

READY FOR NEXT DOCUMENTATION PHASE

or

NOT READY — BASELINE REQUIREMENTS MUST BE REVISED

If NOT READY:

List exactly which source requirements must be revised.

Do not modify those source documents automatically.

---

## FINAL INSTRUCTION

The goal is NOT to produce a pretty table.

The goal is to create a rigorous scenario-validation model for an enterprise-grade:

### **Agent & LLM Inference Optimization Control Plane**

that remains correct under:

- normal execution
- context overflow
- context absence
- context mutation
- workflow mutation
- permission mutation
- policy mutation
- model/provider changes
- coding-agent behavior
- tool/MCP changes
- cache/memory changes
- long waits
- interruptions
- recovery
- supersession
- concurrency
- security threats
- quality degradation



- optimization failure
- compound failures

Do not silently invent requirements.

Do not remove source requirements.

Do not modify the six source documents.

Create only `scenario-matrix.md`.

## One important point

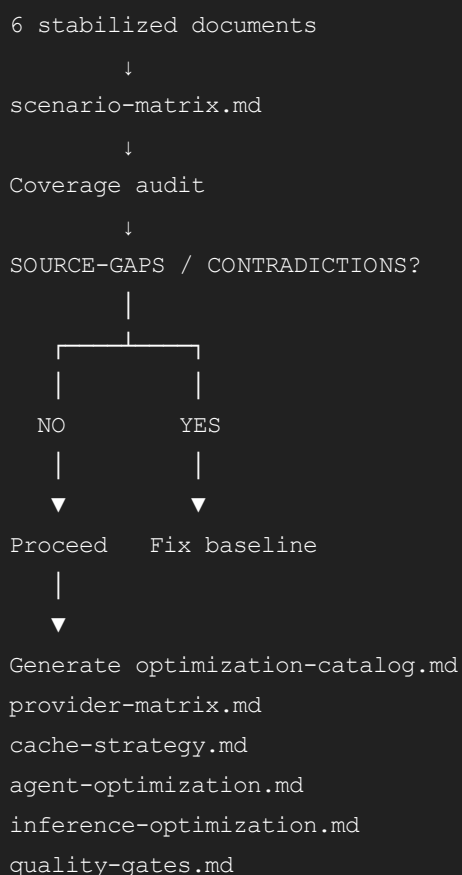
When Antigravity finishes, **don't immediately accept 200+ as proof of completeness.**

The more important output will be the combination of:

**scenario count + domain coverage + source traceability + source gaps + contradictions + P0/P1 coverage + compound scenarios.**

If it produces 240 scenarios but 80% are trivial variations, that's worse than 180 genuinely meaningful scenarios.

The ideal next checkpoint will therefore be:



```
security.md
observability.md
eval.md
SCALING.md
implementation-plan.md
ADRs
```

**That scenario matrix should become the gatekeeper for everything you generate after it.** This will substantially reduce the risk of later documents independently making assumptions that aren't actually supported by your core Control Plane design.

ChatGPT can make mistakes. Check important info.